

หัวข้อโครงการ	ระบบเอนจินเกมมาสเตอร์ไฮบริดแบบหลายเอเจนต์สำหรับเทเบิลท็อป RPG
หน่วยกิต	6
ผู้เขียน	นายภนลภัส สุทธิมาลา
อาจารย์ที่ปรึกษา	ผศ.ดร. สุริยา นัฏสุภักพงษ์ ดร. ไพสิฐ ชันอาสา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
คณะ	สถาบันวิทยาการหุ่นยนต์ภาคสนาม
ปีการศึกษา	2568

บทคัดย่อ

โครงการนี้พัฒนา สถาปัตยกรรมไฮบริดแบบหลายเอเจนต์ เพื่อแก้ปัญหา "การหลอนของปัญญาประดิษฐ์" ผ่านการแยกส่วนตรรกะเชิงกลไกออกจากความคิดสร้างสรรค์อย่างเด็ดขาด ระบบใช้กลไก ตัวจัดการแบบสองเส้นทาง (Two-Path Orchestrator) เพื่อคัดกรองคำสั่งระหว่าง เอนจินกฎเกณฑ์เชิงกำหนด (Path A) สำหรับกลไกเกมมาตรฐาน และ เอเจนต์ผู้ตัดสินใจเชิงตรรกะ (Path B) สำหรับการประมวลผลเหตุการณ์ที่ยืดหยุ่น

โครงสร้างระบบถูกออกแบบเป็น เครื่องจักรสถานะเชิงตรรกะ (Stateless Logic Machine) ที่ใช้โปรโตคอล TOON Serialization ในการประสานงานระหว่างเอเจนต์ให้มีประสิทธิภาพสูงสุด ระบบยังประกอบด้วยหน่วยความจำรีเบทแบบ $O(1)$ Sliding Window เพื่อรักษาความต่อเนื่องของเนื้อเรื่อง และกลไก Static RAG สำหรับการดึงข้อมูลภูมิหลังของโลกเกมโดยเฉพาะ สถาปัตยกรรมนี้ทำให้เอนจินมีสถานะเป็นระบบหลังบ้านที่มีเสถียรภาพสูง สามารถรักษาสมดุลระหว่างความถูกต้องทางคณิตศาสตร์และการเล่าเรื่องเชิงสร้างสรรค์ได้อย่างสมบูรณ์ โครงการนี้จึงมุ่งเน้นการสร้าง ระบบต้นแบบแอปพลิเคชัน (Functional Prototype) ที่รองรับวงจรการเล่นเกมที่สมบูรณ์ (Complete Game Loop) ตั้งแต่การสำรวจไปจนถึงการต่อสู้ โดยให้ความสำคัญกับการพัฒนา ระบบการต่อสู้ (Combat System) เป็นแกนกลางหลัก (Core Engine) เนื่องจากเป็นส่วนที่มีความซับซ้อนเชิงตรรกะสูงสุดและมักเกิดการหลอนของข้อมูลได้ง่ายที่สุด เพื่อให้มั่นใจว่าผลิตภัณฑ์ที่พัฒนาขึ้นจะมอบประสบการณ์การเล่นที่ยุติธรรมและเสถียรเทียบเท่ากับด้นเจียนมาสเตอร์ที่เป็นมนุษย์

คำสำคัญ : ระบบหลายเอเจนต์ / สถาปัตยกรรมไฮบริด / เอนจินเชิงกำหนด / การจัดการหน่วยความจำ ($O(1)$) / Static RAG / การจัดการระบบ (Orchestration)

Project Title	Multi-Agent Hybrid Game Master Engine for TTRPGs
Project Credits	6
Candidate	Mr. Pollapaat Suttimala
Project Advisor	Asst.Prof.Dr. Suriya Natsupakpong Dr. Paisit Khanarsa
Program	Bachelor of Engineering
Field of Study	Robotics and Automation Engineering
Faculty	Institute of Field Robotics
Academic Year	2025

Abstract

This project develops a Multi-Agent Hybrid Engine for TTRPGs, engineered to solve "AI Hallucination" by decoupling creative narration from deterministic logic. The architecture features a Two-Path Orchestrator that autonomously routes player intents between a Deterministic Python Rules Engine (Path A) for rule-heavy mechanics and an LLM-based Creative Arbiter (Path B) for improvisational judgment.

The system is built as a robust Stateless Logic Machine utilizing a custom TOON Serialization pipeline for high-efficiency coordination between specialized agents. To maintain narrative continuity, the engine incorporates an $O(1)$ sliding window buffer for context management and a Static RAG mechanism to inject grounded world-building lore directly into the narrative layer. This architecture provides a scalable, high-fidelity backend capable of synchronizing complex game states with generative narration, ensuring every action is mechanically verified before being narratively described.

Keywords : Multi-Agent Systems / Hybrid Architecture / Deterministic Logic / Contextual Memory ($O(1)$) / Static RAG / System Orchestration

บทที่ 1 บทนำ

1.1 ความสำคัญและที่มาของงานวิจัย

ในช่วงไม่กี่ปีที่ผ่านมา เกมสวมบทบาทบนโต๊ะ (Tabletop Role-Playing Game หรือ Tabletop RPG) โดยเฉพาะ Dungeons & Dragons (D&D) ได้รับความนิยมเพิ่มขึ้นอย่างกว้างขวางผ่านอิทธิพลของสื่อกระแสหลัก เช่น ซีรีส์ Stranger Things หรือการถ่ายทอดสดการเล่นผ่านอินเทอร์เน็ต (Actual Play) โดยผู้มีอิทธิพลทางความคิด เช่น 9Arm เป็นต้น กระแสเหล่านี้ดึงดูดกลุ่มผู้เล่นหน้าใหม่และกลุ่มผู้เล่นทั่วไป (Casual Player) ที่ต้องการสัมผัสประสบการณ์การเล่นเรื่องและการผจญภัยในโลกจินตนาการ แต่การจะเริ่มเล่นนั้นมีอุปสรรคสำคัญ (Entry Barrier) คือความต้องการ "ดันเจี้ยนมาสเตอร์" (Dungeon Master หรือ DM) ที่มีทักษะสูงในการควบคุมกฎกติกาที่ซับซ้อนและการเตรียมเนื้อเรื่องที่ใช้เวลานาน

อุปสรรคดังกล่าวนำไปสู่การเติบโตของความต้องการเล่นแบบ "Solo TTRPG" หรือการเล่นคนเดียว ซึ่งผู้เล่นมักหันไปพึ่งพาเครื่องมือปัญญาประดิษฐ์ (AI) อย่างโมเดลภาษาขนาดใหญ่ (Large Language Models - LLM) เช่น ChatGPT หรือ AI Dungeon อย่างไรก็ตาม ปัญหาสำคัญที่พบคือ "การหลอนของปัญญาประดิษฐ์" (AI Hallucination) เนื่องจาก LLM มีพื้นฐานการทำงานเชิงความน่าจะเป็น (Probabilistic) ทำให้มักเกิดข้อผิดพลาดในการคำนวณตัวเลข บิดเบือนกฎกติกา หรือลืมสถานะสำคัญของเกม (เช่น ค่าพลังชีวิต หรือไอเทม) ซึ่งทำลายความสมดุลและความสมจริง (Immersion) ของการเล่นลงอย่างสิ้นเชิง สำหรับผู้เล่นที่ต้องการ "เกม" ที่มีความยุติธรรมและปฏิบัติตามกฎอย่างเคร่งครัด AI ในปัจจุบันจึงยังไม่สามารถทำหน้าที่เป็น DM ที่ไว้ใจได้

โครงการนี้จึงมีเป้าหมายเพื่อพัฒนา สถาปัตยกรรมเอนจินเกมมาสเตอร์ไฮบริดแบบหลายเอเจนต์ (Multi-Agent Hybrid Game Master Engine) เพื่อตอบโจทย์ผู้เล่นกลุ่ม Solo และ Casual โดยเฉพาะ โดยเปลี่ยนจากการใช้ AI เพียงอย่างเดียว มาเป็นการใช้สถาปัตยกรรมแบบ "Hybrid-Arbitrated" ที่แยกส่วนการเล่นเรื่อง (Narrative) ออกจากตรรกะเชิงกลไก (Logic) อย่างเด็ดขาด หัวใจหลักของระบบคือการประสานงานระหว่างเอเจนต์ (Multi-Agent Handshake) โดยใช้ เอนจินกฎเกณฑ์เชิงกำหนด (Deterministic Rules Engine) ที่เขียนด้วยภาษา Python เพื่อควบคุมกฎ "Lite 5e" และการจัดการสถานะเกมให้มีความแม่นยำ 100% ในขณะที่ LLM จะถูกจำกัดบทบาทให้ดูแลเพียงส่วนการตัดสินใจเชิงสร้างสรรค์และการบรรยายเนื้อเรื่องเท่านั้น

การพัฒนาไม่เพียงแต่ช่วยลดภาระในการอ่านกฎเกณฑ์ที่ซับซ้อนกว่า 300 หน้าสำหรับผู้เล่นใหม่ แต่ยังมอบประสบการณ์การเล่นที่ "ยุติธรรม" และ "คงเส้นคงวา" ผ่านระบบหลังบ้าน (Backend) ที่มีความเสถียรสูง ระบบจะทำหน้าที่เป็นผู้จัดการการเล่นอัตโนมัติที่ช่วยให้ผู้เล่นสามารถกระโดดเข้าสู่การผจญภัยได้ทันที โดยไม่ต้องกังวลเรื่องการค้าบนทางคณิตศาสตร์หรือการจัดการสถานะที่ยุงยาก แต่ยังคงได้รับอรรถรสความสนุกและความท้าทายตามมาตรฐานของเกม Tabletop RPG ไว้อย่างครบถ้วน

1.2 วัตถุประสงค์ของงานวิจัย

1. เพื่อพัฒนาเอนจินเกมมาสเตอร์อัตโนมัติที่สามารถ รักษากฎเกณฑ์ให้มีความแม่นยำ (Deterministic) โดยใช้สถาปัตยกรรมแบบไฮบริดเพื่อแยกส่วนการคำนวณออกจากส่วนการเล่าเรื่อง
2. เพื่อออกแบบกลไก การประสานงานระหว่างหลายเอเจนต์ (Multi-Agent Orchestration) ให้สามารถตัดสินใจและจัดการสถานะเกมได้อย่างสอดคล้องกันโดยปราศจากการหลอนของข้อมูล (Hallucination)
3. เพื่อพัฒนาระบบ การสื่อสารและจัดการหน่วยความจำที่มีประสิทธิภาพสูง เพื่อลดภาระการประมวลผลและเพิ่มความเร็วในการตอบสนองของระบบ
4. เพื่อสร้างระบบต้นแบบที่รองรับการเล่นแบบ Solo/Casual ที่ผู้เล่นสามารถเข้าถึงกฎกติกาที่ซับซ้อนได้ง่ายผ่านการโต้ตอบที่สั้นไหลและเชื่อถือได้

1.3 ประโยชน์และผลที่คาดว่าจะได้รับจากงานวิจัย

1. ได้รับต้นแบบเอนจินเกมมาสเตอร์ไฮบริด (Hybrid Engine) ที่สามารถจัดการการเล่นเทเบิลท็อป RPG ได้อย่างสมบูรณ์ ช่วยลดภาระในการอ่านกฎกติกาที่ซับซ้อนและลดเวลาในการเตรียมการสำหรับผู้เล่นกลุ่มแคชชวล
2. ได้รับแนวทางการออกแบบ สถาปัตยกรรมแบบหลายเอเจนต์ (Multi-Agent Architecture) ที่สามารถแยกส่วนการเล่าเรื่องเชิงสร้างสรรค์ออกจากตรรกะเชิงกลไกได้อย่างเด็ดขาด เพื่อเป็นต้นแบบในการแก้ปัญหาการหลอนของข้อมูล (AI Hallucination) ในระบบ AI อื่นๆ
3. ได้รับระบบจัดการสถานะเกมที่มีความน่าเชื่อถือ ซึ่งช่วยสร้างประสบการณ์การเล่นที่ยุติธรรมและคงเส้นคงวาสำหรับผู้เล่นคนเดียว (Solo Player) โดยไม่ต้องพึ่งพาต้นเขียนมาสเตอร์ที่เป็นมนุษย์
4. ได้รับองค์ความรู้เชิงวิศวกรรมในการเพิ่มประสิทธิภาพการจัดการหน่วยความจำบริบท (Context Management) และการสื่อสารข้อมูลระหว่างเอเจนต์ที่มีทรัพยากรจำกัด ซึ่งสามารถนำไปต่อยอดสู่ระบบการเล่นหลายคน (Multiplayer) หรือการรองรับเนื้อเรื่องที่ซับซ้อนมากขึ้นในอนาคต

1.4 ขอบเขตงานวิจัย

1.4.1 ด้านระบบเกมและกติกา

1. ระบบรองรับการเล่น Tabletop RPG โดยใช้ชุดกติกาแบบ "Lite 5e" ซึ่งเป็นกฎที่ผู้จัดทำได้นิยาม และกำหนดขอบเขตขึ้นเพื่อลดความซับซ้อนเชิงกลไก (ดูรายละเอียดในภาคผนวก ก) โดยเน้นกลไกหลักที่จำเป็น อาทิ การตรวจสอบค่าความสามารถ (d20 check), ระบบการต่อสู้, ค่าพลังชีวิต (HP) และการเคลื่อนที่แบบโซน (Zone-based Movement) การลดทอนรายละเอียดของกฎมีวัตถุประสงค์เพื่อแก้ปัญหาการแทรกกฎของโมเดลภาษาขนาดใหญ่ (LLM-only Hallucination) และลดภาระในการประมวลผลของเอเจนต์

Feature	Standard D&D 5e	Our "Lite 5e"
Rulebook	300+ หน้า (Player's Handbook)	~3-4 หน้า (ในภาคผนวก ก)
Stats (ค่าสถานะ)	6 ค่า (STR, DEX, CON, INT, WIS, CHA)	3 ค่า (PHYS, MENT, SOC)
Map (แผนที่)	ตาราง Grid ชับซ้อน (5ft. squares)	"Zone-based" (ใกล้, กลาง, ไกล)
Skills (ทักษะ)	18 ทักษะ (เช่น Athletics, Stealth, Arcana)	รวมอยู่ใน 3 ค่าสถานะหลัก (เช่น ปีนป่าย = PHYS, สังเกต = MENT)

ตารางที่ X การเปรียบเทียบขอบเขต D&D 5e มาตรฐาน และ Lite 5e

2. ระบบรองรับการเล่นเนื้อเรื่องแบบจบในตัว (One-shot Session) โดยมีระบบบันทึกสถานะเกม (State Persistence) ผ่านไฟล์ JSON เพื่อความต่อเนื่องในการเล่น
3. รูปแบบการเล่นและการตัดสินผล ระบบนี้ใช้สถาปัตยกรรมแบบ 'Hybrid-Arbitrated' ซึ่งแบ่งการประมวลผลคำสั่งของผู้เล่นเป็น 2 ประเภทหลัก เพื่อให้มีความยืดหยุ่นเหมือนเล่นกับมนุษย์ แต่ยังคงความแม่นยำของกฎไว้
- 1) การกระทำแบบมีกฎตายตัว (Fixed Actions): คือการกระทำที่มีกลไกชัดเจนใน Lite 5e Rulebook (ภาคผนวก ก) เช่น 'การโจมตี' (Attacking), 'การเคลื่อนที่' (Movement), หรือ 'การใช้ความสามารถ' (Use Ability) คำสั่งเหล่านี้จะถูก Rules Engine (ส่วนที่เป็นโค้ด) ประมวลผลก่อน เพื่อคำนวณผลลัพธ์ (เช่น Hit/Miss, Damage) จากนั้น LLM จะทำหน้าที่บรรยายผลลัพธ์ ที่คำนวณได้
- 2) การกระทำเชิงสร้างสรรค์ (Creative Actions): คือการกระทำที่ยื่นนอกเหนือกฎตายตัวที่ระบุไว้ชัดเจน เช่น 'การสำรวจสภาพแวดล้อม' (Exploration), 'การไขปริศนา' (Puzzles), หรือการกระทำที่ผู้เล่นคิดขึ้นเอง (เช่น "เผาหีบเพื่อละลายกัญเจ") คำสั่งเหล่านี้จะถูกส่งให้ LLM (ในฐานะ DM) ตีความและทำการตัดสินใจก่อน (Arbitration) เช่น กำหนดค่าความยาก (DC) หรือระบุค่าสถานะ (PHYS, MENT, SOC) ที่ต้องใช้ในการทอยลูกเต๋า จากนั้น Rules Engine จึงจะทำการทอยลูกเต๋าตามที่ LLM กำหนด และ LLM จะบรรยายผลลัพธ์ ตามผลการทอยนั้น

4. ขอบเขตการทำงานของต้นแบบ (Prototype Scope): เพื่อให้สามารถพัฒนาได้ทันตามกำหนดเวลา และตอบสนองต่อข้อเสนอแนะในการลดขอบเขต โครงการนี้จะมุ่งเน้นการพัฒนาตามขอบเขตดังตารางต่อไปนี้

In-Scope (สิ่งที่ทำในโครงการนี้)	Out-of-Scope (สิ่งที่ยังไม่ทำในโครงการนี้)
1. Multi-Agent Backend Engine: การจัดการระบบหลังบ้านแบบแยกส่วนงาน (Router, Rules Engine, Arbiter, Narrator)	1. Graphical User Interface (GUI): ไม่มีการพัฒนากราฟิก 3D หรือ UI ที่ซับซ้อนบน Game Engine (เน้น CLI)
2. Deterministic Logic: ระบบการต่อสู้และเคลื่อนที่ตามกฎ Lite 5e ที่ผ่านการคำนวณด้วยโค้ดโดยปราศจากการหลอนของ AI	2. ASR & TTS Implementation: ระบบไม่รวมการสั่งงานด้วยเสียงและการแปลงข้อความเป็นเสียง (ตัดออกเพื่อเน้นความเสถียรของระบบหลังบ้าน)
3. State Synchronization: การรักษาสมดุลข้อมูลผ่านโปรโตคอล TOON Serialization และหน่วยความจำแบบ O(1)	3. Multiplayer Networking: ไม่รองรับการเล่นหลายคนผ่านระบบออนไลน์ (รองรับเฉพาะ Local Solo Play)
4. Creative Arbitration: การตัดสินใจผลการสำรวจและปริศนาผ่านเอนจินต์ที่สามารถกำหนดค่าความยาก (DC) ได้อย่างเหมาะสมผล	4. Complex Branching: เนื้อเรื่องที่แตกแขนงมหาศาลหรือระบบความสัมพันธ์ NPC ที่ซับซ้อนเกินระดับเลเวล 3

ตารางที่ X ขอบเขตการทำงานของต้นแบบ

1.4.2 ด้านการปฏิสัมพันธ์กับผู้ใช้

- รองรับการป้อนคำสั่งผ่าน ข้อความ (Text Input) ในรูปแบบคำสั่งภาษามนุษย์ (Natural Language) ผ่านทางหน้าจอ Command-Line Interface (CLI) โดยไม่มีการรองรับการสั่งงานด้วยเสียง (ASR) เพื่อมุ่งเน้นความเสถียรของการตีความเจตนา (Intent Classification)
- ส่วนแสดงผลแผงควบคุม (Immersive CLI Dashboard) ออกแบบมาเพื่อแสดงสถานะเกมแบบ Real-time บนหน้าจอเดียว โดยมีองค์ประกอบหลักดังนี้
 - Status Display แสดงค่าพลังชีวิต (HP) และสถานะความพร้อมของตัวละครผ่านตัวอักษรและสัญลักษณ์ (Visual Health Descriptors)
 - Tactical Zone Visualization แสดงแผนที่การต่อสู้ในรูปแบบโซน (NEAR, MID, FAR) โดยใช้สัญลักษณ์ ASCII เพื่อให้ผู้เล่นวางแผนกลยุทธ์ได้โดยไม่ต้องพึ่งพากราฟิกซับซ้อน

3) Integrated Action Log แสดงประวัติการกระทำและผลลัพธ์จากการคำนวณของ Rules Engine (เช่น ผลการทอยลูกเต๋า) ควบคู่ไปกับคำบรรยายเนื้อเรื่องจาก LLM

3. การจัดการหน่วยความจำและสถานะเกม (Memory & Persistence)

1) Full Transaction Logging ระบบจะบันทึกประวัติการเล่นและเหตุการณ์ทั้งหมด (Full Log) ลงในไฟล์ภายนอกอย่างต่อเนื่อง เพื่อรองรับระบบการบันทึกและโหลดข้อมูล (Save/Load) ให้ผู้เล่นสามารถกลับมาเล่นต่อจากจุดเดิมได้

2) Contextual Action Summarization: ระบบหน่วยความจำภายในจะทำการสรุปย่อเหตุการณ์สำคัญ (Action Summarization) เพื่อใช้เป็นบริบทในหน่วยความจำระยะสั้น ช่วยให้ AI สามารถจดจำเหตุการณ์สำคัญที่ผ่านมาได้โดยไม่ทำให้ระบบเกิดความล่าช้าหรือใช้ทรัพยากรเกินจำเป็น

1.4.3 ด้านเทคนิคและการวัดผล

1. ด้านเทคนิคและข้อกำหนดของระบบ

- 1) ระบบให้บริการ Large Language Model (LLM) ผ่าน API เป็นแกนกลางในการตัดสินใจและเล่าเรื่อง โดยเน้นการประมวลผลหลังบ้านแบบ Stateless Symbolic Machine
- 2) ระบบให้ความสำคัญกับการจัดการทรัพยากรและการสื่อสารระหว่างเอนเจิ้นผ่านโปรโตคอล TOON Serialization เพื่อลดปริมาณ Token และ Latency
- 3) ส่วนต่อประสานผู้ใช้ถูกจำกัดอยู่ที่ Immersive CLI Dashboard โดยไม่มีการพัฒนากราฟิกสามมิติหรือแอนิเมชัน เพื่อมุ่งเน้นความเสถียรของตรรกะระบบ (System Logic)

2. เกณฑ์การวัดผลสำเร็จ (Quantitative Metrics) ความสำเร็จของต้นแบบจะถูกวัดผลผ่านชุดทดสอบมาตรฐานจำนวน 50 สถานการณ์ (50-Scenario Functional Test Suite) ซึ่งครอบคลุม 24 Logic Nodes ของสถาปัตยกรรมระบบ โดยใช้ดัชนีชี้วัดดังนี้

- 1) ความแม่นยำในการคัดกรอง (A_route) วัดความสามารถของ Intent Router ในการแยกแยะคำสั่งระหว่าง Path A และ Path B ได้อย่างถูกต้อง
- 2) ความแม่นยำในการระบุข้อมูลอ้างอิง (P_ground) วัดความถูกต้องของเอนเจิ้นผู้ตัดสิน (Arbiter) ในการกำหนดค่าสถานะ (Stats), ค่าความยาก (DC) และสถานะผิดปกติ (Conditions) ให้สอดคล้องกับเจตนาของผู้เล่น
- 3) ความสมบูรณ์ของการประสานสถานะ (S_sync) วัดความสามารถของระบบในการปรับปรุงข้อมูลในไฟล์สถานะ (JSON State) ให้ตรงกับผลลัพธ์ทางคณิตศาสตร์ที่ Rules Engine คำนวณได้
- 4) ความคงเส้นคงวาของการเล่าเรื่อง (C_narr): วัดความสอดคล้องระหว่างเนื้อเรื่องจาก Narrator กับผลลัพธ์ทางตรรกะที่เกิดขึ้นจริง

3. เนื่องจากโครงงานอยู่ในระยะการพัฒนาค้นแบบแกนกลาง (Core Engine) การตรวจสอบในขั้นนี้จะมุ่งเน้นไปที่การทำงานของ สถาปัตยกรรมแบบสองเส้นทาง (Two-Path Architecture) ผ่านชุดทดสอบ 50 สถานการณ์จำลอง ซึ่งครอบคลุมหัวข้อหลักดังนี้
 - 1) การคัดกรองเจตนา (Intent Routing Accuracy) ตรวจสอบความสามารถของระบบในการแยกแยะคำสั่งประเภทกฎตายตัว (Fixed) และคำสั่งเชิงสร้างสรรค์ (Creative) เพื่อส่งไปยังเส้นทางการประมวลผลที่ถูกต้อง
 - 2) ตรรกะเชิงกำหนด (Deterministic Logic - Path A) ตรวจสอบความถูกต้องแม่นยำทางคณิตศาสตร์ตามกฎ Lite 5e ในด้านการโจมตีระยะประชิดและระยะไกล, การคำนวณระยะ (Zone), และการใช้ไอเทมตามเงื่อนไขที่กำหนด
 - 3) การตัดสินใจเชิงสร้างสรรค์ (Creative Arbitration - Path B) ตรวจสอบความสมเหตุสมผลของเอเจนต์ผู้ตัดสินใจในการกำหนดค่าความยาก (DC), การเลือกใช้ค่าสถานะ (Stats) และการส่งค่าสถานะเชิงสัญลักษณ์ (Symbolic Conditions) กลับเข้าสู่ระบบ
 - 4) ระบบป้องกันการละเมิดกฎ (Action Economy Guardrails) ตรวจสอบความสามารถในการตรวจจับและปฏิเสธคำสั่งที่ผิดกฎการเล่น (เช่น การกระทำซ้ำซ้อนในหนึ่งเทิร์น) และคำสั่งที่เป็นไปไม่ได้ (Impossible Actions)
 - 5) ระบบบริหารจัดการสถานะและหน่วยความจำ (System Orchestration) ตรวจสอบความถูกต้องของการจัดลำดับเทิร์น (Initiative Queue), พฤติกรรมการตัดสินใจของศัตรู (Enemy AI), การโอนย้ายข้อมูลไอเทม (Auto-Loot) และการย่อสรุปหน่วยความจำบริบท (Summarization)

1.5 ข้อตกลงเบื้องต้น

1. โครงงานนี้มุ่งเน้นการพัฒนา สถาปัตยกรรมระบบประสานงาน (Orchestration Architecture) โดยเลือกใช้บริการ Large Language Model (LLM) ผ่าน API จากผู้ให้บริการภายนอก (เช่น Gemini 2.5 Flash Lite) เป็นแกนกลางในการประมวลผลภาษา โดยผู้จัดทำไม่ได้มุ่งเน้นการพัฒนาหรือเทรนโมเดลภาษาขึ้นใหม่เอง
2. ผู้ใช้งานระบบจำเป็นต้องมี API Key สำหรับบริการ LLM และเป็นผู้รับผิดชอบค่าใช้จ่ายที่เกิดขึ้นจากการใช้งาน (ถ้ามี) โดยระบบมีกลไกการตั้งค่าอัตโนมัติ (Automated Configuration Wizard) ที่จะช่วยตรวจสอบและสร้างไฟล์สภาพแวดล้อม (.env) ให้ผ่านหน้าจอคำสั่ง (CLI) ในกรณีที่ยังไม่มี การติดตั้งกฎแฉรหัส
3. ค้นแบบของระบบถูกออกแบบมาเพื่อ ผู้เล่นคนเดียว (Solo-player experience) โดยที่ผู้เล่นหนึ่งคนทำหน้าที่ควบคุมกลุ่มตัวละคร ผ่านส่วนต่อประสานผู้ใช้เชิงคำสั่ง (CLI Dashboard) เพื่อให้ระบบสามารถจัดการสถานะเกม (Game State) ได้อย่างมีประสิทธิภาพสูงสุด แม้ระบบจะถูกออกแบบมาให้

รองรับการดำเนินเนื้อเรื่องที่หลากหลาย (Narrative & Exploration) แต่ในการทวนสอบความสำเร็จของต้นแบบในขั้นนี้ จะมุ่งเน้นการทดสอบประสิทธิภาพผ่านระบบการต่อสู้เป็นหลัก เนื่องจากเป็น 'Stress Test' ที่ดีที่สุดในการพิสูจน์ความเสถียรของสถาปัตยกรรม Two-Path หากระบบสามารถจัดการตรรกะการต่อสู้ที่ซับซ้อนได้ ย่อมการันตีได้ว่าส่วนของการเล่าเรื่อง (Storytelling) และการสำรวจจะสามารถทำงานได้อย่างลื่นไหลและมีประสิทธิภาพในฐานะแอปพลิเคชันฉบับสมบูรณ์

4. ชุดกติกา "Lite 5e" ที่ใช้ในโครงการเป็นชุดกติกาที่ผู้จัดทำได้นิยามและกำหนดขอบเขตขึ้นเอง โดยอ้างอิงจากกลไกหลักของเกม Dungeons & Dragons ฉบับที่ 5 เพื่อให้มีความเหมาะสมกับการประมวลผลของปัญญาประดิษฐ์และการทำงานของระบบเชิงตรรกะ

1.6 ขั้นตอนการดำเนินงาน

ในการดำเนินงานวิจัยครั้งนี้ ผู้จัดทำได้ปรับปรุงกระบวนการพัฒนาโดยมุ่งเน้นไปที่การสร้างสถาปัตยกรรมระบบที่มีความเสถียร โดยแบ่งขั้นตอนออกเป็นดังนี้

1. การวิเคราะห์ปัญหาและออกแบบกฎเกณฑ์ใหม่ (Analysis & Rule Design)
 - 1) วิเคราะห์ข้อจำกัดของระบบ LLM-only ในการรักษากฎกติกา (Hallucination Analysis)
 - 2) ออกแบบและนิยามชุดกติกา "Lite 5e" เพื่อลดความซับซ้อนเชิงกลไกให้เหมาะสมกับการประมวลผลของเอเจนต์
2. การออกแบบสถาปัตยกรรมไฮบริด (Hybrid Architecture Design)
 - 1) ออกแบบระบบ Two-Path Orchestrator เพื่อแยกส่วนตรรกะเชิงกำหนด (Deterministic) ออกจากตรรกะเชิงภาษา (Probabilistic)
 - 2) ออกแบบโครงสร้างการสื่อสารระหว่างเอเจนต์ (Multi-Agent Handshake) และระบบจัดการสถานะเกม
3. การพัฒนาเอนจินกฎเกณฑ์เชิงกำหนด (Path A: Rules Engine Development)
 - 1) พัฒนาโมดูล Python สำหรับคำนวณการต่อสู้ (Combat Engine), ลำดับเทิร์น (Initiative), และการจัดการระยะทาง (Zone Logic) ให้มีความแม่นยำ 100%
4. การพัฒนาเอเจนต์ตัดสินใจและระบบคัดกรองเจตนา (Path B & Intent Routing)
 - 1) พัฒนาโมดูล Python สำหรับคำนวณการต่อสู้ (Combat Engine), ลำดับเทิร์น (Initiative), และการจัดการระยะทาง (Zone Logic) ให้มีความแม่นยำ 100%
 - 2) พัฒนา LLM Arbiter สำหรับตัดสินใจการกระทำเชิงสร้างสรรค์ และส่งค่าสถานะกลับเข้าสู่เอนจินหลัก

5. การเพิ่มประสิทธิภาพและการจัดการทรัพยากร (Optimization & Persistence)
 - 1) พัฒนาโปรโตคอล TOON Serialization เพื่อลดการใช้ทोकเกิล (Token)
 - 2) พัฒนาระบบหน่วยความจำแบบ $O(1)$ Sliding Window และระบบบันทึกสถานะเกม (JSON Persistence)
6. การทดสอบและทวนสอบระบบ (Verification & Evaluation)
 - 1) พัฒนาชุดทดสอบอัตโนมัติ 50 สถานการณ์ (50-Scenario Functional Test Suite) เพื่อวัดผลความแม่นยำในด้านการคัดกรอง, การระบุข้อมูล และการประสานสถานะเกม (State Synchronization)
7. การขยายระบบสู่โหมดการสำรวจและเนื้อเรื่อง (Expansion to World Exploration)
 - 1) พัฒนาระบบการเปลี่ยนผ่านสถานะ (State Transition) ระหว่างโหมดการต่อสู้และโหมดการสำรวจแบบอิสระ (Free-form Exploration)
 - 2) บำรุงรักษาระบบ Static RAG เพื่อดึงข้อมูลภูมิหลังโลก (World Lore) มาใช้ในการบรรยายเนื้อเรื่องให้มีความลึกซึ้งมากขึ้น
8. การพัฒนาส่วนต่อประสานผู้ใช้และการสรุปเนื้อหา (User Experience & Summarization)
 - 1) ปรับปรุง CLI Dashboard ให้รองรับการแสดงผลข้อมูลการสำรวจและบันทึกเหตุการณ์ที่ซับซ้อนขึ้น
 - 2) พัฒนาระบบ Campaign Summarizer เพื่อสรุปเหตุการณ์ทั้งหมดจาก Log ไฟล์ ให้ผู้เล่นเห็นภาพรวมของการผจญภัยเมื่อโหลดเกมกลับมาเล่นใหม่
9. การทดสอบกับผู้เล่นจริงและการเก็บข้อมูลเชิงคุณภาพ (Pilot Testing & Qualitative Evaluation)
 - 1) จัดทดสอบการใช้งาน (User Testing) กับกลุ่มตัวอย่างผู้เล่น Solo TTRPG เพื่อประเมินความลื่นไหลของเนื้อเรื่อง (Coherence) และความรู้สึกจจจจกับเกม (Immersion)
 - 2) วัดผลความพึงพอใจและประสิทธิภาพของระบบในสถานการณ์การเล่นจริง
10. การเพิ่มประสิทธิภาพและการจัดการทรัพยากร (Optimization & Persistence)
 - 1) รวบรวมข้อมูลการทดสอบทั้งหมดมาวิเคราะห์ผลเชิงวิศวกรรม
 - 2) จัดทำรูปเล่มวิทยานิพนธ์ฉบับสมบูรณ์และเตรียมการนำเสนอผลงานภาคจบการศึกษา

1.7 นิยามศัพท์เฉพาะ

1. AI Dungeon Master (AI DM) หมายถึง ระบบปัญญาประดิษฐ์ในรูปแบบสถาปัตยกรรมไฮบริดที่ทำหน้าที่เป็นผู้ดำเนินเกม (Narrator) และผู้ตัดสิน (Arbiter) ในเกมเทเบิลท็อป RPG โดยการประสานงานระหว่างเอเจนต์หลายตัว
2. Multi-Agent Hybrid Architecture หมายถึง สถาปัตยกรรมระบบที่ใช้ตัวแทนปัญญาประดิษฐ์หลายส่วนทำงานร่วมกัน โดยแบ่งหน้าที่รับผิดชอบอย่างชัดเจนระหว่างส่วนการประมวลผลเชิงภาษาและส่วนการประมวลผลตรรกะเชิงกำหนด
3. Two-Path Orchestrator หมายถึง กลไกตัวจัดการกลางที่ทำหน้าที่คัดกรองเจตนาของผู้เล่น (Intent Routing) เพื่อเลือกระหว่างเส้นทางการคำนวณตามกฎตายตัว (Path A Fixed) หรือเส้นทางการตัดสินใจเชิงสร้างสรรค์ (Path B Creative)
4. AI Hallucination (การหลอนของปัญญาประดิษฐ์) หมายถึง ปรากฏการณ์ที่แบบจำลองภาษาขนาดใหญ่สร้างข้อความหรือผลลัพธ์ที่ผิดพลาด ไม่สอดคล้องกับกฎเกณฑ์ หรือล้มสถานะความจริงในปัจจุบันของเกม
5. Deterministic Rules Engine หมายถึง ส่วนประกอบของซอฟต์แวร์ที่สร้างขึ้นเพื่อบังคับใช้กฎกติกาโดยใช้ตรรกะเชิงกำหนด (Python Logic) เพื่อให้ได้ผลลัพธ์ที่ถูกต้องแม่นยำ 100% ตามหลักคณิตศาสตร์
6. Lite 5e หมายถึง ชุดกติกาที่ถูกปรับลดความซับซ้อนลงจาก Dungeons & Dragons ฉบับที่ 5 โดยผู้จัดทำ เพื่อเพิ่มประสิทธิภาพในการประมวลผลของเอเจนต์และการจัดการหน่วยความจำ
7. TOON Serialization (Token-Oriented Object Notation) หมายถึง รูปแบบการจัดเตรียมและบีบอัดข้อมูลสถานะเกม (Game State) ให้เป็นข้อความขนาดเล็ก เพื่อลดปริมาณการใช้ทรัพยากร (Token) และความหน่วงในการสื่อสารผ่าน API
8. O(1) Context Memory หมายถึง เทคนิคการจัดการหน่วยความจำบริบทระยะสั้นโดยใช้โครงสร้างข้อมูลแบบ Sliding Window ที่มีความซับซ้อนเชิงเวลาคงที่ เพื่อรักษาความต่อเนื่องของเนื้อเรื่องโดยไม่ทำให้ระบบล่าช้า
9. Symbolic Side Effects หมายถึง ค่าสถานะเชิงสัญลักษณ์ที่เอเจนต์ผู้ตัดสินส่งกลับมายังระบบหลังบ้าน เพื่อนำไปปรับปรุงข้อมูลในฐานข้อมูลเกม เช่น สถานะดาบอด หรือ ดิคสตัน

บทที่ 2 ทฤษฎี/งานวิจัยที่เกี่ยวข้อง

ในบทนี้จะกล่าวถึงทฤษฎีพื้นฐานและเทคโนโลยีที่เกี่ยวข้องซึ่งเป็นหัวใจสำคัญในการพัฒนาโครงการ "AI Dungeon Master" รวมถึงทบทวนงานวิจัยและโครงการที่มีอยู่ก่อนหน้านี้ เพื่อวิเคราะห์แนวทางการพัฒนา จุดแข็ง จุดอ่อน และนำมาเป็นแนวทางในการออกแบบสถาปัตยกรรมของระบบในโครงการนี้ เนื้อหาจะแบ่งออกเป็นสองส่วนหลัก ได้แก่ ส่วนของทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง และส่วนของงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

โครงการนี้เป็นการบูรณาการเทคโนโลยีปัญญาประดิษฐ์หลายแขนงเข้าด้วยกัน เพื่อสร้างระบบที่สามารถทำหน้าที่อันซับซ้อนของ Dungeon Master ได้ เทคโนโลยีหลักที่นำมาใช้ประกอบด้วย แบบจำลองภาษาขนาดใหญ่ (LLM) และสถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน (Hybrid AI Architecture) โดยมีเทคโนโลยีสนับสนุน (Supporting Technologies) ได้แก่ การรู้จำเสียงพูดอัตโนมัติ (ASR) และการแปลงข้อความเป็นเสียง (TTS)

2.1.1 แบบจำลองภาษาขนาดใหญ่ (Large Language Models - LLM)

แบบจำลองภาษาขนาดใหญ่ หรือ LLM คือแบบจำลองปัญญาประดิษฐ์ประเภทหนึ่งที่ได้รับการฝึกฝนด้วยข้อมูลข้อความจำนวนมาก ทำให้มีความสามารถสูงในการทำความเข้าใจและสร้างภาษาที่ใกล้เคียงกับมนุษย์ แบบจำลองที่มีชื่อเสียงในปัจจุบัน เช่น GPT (Generative Pre-trained Transformer) ของ OpenAI หรือ Gemini ของ Google ล้วนมีพื้นฐานมาจากสถาปัตยกรรมที่เรียกว่า Transformer ซึ่งเป็นปัจจัยสำคัญที่ทำให้ LLM สามารถจัดการกับความซับซ้อนของภาษาได้ดี

1. จุดแข็งสำหรับโครงการ

1) การสร้างสรรค์เนื้อหา (Creative Text Generation) จุดแข็งที่สำคัญที่สุดของ LLM คือความสามารถในการสร้างเรื่องราว, คำบรรยายฉาก, และบทสนทนาของตัวละคร (NPC) ได้อย่างสร้างสรรค์และต่อเนื่อง ทำให้โลกของเกมมีความสมจริงและน่าสนใจ การทำงานแบบ Probabilistic ซึ่งอิงตามรูปแบบภาษาที่ได้เรียนรู้มา ช่วยให้ LLM สามารถสร้างผลลัพธ์ที่หลากหลายและคาดไม่ถึง ซึ่งเป็นหัวใจของความสนุกในเกม Tabletop RPG ที่ผู้เล่นมักจะกระทำการนอกกรอบที่คาดไว้ ตัวอย่างเช่น หากผู้เล่นตัดสินใจปีนหน้าต่างของโรงเตี๊ยมแทนที่จะเดินเข้าประตูหน้า LLM สามารถ

สร้างคำบรรยายถึงสภาพของหน้าต่างที่เก่าแก่ เสียงไม้ที่ลั่นเอี๊ยด และลมเย็นที่พัดเข้ามาได้ในทันทีโดยไม่ต้องมีการเตรียมสคริปต์ไว้ล่วงหน้า

2) การจัดการบทสนทนา (Dialogue Management) LLM สามารถเข้าใจเจตนาของผู้เล่นจากคำสั่งที่เป็นภาษาธรรมชาติ และสร้างการตอบสนองที่สอดคล้องกับสถานการณ์ได้อย่างยืดหยุ่น ซึ่งแตกต่างอย่างสิ้นเชิงจากระบบบทสนทนาแบบต้นไม้ (Dialogue Tree) ในเกมดั้งเดิมที่ผู้เล่นถูกจำกัดให้เลือกตอบตามตัวเลือกที่กำหนดไว้ล่วงหน้าเท่านั้น ความสามารถนี้ทำให้ตัวละครในเกม (NPC) มีชีวิตชีวาและตอบสนองได้อย่างสมจริง ตัวอย่างเช่น แทนที่จะมีเพียงตัวเลือกสนทนา 3 ข้อ ผู้เล่นสามารถถามคำถามปลายเปิดกับ NPC เช่น "คุณเคยได้ยินข่าวลือเกี่ยวกับมังกรบนภูเขาบ้างไหม?" และ LLM ก็สามารถสร้างคำตอบที่สอดคล้องกับบทบาทและบุคลิกของ NPC ตัวนั้นได้ทันที

2. ข้อจำกัดและประเด็นที่ต้องพิจารณา

1) ภาวะไร้สถานะ (Statelessness) และข้อจำกัดของหน่วยความจำ: LLM ไม่มีหน่วยความจำถาวร มันจะจดจำข้อมูลได้เท่าที่มีอยู่ใน "หน้าต่างบริบท" (Context Window) เท่านั้น ซึ่งเป็นเหมือนหน่วยความจำระยะสั้นที่มีขนาดจำกัด เมื่อการสนทนาหรือเรื่องราวดำเนินไปนานขึ้น ข้อมูลในช่วงแรกๆ จะหลุดออกจากหน้าต่างบริบท ทำให้ LLM อาจ "ลืม" เหตุการณ์สำคัญ ชื่อตัวละคร หรือข้อมูลที่เคยเกิดขึ้นไปแล้วได้ ปัญหานี้เป็นอุปสรรคสำคัญต่อการสร้างเรื่องราวที่ต่อเนื่องและสมเหตุสมผลในระยะยาว

2) การยึดถือกฎกติกา (Rule Adherence): โดยธรรมชาติแล้ว LLM เป็นแบบจำลองเชิงความน่าจะเป็น (Probabilistic Model) ไม่ใช่ระบบเชิงตรรกะ (Logical System) ทำให้มันไม่สามารถรับประกันได้ว่าจะปฏิบัติตามกฎกติกาที่ซับซ้อนและตายตัวของเกมได้อย่างถูกต้อง 100% บ่อยครั้งที่ LLM อาจสร้างผลลัพธ์ที่ดูสมเหตุสมผลแต่ผิดหลักกลไกของเกม หรือที่เรียกว่า "Hallucination" ซึ่งเป็นสิ่งที่ไม่สามารถยอมรับได้ในบริบทของเกมที่ต้องการความยุติธรรม

ข้อจำกัดทั้งสองข้อนี้เป็นความท้าทายหลักเชิงสถาปัตยกรรม และเป็นเหตุผลสำคัญที่โครงการนี้จำเป็นต้องมีส่วนประกอบอื่นเข้ามาทำงานร่วมกับ LLM แทนที่จะใช้ LLM เพียงอย่างเดียว

2.1.2 สถาปัตยกรรมหลายเอเจนต์ (Multi-Agent Orchestration)

สถาปัตยกรรมหลายเอเจนต์ หรือ Multi-Agent System (MAS) คือแนวคิดในการออกแบบระบบที่ประกอบด้วยตัวแทนปัญญาประดิษฐ์ (Agents) หลายตัวทำงานร่วมกัน โดยแต่ละตัวจะมีบทบาทหน้าที่ และขอบเขตความรับผิดชอบที่เฉพาะเจาะจง (Specialized Roles) เพื่อบรรลุเป้าหมายที่ซับซ้อนร่วมกัน ในบริบทของโครงการนี้ การใช้เอเจนต์หลายตัวทำงานประสานกัน (Orchestration) มีความสำคัญดังนี้

1. การแบ่งแยกหน้าที่ความรับผิดชอบ (Separation of Concerns - SoC) บทบาทของ Dungeon Master (DM) ในการเล่น Tabletop RPG นั้นมีความซับซ้อนสูงเกินกว่าที่ปัญญาประดิษฐ์เพียงตัวเดียว (Single Agent) จะจัดการได้อย่างมีประสิทธิภาพ หากใช้ LLM เพียงตัวเดียวรับผิดชอบทั้งการจดจำกฎ การคำนวณตัวเลข การบันทึกสถานะ และการบรรยายเนื้อเรื่อง จะทำให้เกิดภาระทางปัญญา (Cognitive Load) ที่สูงเกินไป นำไปสู่ปัญหาการสับสนของข้อมูลหรือการหลอน (Hallucination)

การใช้สถาปัตยกรรมหลายเอเจนต์ช่วยให้เราสามารถแยกส่วนงาน (Decoupling) ออกจากกันได้ เช่น การแยกส่วนที่ต้องการความแม่นยำทางตรรกะออกจากส่วนที่ต้องการความคิดสร้างสรรค์ ทำให้ระบบมีความเสถียรและสามารถตรวจสอบข้อผิดพลาด (Debug) ได้ง่ายขึ้นในแต่ละโมดูล

2. การเพิ่มประสิทธิภาพการประมวลผล (Task-Specific Optimization) เอเจนต์แต่ละตัวสามารถถูกกำหนด "คำสั่งระบบ" (System Instructions) และการตั้งค่าพารามิเตอร์ (เช่น Temperature) ที่แตกต่างกันเพื่อให้เหมาะสมกับงานนั้นๆ ที่สุด

1) เอเจนต์ที่เน้นตรรกะ (Logic-Oriented) จะถูกตั้งค่าให้มีความคิดสร้างสรรค์ต่ำ (Low Temperature) เพื่อป้องกันการบิดเบือนข้อมูล

2) เอเจนต์ที่เน้นการบรรยาย (Narrative-Oriented) จะถูกตั้งค่าให้มีความหลากหลายทางภาษาและความคิดสร้างสรรค์สูง (High Temperature) เพื่ออรรถรสในการเล่น

3. บทบาทของเอเจนต์หลักในโครงงาน (Key Agent Roles) เพื่อให้ระบบสามารถทำหน้าที่เป็น Dungeon Master ได้อย่างสมบูรณ์ โครงงานนี้จึงได้ออกแบบเอเจนต์เฉพาะทาง 3 กลุ่มหลัก ได้แก่

1) เอเจนต์คัดกรองเจตนา (Intent Router) ทำหน้าที่เป็น "ด่านหน้า" ในการวิเคราะห์คำสั่งภาษาธรรมชาติของผู้เล่น เพื่อตัดสินใจว่าคำสั่งนั้นควรถูกส่งไปประมวลผลในเส้นทางใด (Fixed หรือ Creative) เอเจนต์ตัวนี้ทำหน้าที่เสมือนตัวควบคุมการไหลของข้อมูล (Traffic Controller) ให้เป็นไปตามสถาปัตยกรรม Two-Path

2) เอเจนต์ผู้ตัดสิน (Action Arbiter) ทำหน้าที่เสมือน "กรรมการ" ในสถานการณ์ที่อยู่นอกเหนือกฎตายตัว (Path B) โดยจะใช้ความสามารถเชิงเหตุผล (Reasoning) ในการกำหนดค่าความยาก (DC) หรือตัดสินผลลัพธ์เชิงตรรกะจากจินตนาการของผู้เล่น แล้วส่งต่อผลลัพธ์นั้นกลับมาเป็นข้อมูลเชิงสัญลักษณ์ให้ระบบหลัก

3) เอเจนต์ผู้เล่าเรื่อง (DM Narrator) ทำหน้าที่แปลงผลลัพธ์ทางคณิตศาสตร์และตรรกะที่แห้งแล้งจาก Rules Engine ให้กลายเป็นคำบรรยายที่สวยงามและน่าติดตาม โดยจะนำบริบทจากหน่วยความจำระยะสั้นมาประกอบเพื่อให้เนื้อเรื่องมีความต่อเนื่อง

การทำงานประสานกันของเอเจนต์เหล่านี้ผ่านกระบวนการ Multi-Agent Handshake ช่วยให้ระบบสามารถรักษาสมดุลระหว่าง "ความถูกต้องของเกม" และ "ความลื่นไหลของจินตนาการ" ได้อย่างมีประสิทธิภาพ ซึ่งเป็นปัจจัยสำคัญที่ทำให้ AI สามารถทำหน้าที่แทน Dungeon Master ที่เป็นมนุษย์ได้

2.1.3 สถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน (Hybrid/Neuro-Symbolic AI)

สถาปัตยกรรมแบบผสมผสานหรือ Neuro-Symbolic AI คือหัวใจหลักเชิงวิศวกรรมของโครงการนี้ โดยเป็นการบูรณาการจุดเด่นของระบบปัญญาประดิษฐ์สองรูปแบบที่มีพื้นฐานการทำงานต่างกันโดยสิ้นเชิง เพื่อสร้างระบบที่มีทั้งความฉลาดลื่นไหลและความแม่นยำทางตรรกะ ซึ่งเปรียบเสมือนการจำลองการทำงานของสมองมนุษย์ที่แบ่งออกเป็น 2 ระบบหลัก (System 1 และ System 2 Thinking) ตามทฤษฎีของ Daniel Kahneman

1. ส่วนประกอบประมวลผลทางภาษา (Neural Component - LLM) เปรียบเสมือน System 1 ที่เน้นความเร็ว สัญชาตญาณ และความคิดสร้างสรรค์ ในโครงการนี้คือการใช้แบบจำลองภาษาขนาดใหญ่ (LLM) ในการรับผิดชอบงานด้านภาษาธรรมชาติและการด้นสด (Improvisation) ซึ่งเป็นส่วนที่ระบบคอมพิวเตอร์แบบดั้งเดิมทำได้ยาก อย่างไรก็ตาม ระบบนี้มีลักษณะเป็น Probabilistic (เชิงความน่าจะเป็น) ซึ่งมีความเสี่ยงต่อการเกิดข้อผิดพลาดทางตรรกะ
2. ส่วนประกอบประมวลผลเชิงตรรกะ (Symbolic Component - Python Code) เปรียบเสมือน System 2 ที่เน้นการคิดอย่างเป็นเหตุเป็นผล มีกฎระเบียบ และความถูกต้องแม่นยำ ในโครงการนี้คือการใช้ Rules Engine ที่เขียนด้วยภาษา Python ซึ่งมีลักษณะเป็น Deterministic (เชิงกำหนด) เพื่อควบคุมกลไกเกมและสถานะข้อมูลให้คงที่ตามกฎคณิตศาสตร์ 100%

การผสานสองระบบนี้เข้าด้วยกันผ่านการออกแบบสถาปัตยกรรมแบบ Two-Path Processing ช่วยให้ระบบสามารถเลือกเส้นทางประมวลผลที่เหมาะสมที่สุดตามเจตนาของผู้เล่น ดังนี้

1) เส้นทางตรรกะเชิงกำหนด (Path A: Deterministic Path) เป็นเส้นทางที่ใช้โค้ดภาษา Python ในการประมวลผลโดยตรงสำหรับคำสั่งที่เป็น "กฎตายตัว" (Fixed Actions) เช่น การคำนวณการทอยลูกเต๋าเพื่อตรวจสอบการโจมตี (Attack Roll) หรือการหักลบค่าพลังชีวิต (HP) ตามกฎ Lite 5e การใช้ระบบ Symbolic ในเส้นทางนี้ช่วยกำจัดปัญหา AI Hallucination ในส่วนของตัวเลขและกฎเกณฑ์ได้อย่างเด็ดขาด ทำให้ระบบมีความน่าเชื่อถือในฐานะ "เกม" ที่มีความยุติธรรม

2) เส้นทางตัดสินใจเชิงสร้างสรรค์ (Path B: Probabilistic Path) เป็นเส้นทางที่ใช้ความสามารถในการให้เหตุผล (Reasoning) ของ LLM ในการประมวลผลสำหรับคำสั่งที่ "อยู่นอกเหนือกฎกติกาตายตัว" (Creative Actions) เช่น ผู้เล่นต้องการใช้อุปกรณ์รอบตัวในการแก้ปริศนา หรือการเจรจาต่อรองกับศัตรู ในเส้นทางนี้ LLM จะทำหน้าที่เป็น Arbiter (ผู้ตัดสิน) เพื่อวิเคราะห์ความสมเหตุสมผลเชิงตรรกะ และกำหนดค่าความยาก (DC) ก่อนจะส่งข้อมูลกลับมาให้ระบบ Symbolic ทำการทอยลูกเต๋าเพื่อตัดสินผลในขั้นตอนสุดท้าย

สถาปัตยกรรมไฮบริดแบบ Two-Path นี้จึงเป็นการเติมเต็มช่องว่างระหว่าง จินตนาการ และ ความถูกต้อง ทำให้ AI Dungeon Master สามารถตอบสนองต่อผู้เล่นได้อย่างไร้ขีดจำกัดเหมือนมนุษย์ ในขณะที่ยังคงรักษากฎเกณฑ์และความแม่นยำของระบบคอมพิวเตอร์ไว้ได้อย่างสมบูรณ์

2.1.4 กลไกหน้าต่างเลื่อน (Sliding Window Mechanism) ปัญหาสำคัญประการหนึ่งในการระบบจะกำหนดขนาดสูงสุดของหน่วยความจำบริบท (Max Buffer Size) ไว้ในระดับที่เหมาะสม เช่น 10-15 เหตุการณ์ล่าสุด เมื่อมีเหตุการณ์ใหม่เกิดขึ้นในเกมและถูกเพิ่มเข้าไปในหน่วยความจำจนเต็มพื้นที่ ข้อมูลเหตุการณ์ที่เก่าที่สุดจะถูก "ผลัก" ออกจากหน้าต่างบริบทโดยอัตโนมัติ กลไกนี้ช่วยให้มั่นใจได้ในด้านต่างๆ ดังนี้

1. ความต่อเนื่องของเนื้อหา (Narrative Continuity) เอเจนต์ผู้เล่าเรื่อง (Narrator) และผู้ตัดสิน (Arbiter) จะได้รับเฉพาะบริบทที่สำคัญและมีความสดใหม่ที่สุด (Fresh Context) เพื่อใช้ในการตัดสินใจและพรรณานำเรื่อง ทำให้ลดโอกาสที่ AI จะนำข้อมูลที่ล้าสมัยหรือเหตุการณ์ที่จบไปแล้วมาปะปนกับสถานการณ์ปัจจุบัน
2. การเพิ่มประสิทธิภาพการใช้ทोकเกิล (Token Optimization) ระบบสามารถควบคุมปริมาณการใช้ทोकเกิลต่อการเรียกใช้งาน API ให้อยู่ในระดับที่คงที่และคุ้มค่าที่สุด ช่วยลดค่าใช้จ่ายและป้องกันความล่าช้าในการประมวลผล (Latency) ที่เกิดจากการส่งชุดข้อมูลที่มีความยาวมากเกินไปเข้าสู่โมเดลภาษา
3. ความเสถียรของหน้าต่างบริบท (Context Stability) การรักษาขนาดข้อมูลให้คงที่ช่วยป้องกันปัญหาหน้าต่างบริบทเต็ม (Context Overflow) ซึ่งมักจะส่งผลให้ปัญญาประดิษฐ์เริ่มสูญเสียความสามารถในการให้เหตุผลเชิงตรรกะ

2.1.5 การสร้างข้อความด้วยการดึงข้อมูลเสริมแบบคงที่ (Static Retrieval-Augmented Generation - Static RAG)

เทคนิค Static RAG คือกลไกสำคัญที่ช่วยให้ AI Dungeon Master สามารถเข้าถึงข้อมูลภูมิหลังของโลกเกม (World Lore) ได้อย่างถูกต้องแม่นยำ โดยไม่ต้องพึ่งพาความจำภายในโมเดล (Parametric Memory) เพียงอย่างเดียว ซึ่งมีรายละเอียดทางเทคนิคดังนี้

1. ฐานความรู้ภายนอกแบบคงที่ (Static Knowledge Base) ระบบจะจัดเก็บข้อมูลรายละเอียดสถานที่, ประวัติศาสตร์ของโลก, กฎเกณฑ์เฉพาะ และลักษณะนิสัยของ NPC ไว้ในรูปแบบไฟล์ข้อความ (Flat-file Database) ภายนอกโมเดล การแยกข้อมูลส่วนนี้ออกมาช่วยให้ระบบสามารถจัดการข้อมูลจำนวนมากได้โดยไม่ติดข้อจำกัดด้าน Context Window ของโมเดลภาษา

2. กลไกการฉีดข้อมูลบริบท (Context Injection) เมื่อผู้เล่นทำการสำรวจ (Explore) หรือมีปฏิสัมพันธ์กับโลกเกม ระบบจะใช้กระบวนการดึงข้อมูล (Retrieval) เพื่อค้นหาเฉพาะส่วนที่เกี่ยวข้องกับสถานการณ์นั้นๆ จากไฟล์ Lore แล้วทำการ "ฉีด" ข้อมูลดังกล่าวเข้าไปใน Prompt ของเอเจนต์ผู้เล่าเรื่อง (Narrator) เพื่อให้ AI ใช้เป็น "ความจริงอ้างอิง" (Source of Truth) ในการพรรณานำเรื่อง ลดปัญหาการสร้างข้อมูลเท็จ (Hallucination) ในระดับโลกของเกม
3. ความยืดหยุ่นและการขยายผล (Scalability & Reconfigurability) การใช้ Static RAG ช่วยให้ระบบมีความยืดหยุ่นสูง ผู้จัดทำสามารถเปลี่ยน "ธีม" หรือขยาย "ขอบเขตของโลก" ได้ทันทีเพียงแค่แก้ไขไฟล์ข้อมูลภายนอก โดยไม่ต้องทำการฝึกฝนโมเดลใหม่ (Fine-tuning) หรือแก้ไขโค้ดโปรแกรมหลัก

2.1.6 การจัดเตรียมข้อมูลด้วยโปรโตคอล TOON (Token-Oriented Object Notation)

ในการสื่อสารระหว่างเอเจนต์ในสถาปัตยกรรมแบบ Multi-Agent ปริมาณข้อมูลสถานะเกม (Game State) ที่ต้องส่งผ่าน API มีขนาดใหญ่ โครงการนี้จึงเลือกใช้โปรโตคอล TOON ซึ่งเป็นรูปแบบการจัดเตรียมข้อมูล (Serialization) ที่ออกแบบมาเพื่อเพิ่มประสิทธิภาพให้กับโมเดลภาษาโดยเฉพาะ

1. การลดปริมาณการใช้ทรัพยากร (Token Efficiency) TOON ถูกออกแบบมาเพื่อมีความกระชับสูง โดยการนำจุดเด่นของโครงสร้างแบบลำดับชั้นของ YAML มาผสมผสานกับการจัดเรียงข้อมูลแบบตาราง (Tabular Layout) ของ CSV ทำให้สามารถลดปริมาณ Token ที่ใช้ลงได้เฉลี่ย 30-40% เมื่อเทียบกับโครงสร้าง JSON แบบดั้งเดิม ส่งผลให้ระบบมีการตอบสนองที่รวดเร็วขึ้น (Lower Latency) และลดค่าใช้จ่ายในการใช้งาน API
2. การเสริมสร้างความเข้าใจของโมเดล (LLM-Friendly Guardrails) แม้จะมีความกระชับ แต่ TOON ยังคงรักษาโครงสร้างข้อมูล (Schema) ที่ชัดเจนผ่านสัญลักษณ์พิเศษ เช่น การระบุความยาวของอาร์เรย์ [N] และการระบุหัวข้อฟิลด์ {fields} ซึ่งช่วยให้โมเดลภาษาทำการวิเคราะห์ข้อมูล (Parsing) และรักษาสมมูลข้อมูล (State Grounding) ได้อย่างแม่นยำสูงถึง 76% (เทียบกับ JSON ที่ 75%) ภายใต้การใช้ทรัพยากรที่น้อยกว่า
3. ความแม่นยำในการรักษาความจริงของข้อมูล (Deterministic Round-trips) การใช้ TOON ช่วยให้เอเจนต์ได้รับข้อมูลสถานะที่ชัดเจนและไม่คลุมเครือ ลดโอกาสที่จะเกิดการบิดเบือนข้อมูลสถานะตัวละคร (เช่น ค่า HP หรือไอเทม) ในขณะที่เอเจนต์ผู้เล่าเรื่องกำลังประมวลผลคำบรรยาย ทำให้การซิงโครไนซ์สถานะเกม (State Synchronization) มีความเสถียรสูงสุด

2.2 งานวิจัยและโครงการที่เกี่ยวข้อง

ในช่วงไม่กี่ปีที่ผ่านมา มีความพยายามในการนำแบบจำลองภาษาขนาดใหญ่ (LLM) มาประยุกต์ใช้เป็น Dungeon Master (DM) ในหลากหลายรูปแบบ การทบทวนและวิเคราะห์งานวิจัยเหล่านี้ช่วยให้เห็นถึง

วิวัฒนาการของเทคโนโลยี สถาปัตยกรรม (Settings), ข้อสันนิษฐาน (Assumptions), และข้อจำกัด (Gaps) ซึ่งเป็นข้อมูลสำคัญในการออกแบบสถาปัตยกรรมของโครงการนี้

2.2.1 Triyason (2023) - การใช้ ChatGPT เป็น DM สำหรับผู้เล่นคนเดียว (The "LLM-only" Baseline)

งานวิจัยของ Triyason (2023) นับเป็นหนึ่งในงานวิจัยกลุ่มแรกที่ทดลองใช้ ChatGPT ในการดำเนินเกม D&D พบว่า LLM สามารถสร้างเรื่องราวที่ต่อเนื่องและตอบสนองต่อการกระทำของผู้เล่นได้ดีพอที่จะทำให้ผู้เล่นใหม่ได้สัมผัสกับประสบการณ์ D&D อย่างไรก็ตาม ข้อจำกัดที่พบคือการตอบสนองที่ล่าช้าในบางครั้ง และการขาดการแสดงอารมณ์ร่วม (Limited Emotional Engagement) ซึ่งส่งผลต่อความอินของผู้เล่น

เพื่อทำความเข้าใจบริบทของงานวิจัยนี้อย่างลึกซึ้ง (Deep Dive) และเพื่อสนับสนุนการออกแบบสถาปัตยกรรมของโครงการนี้ สามารถวิเคราะห์องค์ประกอบของงานวิจัยดังกล่าวได้ดังนี้

1. สถาปัตยกรรมและกติกาที่ใช้ (Their Setting)

1) Rulebook/Tech: งานวิจัยนี้ใช้กฎ Dungeons & Dragons 5e (D&D 5e) เป็นพื้นฐาน โดยนำเสนอในรูปแบบข้อความล้วน (Text-only)

2) Architecture: สถาปัตยกรรมทางเทคนิคเป็นแบบ Monolithic (LLM-only) คือใช้ ChatGPT เพียงตัวเดียว และอาศัยเทคนิค Prompt Engineering (การออกแบบคำสั่ง) ในการกำหนดบทบาทและควบคุมพฤติกรรมให้ AI ทำหน้าที่เป็น DM

3) Engine: ระบบนี้ไม่มี Custom Game Engine หรือ Rules Engine แยกต่างหาก การตัดสินใจผลและการปฏิบัติตามกฎ (Rule Adherence) ทั้งหมดขึ้นอยู่กับความรู้ภายใน (Internal Knowledge) ของ ChatGPT เท่านั้น ซึ่งเปรียบเสมือนการใช้งาน "ChatGPT-as-DM" ผ่านหน้าต่างสนทนาปกติ

2. ข้อสันนิษฐานและข้อจำกัด (Their Assumption)

1) Solo Player: งานวิจัยตั้งข้อสันนิษฐานหลักคือผู้เล่นเป็นแบบ Solo Player (เล่นคนเดียวกับ AI)

2) Text-only Constraints: มีข้อจำกัดด้านรูปแบบการเล่นที่เป็น Textual ทั้งหมด ไม่มีการใช้กราฟิก แผนที่ หรือระบบทอยลูกเต๋าจำลอง (Dice Simulator) ภายนอก

3) Out-of-the-box: AI ที่ใช้เป็นแบบ "Out-of-the-box" (ไม่ผ่านการ Fine-tune) โดยอาศัย Prompt Engineering เพียงอย่างเดียว

3. การเปรียบเทียบ (Their Comparison)

1) Benchmark: งานวิจัยทำการเปรียบเทียบประสิทธิภาพแบบตัวต่อตัว (Head-to-head) ระหว่าง AI Dungeon Master กับ Human Dungeon Master

2) Method: ผู้เข้าร่วมการทดลองจะได้เล่นเกมกับทั้งสองระบบ จากนั้นจึงทำแบบประเมิน (Score and Evaluate) เพื่อเปรียบเทียบประสิทธิภาพที่ได้รับในด้านต่างๆ

4. ผลลัพธ์การทดลอง (Their Performance)

1) Positive: ผลลัพธ์แสดงให้เห็นว่า LLM สามารถสร้าง "Coherent Narratives" (เนื้อเรื่องที่ต่อเนื่อง สมเหตุสมผล) และ "Reacted Dynamically" (ตอบสนองต่อการกระทำของผู้เล่น) ได้ในระดับที่น่าพอใจ

2) Negative (Performance): ผู้เล่นรายงานข้อจำกัดด้าน "Delayed Responses" (การตอบสนองล่าช้า/Latency) และ "Limited Emotional Engagement" (การขาดอารมณ์ร่วม) ซึ่งส่งผลกระทบต่อ "Player Immersion" (ความรู้สึกดื่มด่ำในเกม)

3) Negative (Logic/Rules): เนื่องจากขาดระบบจัดการกฎภายนอก AI จึงประสบปัญหาในการ "Update the Game State Consistently" (การรักษาสถานะเกมให้สม่ำเสมอ) เช่น การลืมหักพลังชีวิต (HP), ลืมไอเทม, หรือลืมหเหตุการณ์ที่เพิ่งเกิดขึ้น ซึ่งเป็นข้อจำกัดร้ายแรงของสถาปัตยกรรมแบบ LLM-only

5. บทเรียนที่นำมาปรับใช้ (What We Will Use)

1) การออกแบบ Prompt (Prompt Design) เป็นปัจจัยสำคัญในการควบคุมพฤติกรรมของ LLM

2) ที่สำคัญที่สุด: ข้อค้นพบของ Triyason (เรื่องความล้มเหลวในการจัดการ State และ Rules) ถูกนำมาใช้เป็น Problem Statement หลัก ของโครงการนี้ เพื่อเป็นเหตุผลสนับสนุน (Justification) ว่าทำไมเราจึงจำเป็นต้องพัฒนา Rules Engine และ Game State Tracker แยกต่างหาก แทนที่จะพึ่งพา LLM เพียงอย่างเดียว

6. ช่องว่างและข้อจำกัดของงานวิจัย (The Gap/Limit)

1) ระบบของ Triyason มีลักษณะ "Minimalist and Fragile" (เรียบง่ายแต่เปราะบาง) เนื่องจากขาดกลไกเกม (Mechanics) ที่ชัดเจนและไม่มีส่วนต่อประสานกับผู้ใช้ (UI) ทำให้ประสิทธิภาพการเล่นเป็นแบบ "Fast but Shallow" (เร็วแต่ตื้นเขิน)

2) Gap ที่โครงการนี้จะเติมเต็ม

(1) เพิ่ม Lite 5e Rules Engine และ Game State Tracker เพื่อแก้ปัญหา "Consistency" (ความถูกต้องแม่นยำของกฎและสถานะ)

(2) เพิ่ม UI (Character Sheets, Zone Maps) และ Voice Input (ASR) เพื่อให้ระบบเข้าถึงง่ายและใช้งานสะดวกกว่า (More Accessible) การเพิ่มฟังก์ชันการพิมพ์ข้อความเพียงอย่างเดียว

7. แนวทางการประเมินผล (Their Evaluation)

- 1) เราจะนำแนวคิดการใช้ "Post-game Surveys" (แบบสอบถามหลังการเล่น) ของงานวิจัยนี้มาปรับใช้ (Adapt)
- 2) ในการทดสอบ Pilot Test (หัวข้อ 3.5) เราจะใช้คำถามที่วัดผลด้าน Immersion, Narrative Coherence, และ Rule Adherence (เช่น Q1, Q3, Q4, Q5) เพื่อตรวจสอบว่าสถาปัตยกรรมแบบ Hybrid ของเราสามารถแก้ไขปัญหาที่พบในงานวิจัยของ Triyason ได้จริงหรือไม่

2.2.2 Sakellaridis (2024) - การเปรียบเทียบ LLM ที่ผ่านการ Fine-tune กับ DM มนุษย์

งานวิจัยนี้ได้ดำเนินการศึกษาต่อโดยการนำแบบจำลองภาษาขนาดใหญ่ (LLM) มาผ่านกระบวนการปรับแต่งละเอียด (Fine-tuning) ด้วยชุดข้อมูลบทสนทนาจากเกม D&D และข้อมูลกฎกติกาโดยเฉพาะ เพื่อเปรียบเทียบประสิทธิภาพการดำเนินเกมกับ Dungeon Master (DM) ที่เป็นมนุษย์ ผลการศึกษาเบื้องต้นพบประเด็นที่น่าสนใจคือ ผู้เล่นเกมให้คะแนนความรู้สึกดื่มด่ำ (Immersion) ต่อคำบรรยายฉากของ AIDM ในระดับสูงเทียบเท่าหรือสูงกว่า DM มนุษย์ ซึ่งแสดงให้เห็นถึงศักยภาพของ LLM ในการสร้างสรรค์เนื้อหาเชิงพรรณนา อย่างไรก็ตาม ข้อจำกัดสำคัญที่พบคือ AI มักมีแนวโน้มที่จะดำเนินเรื่องไปในทิศทางที่กำหนดไว้ล่วงหน้าอย่างเคร่งครัด (Railroading) ขาดความยืดหยุ่นในการตอบสนองต่อการตัดสินใจนอกกรอบของผู้เล่น และยังไม่สามารถสร้างความท้าทายในเกมได้ดีพอ

เพื่อทำความเข้าใจบริบทของงานวิจัยอย่างลึกซึ้ง (Deep Dive) และเพื่อสนับสนุนการออกแบบสถาปัตยกรรมของโครงการนี้ สามารถวิเคราะห์องค์ประกอบของงานวิจัยดังกล่าวได้ดังนี้

1. สถาปัตยกรรมและกติกาที่ใช้ (Their Setting)

- 1) Rulebook/Tech: งานวิจัยนี้ใช้บริบทของ Dungeons & Dragons 5th Edition (D&D 5e) ในรูปแบบสภาพแวดล้อมเกมสวมบทบาทแบบข้อความ (Text-based RPG Environment)
- 2) Architecture: สถาปัตยกรรมทางเทคนิคใช้ ChatGPT (LLM-based agent) ที่ผ่านการ Fine-tune ด้วยชุดข้อมูล "Critical Role Transcripts" (บันทึกบทสนทนาจากการเล่น D&D จริง) เพื่อให้โมเดลเรียนรู้รูปแบบการเล่าเรื่องและบทบาทสมมติ
- 3) Knowledge: ระบบ AI สามารถเข้าถึงกฎ D&D 5e ฉบับเต็มและเนื้อหาของโมดูลการผจญภัยสำเร็จรูป (Adventure Module) เรื่อง "The Sunless Citadel" ซึ่งถูกป้อนให้ AI ใช้เป็นฐานข้อมูลอ้างอิง

2. ข้อสันนิษฐานและข้อจำกัด (Their Assumption)

- 1) Solo Play Scenario: การทดลองตั้งอยู่บนสมมติฐานการเล่นแบบผู้เล่นเดี่ยว (Solo Player)
- 2) Pre-written Adventure: AI จะดำเนินเรื่องตามบทการผจญภัยที่เขียนไว้ล่วงหน้า ("The Sunless Citadel") ไม่ใช่การสร้างเนื้อเรื่องใหม่แบบสุ่ม (Procedural Generation)

3) Fine-tuning Hypothesis: มีข้อสันนิษฐานว่าการ Fine-tune ด้วยข้อมูลคุณภาพสูง (Critical Role) จะช่วยยกระดับคุณภาพการบรรยายและสไตล์การเล่นให้ใกล้เคียงมนุษย์

4) Prompt Constraints: ระบบถูกควบคุมอย่างเข้มงวดด้วย "Huge System Prompt" เพื่อกำกับพฤติกรรมและขอบเขตการตอบสนองของโมเดล

3. การเปรียบเทียบ (Their Comparison)

1) Benchmark: งานนี้เปรียบเทียบ (head-to-head) ประสิทธิภาพระหว่าง AI DM vs. Human DM

2) Method: ผู้เข้าร่วมทดลองถูกแบ่งเป็น 2 กลุ่ม (กลุ่มเล่นกับ AI และกลุ่ม Control ที่เล่นกับมนุษย์) โดยเล่นใน Scenario เดียวกัน ("Sunless Citadel"). จากนั้นนำคะแนนความพึงพอใจ (Player satisfaction scores) มาเปรียบเทียบกัน

4. ผลลัพธ์การทดลอง (Their Performance)

ผลลัพธ์ (Performance) ออกมา "ผสมผสาน" (Mixed) โดยใช้การวัดผลด้วย Game Experience Questionnaire (GEQ) และการสัมภาษณ์

1) Positive: AI ทำได้ "ใกล้เคียงมนุษย์" ในหลายด้าน และทำได้ ดีกว่ามนุษย์ (อย่างมีนัยสำคัญทางสถิติ $P < 0.05$) ในการสร้าง "Sensory and Imaginative Immersion" (การบรรยายฉากที่สมจริง) (คะแนนเฉลี่ย 4.13/5 เทียบกับมนุษย์ 3.35)

2) Negative: มนุษย์ยังคงมี "นัยสำคัญทางสถิติ" ($P < 0.05$) ในด้าน Competence (ความสามารถโดยรวม) และ Flow (ความลื่นไหลของเนื้อเรื่อง)

3) AI มีจุดอ่อนร้ายแรง 3 ข้อ (จากการสัมภาษณ์)

(1) "Railroading" ผู้เล่น: (3 ใน 7 instances) AI ไม่ยืดหยุ่นและพยายามบังคับผู้เล่นเดินตามเส้นเรื่อง (Struggled with unexpected choices)

(2) "Poor Information Delivery": (1 ใน 7 instances) AI ลืมให้ Clue หรือข้อมูลสำคัญในเวลาที่เหมาะสม ทำให้ผู้เล่นสับสน

(3) "Lack of Challenge": (3 ใน 7 players) AI สร้างความท้าทายหรือ "ความรู้สึกอันตราย" (Perceived lack of danger) ได้ไม่ดีพอ (เกมง่ายเกินไป)

5. บทเรียนที่นำมาปรับใช้ (What We Will Use)

1) เราจะนำแนวคิดการ "Grounding" AI (การยึดโยงกับข้อมูลจริง) มาปรับใช้ โดยในโครงการนี้จะใช้ Lite 5e Rulebook เป็นฐานข้อมูลอ้างอิง

2) เรานำปัญหา "Railroading" ที่เราพบ มาเป็น "ปัญหาหลัก" (Key Problem) ที่เราต้องออกแบบสถาปัตยกรรมเพื่อแก้ไข

6. ช่องว่างและข้อจำกัดของงานวิจัย (The Gap/Limit)

1) Complexity Gap: ระบบของ Sakellaridis ใช้วิธีการ Fine-tuning ซึ่งมีความซับซ้อนสูงและไม่เอื้อต่อการนำไปใช้งานจริงแบบพกพา (Not Portable) สำหรับผู้เล่นทั่วไป (Casual User)

2) Our Solution: สถาปัตยกรรม "Two-Path" (โดยเฉพาะ Path B: Creative Actions) ของเราจะเข้ามาเติมเต็มช่องว่างนี้ โดยแก้ปัญหา "Railroading" ด้วยวิธีการที่ต่างออกไป คือการใช้ LLM (ในฐานะ DM) เป็นผู้ตัดสินใจ (Arbitrate) ในสถานการณ์นอกกรอบ และใช้ Rules Engine จัดการกลไกกลุ่ม ซึ่งจะมอบความยืดหยุ่น (Flexibility) ได้ดีกว่าโดยไม่ต้องพึ่งพาการ Fine-tune ที่ซับซ้อน ทำให้ระบบของเราติดตั้งง่ายกว่า (Simpler to Deploy) และมีความยืดหยุ่นสูงกว่า (Less Rigid)

7. แนวทางการประเมินผล (Their Evaluation)

1) เราจะนำหมวดหมู่การวัดผล (Evaluation Categories) ของงานวิจัยนี้มาปรับใช้โดยตรง ได้แก่ Immersion, Narrative Flow, DM Competence (Rule Adherence), และ Challenge Level

2) คำถามในการทดสอบ Pilot Test (Q1-Q6) ในหัวข้อ 3.5 ของโครงงานนี้ ได้รับแรงบันดาลใจมาจากตัวชี้วัดเหล่านี้ เพื่อตรวจสอบว่าสถาปัตยกรรม "Two-Path" สามารถแก้ไขปัญหาที่ Sakellaridis พบ (เช่น ปัญหา Railroading ใน Q5 และ Challenge ใน Q1) ได้จริงหรือไม่

2.2.3 Jørgensen et al. (2024) - สถาปัตยกรรมแบบ Multi-Agent

งานวิจัยของ Jørgensen และคณะ (2024) ได้นำเสนอระบบ "ChatRPG" ซึ่งเป็นแนวทางที่น่าสนใจในการออกแบบ AI Dungeon Master ด้วยสถาปัตยกรรมแบบหลายเอเจนต์ (Multi-Agent Architecture) โดยประยุกต์ใช้กรอบแนวคิด ReAct (Reason + Act) เพื่อให้ AI มีกระบวนการ "คิด" (Reasoning) และ "กระทำ" (Action) ที่เป็นขั้นตอน ซึ่งแนวทางนี้ถือเป็นแรงบันดาลใจสำคัญสำหรับการออกแบบเชิงโมดูล (Modular Design) ของโครงงานนี้

เพื่อทำความเข้าใจบริบทของงานวิจัยนี้อย่างลึกซึ้ง (Deep Dive) และเพื่อสนับสนุนการออกแบบสถาปัตยกรรมของโครงงานนี้ สามารถวิเคราะห์องค์ประกอบของงานวิจัยดังกล่าวได้ดังนี้

1. สถาปัตยกรรมและกติกาที่ใช้ (Their Setting)

1) Architecture: ระบบถูกออกแบบเป็น Multi-agent System ที่ขับเคลื่อนด้วย OpenAI's GPT-4 ภายใต้กรอบการทำงาน ReAct Framework

2) Separation of Concerns: พวกเขา "แยกส่วน" (split) DM ออกเป็นเอเจนต์เฉพาะทาง

(1) "Narrator agent" (ทำหน้าที่เล่าเรื่อง)

(2) "Archivist agent" (ทำหน้าที่จำ State/กฎ)

3) ระบบมีการใช้ Rules Logic Layer หรือเครื่องมือ (Tools/Functions) สำหรับจัดการกลไกเกมที่ซับซ้อน เช่น ฟังก์ชันการต่อสู้ (Battle()) หรือการรักษา (HealCharacter()) ร่วมกับการใช้ฐานข้อมูล (Entity Framework) ในการจดจำสถานะ

2. ข้อสันนิษฐานและข้อจำกัด (Their Assumption)

- 1) Solo Player Focus: งานวิจัยนี้ตั้งอยู่บนสมมติฐานการเล่นแบบผู้เล่นเดี่ยว (Single-player)
- 2) Hybrid Superiority: มีข้อสันนิษฐานว่าการ "แยกหน้าที่" (Splitting DM Duties) ระหว่างการเล่าเรื่องและการคุมกฎ จะให้ผลลัพธ์ที่ดีกว่าการใช้ LLM ตัวเดียวทำทุกอย่าง (Monolithic Approach)
- 3) Stateless Handling: เนื่องจาก LLM มีลักษณะ Stateless (ไม่จดจำบริบทข้ามรอบ) ระบบจึงจำเป็นต้องมี Archivist Agent คอยป้อนข้อมูลความทรงจำ (Memory Injection) จากฐานข้อมูลกลับเข้าไปในการประมวลผลทุกครั้ง

3. การเปรียบเทียบ (Their Comparison)

- 1) Benchmark: การศึกษานี้เน้นการเปรียบเทียบระหว่าง AI vs. AI เพื่อวัดประสิทธิภาพของสถาปัตยกรรม
- 2) Method: เปรียบเทียบประสิทธิภาพระหว่าง ระบบ v2 (Agentic, Multi-agent) ซึ่งมีความซับซ้อนสูง กับ ระบบ v1 (Static, Single-prompt) ที่ใช้เพียง Prompt เดียวแบบดั้งเดิม เพื่อแยกแยะผลกระทบและข้อดีของการใช้สถาปัตยกรรมแบบหลายเอเจนต์

4. ผลลัพธ์การทดลอง (Their Performance)

- 1) Positive: ผู้เล่น "Strongly preferred v2" (ชอบ v2 มากกว่าชัดเจน) โดย v2 ได้คะแนนสูงกว่า v1 (อย่างมีนัยสำคัญทางสถิติ $P < 0.05$) ในด้าน
 - (1) "Perceived Intelligence" (ความฉลาด)
 - (2) "Narrative Flow" (ความลื่นไหลของเรื่อง)
 - (3) "Immersion" (ความดื่มด่ำ)
- 2) Positive: ระบบ v2 นั้น "Robust" (ไม่พังง่าย) และ "Consistent" (จำเริญ) เพราะ "Archivist" คอยจํารายละเอียดให้

5. บทเรียนที่นำมาปรับใช้ (What We Will Use)

- 1) งานวิจัยนี้ถือเป็น "ต้นแบบแนวคิดหลัก" (Core Idea) ของโครงการ โดยเรานำหลักการ "แยกส่วนตรรกะออกจากส่วนเนื้อเรื่อง" (Separating Logic from Narrative) มาประยุกต์ใช้โดยตรง

2) สถาปัตยกรรม "Two-Path" ของเรา คือ "Direct Implementation" (การนำมาประยุกต์ใช้โดยตรง) จากแนวคิดนี้

- (1) Rules Engine & State Tracker ของเรา ทำหน้าที่เทียบเท่า "Archivist Agent"
- (2) LLM (as DM/Narrator) ของเรา ทำหน้าที่เทียบเท่า "Narrator Agent"
- (3) "Router" ของเรา ทำหน้าที่เทียบเท่ากระบวนการคิดแบบ "ReAct" เพื่อตัดสินใจเลือกเส้นทางการทำงาน

6. ช่องว่างและข้อจำกัดของงานวิจัย (The Gap/Limit)

1) High Complexity: ระบบของ Jørgensen มีความซับซ้อนสูงและใช้ทรัพยากรมาก (Powerful but Complex) เนื่องจากต้องใช้ GPT-4, ระบบ Multi-agent, และฐานข้อมูลขนาดใหญ่ ซึ่งทำให้ "ไม่สามารถพกพาได้สะดวก" (Not Portable) และ "เข้าถึงยาก" (Not Casual) สำหรับผู้ใช้ทั่วไป

2) Our Solution: โครงการนี้จะเข้ามาเติมเต็มช่องว่างดังกล่าว โดยใช้ตรรกะการแยกส่วนเดียวกัน แต่พัฒนาบนระบบที่ "เรียบง่ายและเบากว่า" (Simpler, Lightweight) โดยใช้กฎ Lite 5e และพัฒนาเป็นแอปพลิเคชันแบบ Self-contained บน Flutter พร้อมเพิ่มฟีเจอร์ UI และ Voice Input ที่ระบบต้นแบบยังขาดไป

7. แนวทางการประเมินผล (Their Evaluation)

1) เราจะนำ เกณฑ์วัดผลประสบการณ์ผู้ใช้ (User Experience Metrics) ของงานวิจัยนี้ ซึ่งอ้างอิงจาก PXI (Player Experience Inventory) มาปรับใช้

2) โดยเฉพาะตัวชี้วัดด้าน Immersion, Flow, และ Perceived Intelligence ซึ่งได้ถูกนำมาแปลงเป็นคำถามสำหรับการทดสอบ Pilot Test (Q3, Q5) ในหัวข้อ 3.5 เพื่อตรวจสอบคุณภาพของประสบการณ์การเล่น

2.2.4 Tool-Assisted AI DM (Song et al., 2024) - การใช้ RAG และ Function Calling

งานวิจัยล่าสุดโดย Song et al. (2024) ได้นำเสนอแนวคิดการพัฒนา AI DM ด้วยสถาปัตยกรรมแบบผสมผสาน (Hybrid Approach) ที่ผสานการทำงานของ LLM เข้ากับเครื่องมือภายนอก (External Tools) ผ่านกลไก Function Calling และ RAG เพื่อแก้ไขปัญหาความไม่แน่นอนของ LLM ผลการทดลองยืนยันว่าสถาปัตยกรรมนี้ช่วยให้ AI มีความสม่ำเสมอในการรักษาสถานะของเกม (State Consistency) และปฏิบัติตามกฎได้อย่างน่าเชื่อถือ (Reliability) สูงกว่าการใช้ LLM เพียงอย่างเดียวอย่างมีนัยสำคัญ เพื่อทำความเข้าใจบริบทของงานวิจัยนี้อย่างลึกซึ้ง (Deep Dive) และเพื่อสนับสนุนการออกแบบสถาปัตยกรรมของโครงการนี้ สามารถวิเคราะห์องค์ประกอบของงานวิจัยดังกล่าวได้ดังนี้

1. สถาปัตยกรรมและกติกาที่ใช้ (Their Setting)

- 1) Rulebook/Tech: งานวิจัยนี้ใช้ Setting ของเกม "Jim Henson's Labyrinth: The Adventure Game" (ซึ่งเป็น TTRPG ที่มีกฎง่ายกว่า D&D)
- 2) Architecture: สถาปัตยกรรม (Technical Setup) เป็นแบบ Hybrid LLM + External Tools โดยใช้ LLM (GPT-4/3.5) ที่มีความสามารถ Function Calling
- 3) Tools: พวกเขาได้ Implement "เครื่องมือ" (โค้ด) แยกออกมา 2 ประเภทหลัก
 - (1) Dice Roll Functions (เช่น activate_test())
 - (2) State Functions (เช่น add_item(), create_npc())
- 4) RAG: มีการใช้เทคนิค Retrieval-Augmented Generation (RAG) ในการดึง "สรุปกฎ" (Rule Summary) และ "สถานะเกมปัจจุบัน" (Game State) เข้าไปใน Prompt เพื่อให้ AI รับรู้บริบทที่ถูกต้องเสมอ

2. ข้อสันนิษฐานและข้อจำกัด (Their Assumption)

- 1) Tools Mitigate Limitations: งานวิจัยตั้งสมมติฐานว่า ข้อจำกัดของ LLM ด้านความจำและความแม่นยำ สามารถแก้ไขได้ด้วยการ "มอบเครื่องมือ" (Giving the model tools) ให้ใช้งาน
- 2) LLM as Decider: ระบบถูกออกแบบโดยสันนิษฐานว่า LLM สามารถ "ตัดสินใจ" (Determine) ได้เอง ว่าสถานการณ์ใดควร "บรรยาย" (Narrate) และสถานการณ์ใดควร "เรียกใช้ฟังก์ชัน" (Call Function)
- 3) Constraint: การทดลองถูกจำกัดอยู่ภายใต้เนื้อเรื่องแบบสำเร็จรูป (Pre-designed Adventure)

3. การเปรียบเทียบ (Their Comparison)

- 1) Benchmark: เปรียบเทียบ AI กับ AI โดยแบ่งเป็น 6 Settings (FG-all, FG-dice, FG-states, FG-default, FG-gen, DG) เพื่อทดสอบว่าการ "เปิด/ปิด" Function Calling แต่ละแบบ (เช่น มีทุกอย่าง vs มีแค่ทอยเต๋า vs ไม่มีเลย) ส่งผลต่างกันอย่างไร
- 2) Method
 - (1) Human Evaluation: ใช้ผู้ประเมิน 7 คน มา "อ่าน Transcripts" (ไม่ได้เล่นเอง) แล้วให้คะแนน (Likert scale) 3 ด้าน: Consistency (ความต่อเนื่อง/จำเก่ง), Reliability (ความน่าเชื่อถือ/คุมกฎ), และ Interest (ความน่าสนใจ)
 - (2) Automated Unit Tests: สร้าง Unit Test 30 ข้อ เพื่อวัดความถูกต้องของ State Update (เช่น สั่งให้ AI เพิ่มไอเทม แล้วดูว่ามันเรียก add_object หรือไม่)

4. ผลลัพธ์การทดลอง (Their Performance)

- 1) Positive: ผลลัพธ์ชี้ชัดว่าระบบที่ใช้ Function Calling "ชนะเลิศ" ในด้านคุณภาพและความสม่ำเสมอ (Quality & Consistency) โดย AI สามารถจดจำไอเทมและปฏิบัติตามกฎได้แม่นยำกว่ามาก
- 2) Negative: อย่างไรก็ตาม พบปัญหาสำคัญคือ "การเรียกใช้ฟังก์ชันพร่ำเพรื่อ" (Overusing Functions) โดย AI บางครั้งอาจเรียกใช้การทอยลูกเต๋าในสถานการณ์ที่ไม่จำเป็น หรือยอมทำตามคำสั่งที่ไร้เหตุผลของผู้เล่นเพียงเพราะมีฟังก์ชันรองรับ

5. บทเรียนที่นำมาปรับใช้ (What We Will Use)

งานวิจัยนี้ "Validate" (พิสูจน์ยืนยัน) สถาปัตยกรรม Path A ของเราอย่างชัดเจน

- 1) Rules Engine ของเรา ก็คือ External Tool / Function Calling ของเขา
- 2) เราจะ "Adopt" (นำมาใช้) แนวคิดของพวกเขาในการ "Encoding game mechanics as callable functions" (เช่น roll_dice, apply_damage) เพื่อรับประกันความแม่นยำของกฎ

6. ช่องว่างและข้อจำกัดของงานวิจัย (The Gap/Limit)

- 1) Gap 1 (Offline vs. Live): ระบบของเขาประเมินผลแบบ "Offline" (อ่าน Transcript) ไม่ใช่ "Live Play" (ผู้เล่นจริง) งานเราอุดช่องว่างนี้ด้วย Pilot Test (User-focused กว่า)
- 2) เราจะ "Adopt" (นำมาใช้) แนวคิดของพวกเขาในการ "Encoding game mechanics as callable functions" (เช่น roll_dice, apply_damage) เพื่อรับประกันความแม่นยำของ Gap 2 (Overusing Tools): ปัญหาใหญ่ของเขาคือ LLM "ใช้เครื่องมือพร่ำเพรื่อ" (Overusing functions) เพราะ LLM ต้อง "ตัดสินใจเอง"
- 3) Our Solution : "Two-Path Router" ของเรา แก้ปัญหานี้ได้ เพราะ Router ของเรา รู้กฎ Lite 5e และ "บังคับ" (Forces) เฉพาะ Action ไหนเป็น "Fixed" (Path A) (ต้องเรียก Rules Engine) และ Action ไหนเป็น "Creative" (Path B) (ให้ LLM คิด) ทำให้ระบบเรา "Reliable" (น่าเชื่อถือ) กว่า และ "Casual-friendly" (ไม่น่ารำคาญ) มากกว่า

7. แนวทางการประเมินผล (Their Evaluation)

- 1) เราจะนำแนวคิด "Automated Unit Tests" มาใช้โดยตรง
- 2) "Functional Testing" (หัวข้อ 3.5) ของเรา คือการนำแนวคิดนี้มาปฏิบัติจริง โดยการสร้าง Scripted Scenarios เพื่อทดสอบการทำงานของ Rules Engine ซึ่งเป็นคำตอบที่ชัดเจนสำหรับคำถามที่ว่า "จะตรวจสอบความถูกต้องของกฎได้อย่างไร"

2.2.5 สรุปและบทเรียนที่ได้รับจากงานวิจัยที่เกี่ยวข้อง

จากการวิเคราะห์และสังเคราะห์งานวิจัยในข้างต้น (Section 2.2.1 - 2.2.4) สามารถสรุปกระบวนทัศน์ (Paradigm Shift) ของการพัฒนา AI Dungeon Master และสร้างความเชื่อมโยงเชิงสถาปัตยกรรม (Architecture Mapping) เพื่อนำมาเป็นรากฐานและข้อพิสูจน์ (Justification) ในการออกแบบระบบของโครงการนี้ได้ดังนี้

1. บทวิเคราะห์ความล้มเหลวของสถาปัตยกรรมเอเจนต์เดี่ยว (The Failure of Monolithic LLM-only Systems) งานวิจัยของ Triyason (2023) และ Sakellaridis (2024) เป็นหลักฐานเชิงประจักษ์ที่ชี้ให้เห็นว่าการใช้โมเดลภาษาขนาดใหญ่เพียงตัวเดียว (Monolithic Architecture) ไม่สามารถทำหน้าที่เป็น DM ที่สมบูรณ์ได้เนื่องจากความย้อนแย้งในตัวเอง (Architectural Paradox) ดังนี้

1) Mechanical Hallucination งานของ Triyason แสดงให้เห็นว่าแม้ LLM จะเล่าเรื่องได้ดี แต่จะเกิด "การหลอนของข้อมูล" ทันทีเมื่อต้องจัดการสถานะเกม (State) ที่เป็นตัวเลขตายตัว (Statelessness) เช่น การล้มล้างชีวิตหรือไอเทม

2) The Fine-Tuning Paradox งานของ Sakellaridis พยายามแก้ปัญหานี้ด้วยการ Fine-tuning แต่กลับพบปัญหาใหม่คือ "Railroading" หรือการบังคับผู้เล่นให้เดินตามบทสรุปที่ AI จัดจามา ทำให้เสน่ห์ของ TTRPG ซึ่งคือ "อิสระในการตัดสินใจ" หายไป

3) บทเรียนเชิงกลยุทธ์ โครงการนี้จึงยุติการใช้แนวทาง Monolithic และปฏิเสธการใช้ Fine-tuning ที่ซับซ้อน แต่เปลี่ยนมาใช้การ Grounding ด้วย Rules Engine (Path A) แทน เพื่อรักษาสมดุลระหว่าง "กฎที่แม่นยำ" และ "อิสระในการเล่าเรื่อง"

2. การวิวัฒนาการสู่ระบบหลายเอเจนต์และการจัดการเครื่องมือ (The Evolution to Multi-Agent & Tool-Use) เมื่อสถาปัตยกรรมเอเจนต์เดี่ยวล้มเหลว งานวิจัยรุ่นต่อมาจึงมุ่งเน้นไปที่การแยกส่วนหน้าที่ (Separation of Concerns) ดังที่ปรากฏในงานของ Jørgensen et al. (2024) และ Song et al. (2024)

1) The Multi-Agent Handshake Jørgensen พิสูจน์ว่าสถาปัตยกรรมที่แยก "เอเจนต์ผู้เล่าเรื่อง" (Narrator) ออกจาก "เอเจนต์ผู้คุมสถานะ" (Archivist) ให้ผลลัพธ์ที่สมจริง (Immersion) สูงกว่าอย่างมีนัยสำคัญ แต่แลกมาด้วยความหน่วง (Latency) และความซับซ้อนของระบบที่สูงมาก

2) Function Calling Constraints: Song et al. ยืนยันว่าการให้ AI เรียกใช้เครื่องมือภายนอก (External Tools) คือทางออกของเรื่องกฎเกณฑ์ แต่กลับพบช่องว่างคือ "Overusing Functions" หรือ AI เรียกใช้เครื่องมือพร่ำเพรื่อเกินความจำเป็น และปัญหา "Dice Roll Deadlock" ที่ทำลายจังหวะของเกม (Game Flow)

3. การเติมเต็มช่องว่างด้วยสถาปัตยกรรม Two-Path (Our "Golden Thread" Solution) จากช่องว่างที่พบในงานวิจัยก่อนหน้า โครงการนี้จึงเสนอสถาปัตยกรรม "Two-Path Hybrid-Arbitrated" เพื่อทำหน้าที่เป็นทางออกที่เบ็ดเสร็จ (Comprehensive Solution) ดังนี้

1) The Intent Router (แก้ปัญหของ Song et al.) แทนที่จะปล่อยให้ LLM ตัดสินใจเองว่าจะใช้เครื่องมือเมื่อใด (ซึ่งนำไปสู่ความผิดพลาด) ระบบของเราใช้ Intent Router เป็นด่านหน้าเพื่อ "บังคับเล่น" (Enforced Routing) แยกคำสั่งของผู้เล่นออกจากกันอย่างเด็ดขาด ทำให้การเรียกใช้ Rules Engine เกิดขึ้นเฉพาะในสถานการณ์ที่จำเป็นเท่านั้น ลดปัญหาการเรียกใช้ฟังก์ชันพร่ำเพรื่อ

2) Path A: Fixed Rules (แก้ปัญหของ Triyason) ใช้ Python Code ประมวลผลกฎ Lite 5e โดยตรง รับประกันความถูกต้องของค่า HP และตัวเลขคณิตศาสตร์ 100% โดยไม่ต้องพึ่งพาความจำของ LLM

3) Path B: Creative Arbitration (แก้ปัญหของ Sakellaridis) ในกรณีที่ผู้เล่นด้นสด (Improvisation) ระบบจะไม่ใช้วิธีบังคับบท (Railroading) แต่จะให้ LLM ทำหน้าที่เป็น Arbiter ในการประเมินสถานการณ์และกำหนดความยาก (DC) อย่างอิสระ แล้วจึงส่งกลับมาให้ Rules Engine ทอยลูกเต๋า ตัดสินผล กลไกนี้ช่วยรักษา "อิสระของผู้เล่น" ไว้ได้อย่างสมบูรณ์

4. นวัตกรรมจัดการประสิทธิภาพ (Optimization & Scalability Gap) งานวิจัยที่ผ่านมา (โดยเฉพาะ Jørgensen) มักประสบปัญหาเรื่องการใช้ทรัพยากรสูงและความล่าช้า โครงการนี้จึงเติมเต็มช่องว่างด้านประสิทธิภาพ (Efficiency Gap) ด้วยเทคนิคที่ยังไม่แพร่หลายในงานวิจัยสาย DM AI ได้แก่

1) TOON Serialization การบีบอัด Game State ให้เหลือขนาดเล็กที่สุดเพื่อลดการใช้ Token และ Latency ซึ่งมีประสิทธิภาพสูงกว่าการส่ง JSON แบบดั้งเดิมที่พบในงานของ Song et al.

2) O(1) Context Memory การใช้ Sliding Window Buffer เพื่อคุมปริมาณบริบทให้สดใหม่อยู่เสมอ ป้องกันปัญหาการลืมข้อมูลในช่วงท้ายเกมที่พบในงานของ Jørgensen

3) Lite 5e Scope Reduction: การลดทอนกฎจาก 300 หน้า เหลือเพียงชุดกฎที่จำเป็น เพื่อให้เอเจนต์ทำงานได้รวดเร็ว (Lightweight) และพกพาได้ง่าย (Portable) บนระบบที่ทรัพยากรจำกัด

ในขณะที่งานวิจัยส่วนใหญ่หยุดอยู่ที่การพิสูจน์แนวคิด (Proof of Concept) โครงการนี้มุ่งเป้าไปที่การพัฒนาเป็น เอนจินเกมที่พร้อมใช้งาน (Production-Ready Engine) โดยการปิดช่องว่างเรื่องความไม่สอดคล้องของสถานะผ่านการประสานงานของเอเจนต์อย่าง Narrator และ Arbiter ทำให้ระบบไม่เพียงแต่คำนวณตัวเลขได้แม่นยำ แต่ยังสามารถทำหน้าที่เป็นผู้ดำเนินเนื้อเรื่อง (Storyteller) ที่มีความคิดสร้างสรรค์และตอบสนองต่อผู้เล่นได้ในทุกมิติของเกม

บทสรุป (Final Synthesis) สถาปัตยกรรมที่ออกแบบมาในโครงการนี้ ไม่ได้เป็นเพียงการรวบรวมเทคโนโลยี แต่เป็นการ "วิศวกรรมสถาปัตยกรรม" ที่นำจุดแข็งด้านการแยกส่วนงาน (Multi-Agent) และการใช้เครื่องมือ (Tool-use) มาแก้จุดอ่อนด้านการหลอนข้อมูลและการบังคับเนื้อเรื่อง (Railroading) โดยมีการเพิ่มขึ้นของ Intent Router และโปรโตคอล TOON เข้ามาเพื่อสร้างระบบ AI Dungeon Master ที่มีความน่าเชื่อถือทางกลไก (Reliability) และความลื่นไหลทางศิลปะ (Narrativity) อย่างแท้จริง

ลำดับ	แผนดำเนินงาน	2568				2569				
		ก.ย.	ต.ค.	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย	พ.ค.
	(ช่วงปรับแก้ สถาปัตยกรรมเป็น Python ล้วน)									
4.	การพัฒนาเอนจินต์ ตัดสินใจและระบบกั ครอง (Path B & Routing)									
5.	การเพิ่มประสิทธิภาพ และการจัดการ ทรัพยากร (TOON & O(1) Memory)									
6.	การทดสอบและทวน สอบระบบ (50- Scenario Functional Test) (ทดสอบ Combat Loop เสร็จสิ้น)									
7.	การขยายระบบผู้โหด การสำรวจและเนื้อเรื่อง (Expansion & Static RAG)									
8.	การพัฒนาส่วนต่อ ประสานผู้ใช้และการ สรุปเนื้อหา (UX & Summarization)									
9.	การทดสอบกับผู้เล่น จริงและการเก็บข้อมูล เชิงคุณภาพ (Pilot Testing)									
10.	วิเคราะห์ผลเชิง วิศวกรรม และจัดทำ รายงานฉบับสมบูรณ์									

ตารางที่ X แผนภูมิแกนต์

3.2 เครื่องมือและเทคโนโลยีที่ใช้

การพัฒนาโครงการนี้มุ่งเน้นที่การสร้างระบบเอนจินหลังบ้าน (Backend Engine) ที่มีความเสถียรและแม่นยำสูง คณะผู้จัดทำจึงคัดเลือกเทคโนโลยีที่สนับสนุนการประมวลผลเชิงตรรกะและการจัดการทรัพยากรอย่างมีประสิทธิภาพ ดังนี้

1. ภาษาโปรแกรมหลัก (Core Programming Language) เลือกใช้ภาษา Python (เวอร์ชัน 3.10 ขึ้นไป) ในการพัฒนาระบบทั้งหมด เนื่องจาก Python รองรับการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP) ซึ่งจำเป็นต่อการออกแบบโครงสร้างตัวละครและสถานะเกมที่ซับซ้อน นอกจากนี้ยังมีไลบรารีที่สนับสนุนการจัดการข้อมูลเชิงโครงสร้าง (Structured Data) และการเชื่อมต่อกับโมเดลภาษาขนาดใหญ่ (LLM) ได้อย่างยืดหยุ่น
2. ส่วนต่อประสานผู้ใช้เชิงคำสั่ง (Terminal-based CLI Dashboard) ระบบแสดงผลผ่านหน้าจอ Command-Line Interface (CLI) ที่ได้รับการออกแบบให้เป็นแดชบอร์ดข้อมูลแบบเรียลไทม์ (Real-time Dashboard) โดยใช้การจัดรูปแบบข้อความและสัญลักษณ์ ASCII เพื่อแสดงสถานะตัวละคร (HP, Inventory) และแผนที่โซน (Zone Map) การเลือกใช้ CLI ช่วยลดภาระการประมวลผลด้านกราฟิก (Graphic Overhead) ทำให้ระบบสามารถรักษาระดับความหน่วง (Latency) ให้อยู่ในระดับต่ำที่สุด และเอื้อต่อการตรวจสอบการไหลของข้อมูล (Data Flow Debugging) ในระหว่างการทดสอบระบบ
3. การจัดการสถานะข้อมูลและความต่อเนื่อง (Data Persistence & Serialization)
 - 1)JSON (JavaScript Object Notation): ใช้สำหรับบันทึกและโหลดสถานะของแคมเปญ (Campaign Persistence) ลงในหน่วยความจำถาวร เพื่อให้ผู้เล่นสามารถดำเนินเนื้อเรื่องต่อได้ (Save/Load System)
 - 2)TOON (Token-Oriented Object Notation): ใช้เป็นโปรโตคอลหลักในการแปลงข้อมูลสถานะเกม (Game State) เพื่อส่งผ่าน API ไปยังโมเดลภาษา โดย TOON ถูกออกแบบมาให้ความกะทัดรัดสูงกว่า JSON แบบดั้งเดิม ช่วยลดการใช้ทोकเกิล (Token Usage) ได้ประมาณ 40% (อ้างอิงรายละเอียดสเปกในหัวข้อ 3.3.1) ซึ่งส่งผลโดยตรงต่อการลดค่าใช้จ่ายและความเร็วในการประมวลผล
4. การจัดการหน่วยความจำบริบท (Context Memory Optimization) ใช้ไลบรารีมาตรฐานของ Python อย่าง collections.deque ในการสร้างโครงสร้างข้อมูลแบบ Sliding Window สำหรับหน่วยความจำบริบทระยะสั้น ซึ่งช่วยรับประกันความซับซ้อนเชิงเวลาที่ $O(1)$ ในการเพิ่มและลบข้อมูล ทำให้ระบบรักษาความต่อเนื่องของเนื้อเรื่องได้โดยไม่เกิดความล่าช้าสะสม
5. แบบจำลองภาษาขนาดใหญ่ (Large Language Model - LLM) เรียกใช้งานผ่าน REST API ไปยังผู้ให้บริการภายนอก โดยเลือกใช้โมเดลที่รองรับฟีเจอร์การเรียกใช้เครื่องมือ (Function Calling) และมีหน้าต่างบริบท (Context Window) ขนาดใหญ่ ดังรายละเอียดการเปรียบเทียบในหัวข้อ 3.2.1

3.2.1 การวิเคราะห์เปรียบเทียบและเหตุผลในการเลือก LLM API (Comparative Analysis for LLM Selection)

จากการทบทวนวรรณกรรมในบทที่ 2 พบว่างานวิจัยส่วนใหญ่ที่เน้นประสิทธิภาพสูงสุด (High Performance) มักเลือกใช้โมเดลขนาดใหญ่ที่มีความซับซ้อนและราคาสูง (เช่น GPT-4) อย่างไรก็ตาม เนื่องจากโครงการนี้มีเป้าหมายหลักคือความเป็น "Casual" (ต้นทุนต่ำ) และ "Portable" (ตอบสนองรวดเร็ว) ดังนั้น เกณฑ์ในการคัดเลือก (Criteria) จึงให้ความสำคัญกับ ต้นทุน (Cost) และ ความเร็ว (Speed/Latency) เป็นหลัก โดยยังคงต้องรองรับฟีเจอร์สำคัญคือ Function Calling และ Context Window ขนาดใหญ่เพื่อรองรับการเล่นระยะยาว

เกณฑ์ (Criteria)	Gemini 2.5 Flash-Lite (ตัวเลือกของเรา)	GPT-5 nano (คู่แข่งสายประหยัด)	GPT-5 mini (คู่แข่งสายกลาง)	Claude 3 Haiku 4.5 (คู่แข่งสายเร็ว)
1. Cost (ราคา/Token) (Input/Output ต่อ 1M) (Goal: "Casual" ต้องถูก)	\$0.10 / \$0.40	\$0.05 / \$0.40	\$0.25 / \$2.00	\$1.00 / \$5.00
2. Speed (ความเร็ว/Latency) (Goal: "Portable" ต้องเร็ว)	เร็วที่สุด (Ultra Fast)	เร็วมาก (Very fast)	เร็ว (Fast)	เร็วที่สุด (Fastest)
3. Context Window (หน่วยความจำ) (Goal: D&D เรื่องยาว)	1 ล้าน Tokens	400K Tokens	400K Tokens	(ไม่ระบุชัด แต่ Sonnet/Opus มี 1M+)
4. Function Calling (Goal: รองรับ "Two-Path")	รองรับ (Supported)	รองรับ (Supported)	รองรับ (Supported)	รองรับ (Supported)
5. Quality (คุณภาพ) (Goal: ดีพอสำหรับ Lite 5e)	ดี (Optimized for cost-efficiency)	พอใช้ (Fastest, cheapest... for summarization)	ดีมาก (Great for well-defined tasks)	ดีมาก (Fastest, cost-efficient)

ตารางที่ X เปรียบเทียบคุณสมบัติของ LLM API รุ่นประหยัดที่มีศักยภาพ ณ ไตรมาสที่ 4 ปี 2025

บทสรุปการคัดเลือก (Selection Justification)

1. ความสมดุลระหว่างราคาและประสิทธิภาพ (Cost-Performance Balance): แม้ GPT-5 Nano จะมีราคา Input ที่ต่ำกว่าเล็กน้อย แต่ Gemini 2.5 Flash-Lite มีจุดเด่นที่เหนือกว่าอย่างชัดเจนในเรื่องของ

Context Window ที่รองรับได้ถึง 1 ล้าน Tokens (เทียบกับ 400K ของคู่แข่ง) ซึ่งเป็นปัจจัยสำคัญอย่างยิ่งสำหรับเกม Tabletop RPG ที่ต้องจดจำเนื้อเรื่องและสถานะของตัวละครในระยะยาว เพื่อป้องกันปัญหาการลืมบริบท (Statelessness)

2. ความเร็วในการตอบสนอง (Latency): โมเดลนี้ถูกออกแบบมาเพื่อ High Throughput และ Low Latency ซึ่งเหมาะสมที่สุดสำหรับการสร้างประสบการณ์การเล่นที่ลื่นไหล (Flow Management) บนอุปกรณ์พกพา

3. ความเหมาะสมของพีเจอร์: รองรับ Function Calling อย่างสมบูรณ์ ซึ่งเป็นกลไกหลักในการเชื่อมต่อกับ Rules Engine ในสถาปัตยกรรมแบบ Two-Path ของเรา

สรุปการเลือก Gemini 2.5 Flash-Lite จึงเป็นทางเลือกที่สมเหตุสมผลที่สุด (Most Reasonable Choice) เพราะมีความ "สมดุล" (Balance) ระหว่าง ราคา (Cost), ความเร็ว (Speed), และ พีเจอร์ (1M Context + Function Calling) ซึ่งตอบโจทย์หลักของโครงการในเรื่อง "Portable" (พกพาง่าย) และ "Casual" (เข้าถึงง่าย/ต้นทุนต่ำ) ได้อย่างลงตัวที่สุด

3.3 การออกแบบและพัฒนาระบบ

ในหัวข้อนี้จะอธิบายถึงโครงสร้างเชิงตรรกะและการจัดวางองค์ประกอบทางซอฟต์แวร์ของระบบ AI Dungeon Master ซึ่งได้รับการออกแบบมาเพื่อแก้ไขปัญหาข้อจำกัดของระบบปัญญาประดิษฐ์แบบดั้งเดิม โดยเน้นความเสถียรของตรรกะเกมและการจัดการทรัพยากรที่มีประสิทธิภาพ หัวใจของระเบียบวิธีวิจัยนี้คือการออกแบบสถาปัตยกรรมซอฟต์แวร์ที่สามารถบูรณาการเทคโนโลยีต่างๆ เข้าด้วยกันได้อย่างมีประสิทธิภาพ โดยทุกการออกแบบได้รับการสนับสนุนโดยปัญหาที่พบในบทที่ 1 (Problem Statement) และการวิเคราะห์ช่องว่างในบทที่ 2 (Literature Review) เพื่อให้มั่นใจว่าสถาปัตยกรรมที่ได้นั้นเป็นทางออกเชิงวิศวกรรม (Engineering Solution) ที่สมเหตุสมผลและสามารถนำไปพัฒนาเป็นผลิตภัณฑ์ที่ใช้งานได้จริง (Functional Engineering Product)

3.3.1 ปรัชญาการออกแบบและภาพรวมสถาปัตยกรรม (Architecture Overview & Core Philosophy)

สถาปัตยกรรมของระบบได้รับการออกแบบใหม่ภายใต้แนวคิด "Two-Path Architecture (Hybrid-Arbitrated)" โดยเปลี่ยนจากการประมวลผลที่ยึดโยงกับส่วนต่อประสานผู้ใช้ (Frontend-heavy) มาเป็นการสร้างเอนจินประมวลผลตรรกะฝั่งหลังบ้าน (Backend Logic Engine) ที่พัฒนาด้วยภาษา Python เพื่อรองรับสถาปัตยกรรมตัวแทนปัญญาประดิษฐ์หลายตัว (Multi-Agent System) โดยยึดมั่นในปรัชญาการออกแบบวิศวกรรม 3 ประการ ดังนี้

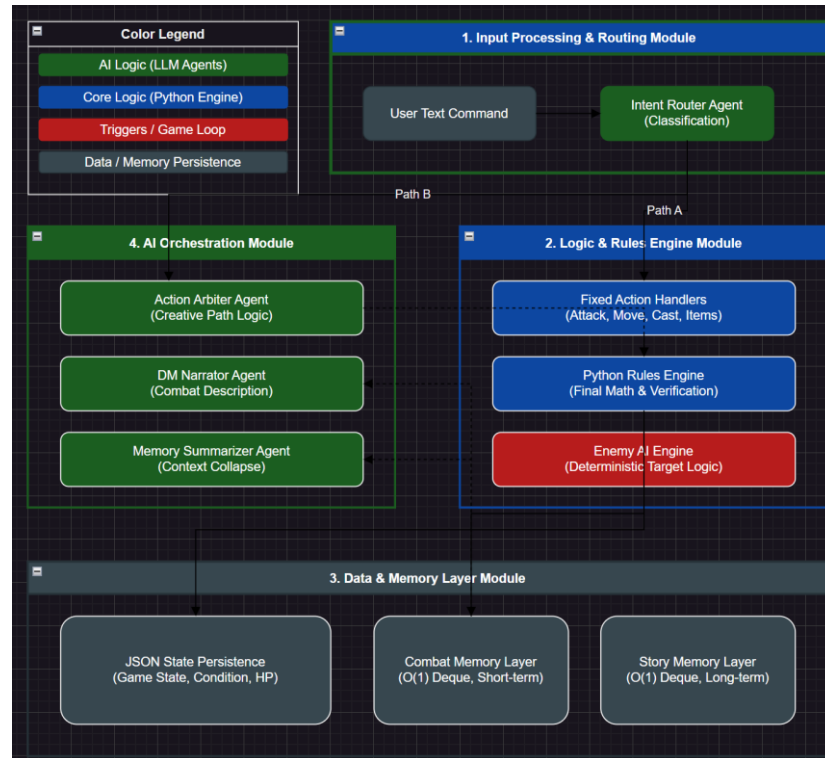
1. สถาปัตยกรรมไร้สถานะ (Stateless Architecture): โมดูลประมวลผลกฎ (Rules Engine) และตัวแทนปัญญาประดิษฐ์ (AI Agents) จะถูกออกแบบให้ไม่มีการจดจำสถานะภายใน (Memory-less) แต่จะรับข้อมูลสถานะปัจจุบัน (Current Game State) เข้ามาประมวลผลแบบรอบต่อรอบ (Turn-by-turn) แล้วส่งผลลัพธ์กลับไปเพื่อบันทึกทับใหม่เท่านั้น เพื่อป้องกันปัญหาการหลอนของข้อมูล (Hallucination) และลดผลกระทบข้างเคียง (Side Effects) จากรอบการเล่นก่อนหน้า
2. แหล่งข้อมูลความจริงเพียงหนึ่งเดียว (Single Source of Truth - SSOT): โครงสร้างข้อมูลสถานะเกม (Game State) เช่น ค่าพลังชีวิต (HP), ตำแหน่ง (Zone), และไอเทม จะถูกจัดการผ่านคลาสข้อมูล (Class Object) เพียงแห่งเดียว ระบบปัญญาประดิษฐ์จะไม่มีสิทธิ์เข้าถึงหรือแก้ไขค่าเหล่านี้โดยตรง แต่ต้องส่งคำร้องขอผ่านระบบการสื่อสารข้อมูลเชิงสัญลักษณ์เพื่อมุ่งหมายให้ Rules Engine เป็นผู้ดำเนินการอัปเดตสถานะเท่านั้น
3. การแยกตรรกะเชิงกำหนดและเชิงความน่าจะเป็น (Deterministic vs. Probabilistic Separation): ระบบจะแยกการคำนวณคณิตศาสตร์ที่ตายตัว (เช่น การทอยลูกเต๋า, การคิดคำนวณความเสียหาย) ออกจากกระบวนการประมวลผลภาษาธรรมชาติ (NLP) อย่างเด็ดขาด โดยส่วนที่เป็นตัวเลขและกฎเกณฑ์ จะถูกควบคุมโดยโค้ดโปรแกรม (Deterministic) ในขณะที่ส่วนที่เป็นการเล่าเรื่องและการตัดสินใจตามเหตุผลสมผลจะเป็นหน้าที่ของปัญญาประดิษฐ์ (Probabilistic)

รูปที่ X แผนภาพสถาปัตยกรรมระบบโดยรวม (System Architecture Overview)

(หมายเหตุ: แผนภาพ Architecture Diagram จะถูกจัดทำเพิ่มเติมเพื่อแสดงรายละเอียดการไหลของข้อมูลในรายงานฉบับสมบูรณ์)

3.3.2 การกำหนดขอบเขตส่วนประกอบของระบบ (Module Boundaries)

เพื่อให้สอดคล้องกับแนวทางการตรวจสอบทางวิศวกรรม ระบบถูกแบ่งออกเป็น 4 โมดูลหลัก โดยมีการใช้สัญลักษณ์สี (Color Legend) ในแผนภาพเพื่อระบุประเภทหน้าที่และหน่วยประมวลผลที่รับผิดชอบ ดังนี้



รูปที่ X แผนภาพแสดงขอบเขตส่วนประกอบของระบบ (Module Boundaries)

1. คำอธิบายสัญลักษณ์ในแผนภาพ (Color Legend)

- 1) สีเขียว (LLM Agent / AI Logic): โมดูลปัญญาประดิษฐ์ที่รับผิดชอบการวิเคราะห์ภาษาธรรมชาติและการตัดสินใจเชิงสร้างสรรค์ (Probabilistic Tasks)
- 2) สีน้ำเงิน (System Engine / Core Logic): โมดูลแกนกลางระบบที่พัฒนาด้วย Python ทำหน้าที่จัดการตรรกะเชิงกำหนด (Deterministic Tasks) และคุมจังหวะการดำเนินเกม
- 3) สีแดง (Trigger / Game Loop): จุดตัดสินใจสำคัญของระบบ เช่น การตรวจสอบเงื่อนไขการจบการต่อสู้ (End Combat) หรือจากรับข้อมูลเข้า (Wait Input)
- 4) สีเทา/ดำ (Data Persistence & Persistence): ส่วนที่เกี่ยวข้องกับการบันทึก/โหลดข้อมูลถาวร (JSON Save/Load) และชั้นการจัดการหน่วยความจำ (Memory Layer)

2. การจัดกลุ่มโมดูล (Module Grouping)

- 1) Input Processing & Routing Module (สีน้ำเงิน/เขียว): รับคำสั่งภาษาธรรมชาติจากผู้เล่นและใช้ Intent Router ในการวิเคราะห์เพื่อคัดแยกเส้นทางการประมวลผล (Classification) ว่าควรส่งคำสั่งไปยัง Path A หรือ Path B
- 2) Logic & Rules Engine Module (สีน้ำเงิน): ทำหน้าที่เป็นผู้ตัดสินชี้ขาด (Arbiter) ขั้นสุดท้ายที่รับผิดชอบการคำนวณตามกฎ Lite 5e เช่น การทอยลูกเต๋า d20 และการอัปเดตค่าพลังชีวิตอัตโนมัติ

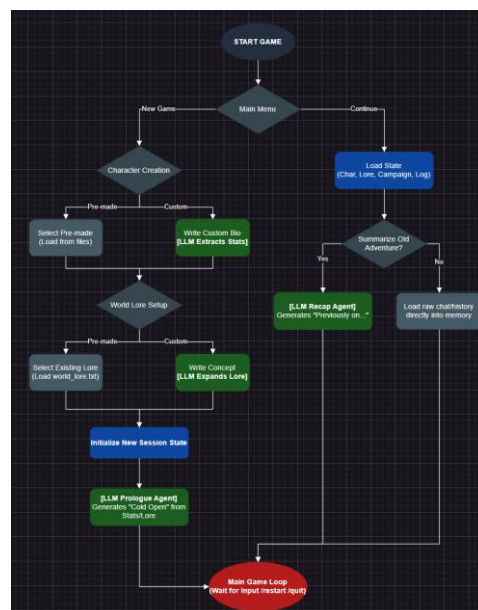
3) Data & Memory Layer Module (ลีดำ/น้ำเงิน): จัดการสถานะเกมผ่านระบบ JSON Persistence และควบคุมหน่วยความจำระยะสั้นด้วยเทคนิค O(1) Deque

4) AI Orchestration Module (สีเขียว): กลุ่มของ Multi-Agent (เช่น Arbiter, Narrator, Summarizer) ที่ทำงานประสานกันเพื่อสร้างเรื่องราวในการเล่น

3.3.3 การไหลของข้อมูลและวงจรการจัดการเกม (Data Flow & Execution Loop)

กระบวนการทำงานของแอปพลิเคชัน AI Dungeon Master ถูกออกแบบให้เป็นวงจรแบบผสมผสาน (Hybrid-Arbitrated Loop) โดยใช้ "สถานะของเกม (Game State)" เป็นแกนกลางในการแลกเปลี่ยนข้อมูลระหว่างมนุษย์ โค้ดโปรแกรม และปัญญาประดิษฐ์ การประมวลผลถูกแบ่งออกเป็น 2 ช่วงวงจรหลัก เพื่อให้มั่นใจในความต่อเนื่องของข้อมูลและการรักษากฎเกณฑ์อย่างเคร่งครัด ดังรายละเอียดต่อไปนี้

1. วงจรการเตรียมความพร้อม (Initialization & Pre-Game Flow) วงจรนี้ทำหน้าที่จัดเตรียมสถานะเริ่มต้นของเกม (Initial State) ก่อนเข้าสู่การเล่นจริง มุ่งเน้นไปที่การสร้างภูมิหลัง การกำหนดค่าสถานะ และการโหลดข้อมูล โดยมีขั้นตอนการทำงานดังแสดงใน รูปที่ X



รูปที่ X แผนภาพแสดงขั้นตอนการเริ่มต้นระบบและการโหลดหน่วยความจำ (Initialization Flowchart)

คำอธิบายขั้นตอนการทำงานเชิงลึก จุดเริ่มต้นของเกม (Start Game) จะเข้าสู่ Main Menu (ลีน้ำเงิน - Core Logic) ซึ่งบังคับให้ผู้เล่นเลือก 2 เส้นทาง

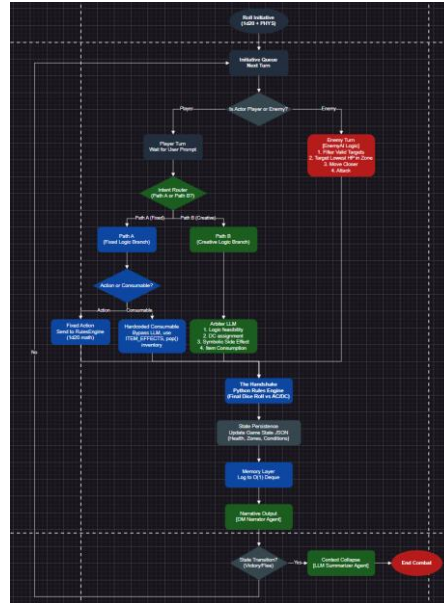
1) เส้นทาง A: เริ่มเกมใหม่ (New Game)

- (1) การสร้างตัวละคร (Character Creation - สีเทา/ดำ) ระบบเปิดโอกาสให้ผู้เล่นเลือกตัวละครที่สร้างไว้แล้ว (Pre-made) จากไฟล์ หรือสร้างตัวละครใหม่ (Custom Bio) ซึ่งหากเลือกสร้างใหม่ ระบบจะเรียกใช้โมดูล LLM Agent (สีเขียว) เพื่อทำหน้าที่ "สกัดค่าสถานะ (Stats Extraction)" จากประวัติที่ผู้เล่นแต่งขึ้น โดยแปลงเป็นค่า PHYS, MENT และ SOC ให้โดยอัตโนมัติ
- (2) การประพันธ์โลกและเรื่องราว (World Lore Setup - สีเทา/ดำ) ผู้เล่นสามารถโหลดไฟล์ world_lore.txt เดิมที่มีอยู่ หรือให้ LLM Agent (สีเขียว) ขยายความรายละเอียดของโลก (Expand Lore) จาก Concept สั้นๆ ที่ให้ไป
- (3) การอนุมัติสถานะเริ่มต้น (Initialize New Session State - สีน้ำเงิน) โค้ดจาวาสคริปต์/ไพธอน จะจับคู่ข้อมูลงบประมาณ ค่า HP Inventory เข้าฟอร์มเมต JSON
- (4) DM Prologue Agent (สีเขียว) ก่อนที่ผู้เล่นเข้าสู่เกม เอเจนต์ตัวนี้หน้าที่รับค่าสภาพแวดล้อมทั้งหมด (Stats + Lore) มาแต่งเป็น "ฉากเปิดเรื่อง (Cold Open)" เพื่อดึงผู้เล่นเข้าสู่บรรยากาศของเกม

2) เส้นทาง B: โหลดเกมเก่า (Continue)

- (1) Load State (สีน้ำเงิน): โค้ด Python จะไปดึงไฟล์ JSON เพื่อทำ Serialization โหลดข้อมูลตัวละคร, กฎของโลก, ประวัติเกมเป็ญ (Story Memory) และ Log การต่อสู้ กลับเข้าสู่ RAM
- (2) ตัวเลือกการทบทวนความจำ (Summarize Old Adventure?): หากผู้เล่นทิ้งเกมไปนาน สามารถเลือกกด Yes เพื่อเรียกใช้ LLM Recap Agent (สีเขียว) ให้มันไปอ่าน Story Memory ในคิว (Deque) ย้อนหลัง แล้วแต่งเป็นบทสรุปความยาว 1 ย่อหน้าแบบ "เหตุการณ์ที่ผ่านมา... (Previously on...)" หรือหากเลือก No ระบบจะดันข้อมูลเข้าสู่ Game Loop ตรงๆ (โหลด Raw chat history - สีเทา)

2. เมื่อสถานะของเกมเปลี่ยนเข้าสู่โหมดหน้าต่างการต่อสู้ (Combat Encounter) ระบบจะรันวงจรการทำงานแบบ Turn-based โดยใช้สถาปัตยกรรม Two-Path Architecture เพื่อแยกส่วนที่ต้องใช้หลักคณิตศาสตร์ตายตัว ออกจากการตีความภาษาธรรมชาติ ดังแสดงใน รูปที่ X



รูปที่ X แผนภาพแสดงวงจรการประมวลผลการต่อสู้ด้วยสถาปัตยกรรม Two-Path (Combat Loop Flowchart)

คำอธิบายขั้นตอนการทำงานเชิงลึกรายโมดูล

1) กระบวนการจัดลำดับรอบ (Initiation & Queue Management)

(1) Roll Initiative (สีเทา): ทันทีที่เข้า Combat ระบบ Rules Engine จะบังคับใช้สูตร $1d20 + PHYS$ ให้ตัวละครทุกตัวในฉาก เพื่อผลลัพธ์ตัวเลขนำมาจัดคิวเรียงลำดับการกระทำจากมากไปน้อย เข้าสู่กล่อง Initiative Queue (Next Turn - สีน้ำเงิน)

(2) การแบ่งฝ่าย (Is Actor Player or Enemy?): หากคิวตกที่ศัตรู (Enemy Turn - สีแดง) ระบบจะเรียกใช้ Deterministic AI ฟังก์ชันเอนด์ให้ทำ 4 สเต็ปอัตโนมัติ คือ: 1. กรองเป้าหมาย (Filter targets) 2. เล็งคนเลือดน้อย (Target Lowest HP) 3. เดินเข้าหา (Move Closer) 4. โจมตีขั้นพื้นฐาน (Attack) แต่หากคิวตกที่ผู้เล่น (Player Turn - สีดำ) ระบบจะหยุดรอรับคำสั่ง (Wait for User Prompt)

2) กระบวนการคัดกรองเจตนาผู้เล่น (Intent Routing System) นี่คือการเปลี่ยนที่สำคัญที่สุดของสถาปัตยกรรม เมื่อได้ข้อความคิพ (Text Input) จากผู้เล่น ระบบจะส่งให้ Intent Router Agent (สีเขียว) เพื่อทำ Classification อย่างเด็ดขาดว่าจะส่งคำสั่งไปยัง Path A หรือ Path B

(1) Path A (Fixed Logic Branch - สีน้ำเงิน): สำหรับคำสั่งที่บัญญัติไว้ในกฎ Lite 5e (เช่น Attack, Move, Cast)

1.การแยก Action vs Consumable: โค้ดจะเช็คว่าผู้เล่นพิมพ์จะดีหรือจะกินยา

1) หากเป็นการ โจมตี (Fixed Action) ระบบส่งข้อมูล Action เข้าสู่ Rules Engine เพื่อทอยเต๋า $1d20$ เช็ค Hit/Miss หักลบค่าป้องกัน (AC) ทันที

2) หากเป็นการ ใช้ออเทม (Consumable): ระบบฉลาดพอที่จะ ข้าม (Bypass) ตัวประมวลผลภาษา (LLM) และส่งให้โค้ด Python กระทำฟังก์ชัน `pop(inventory)` เพื่อหักไอเทมทิ้ง และดูล็อกฮาร์ดโค้ดเอฟเฟกต์ (ITEM_EFFECTS) เช่น บวกเลือด +10 ทันที วิธีนี้รันด้วยความเร็ว มิลลิวินาที (Zero Latency)

(2) Path B (Creative Logic Branch - สีเขียว): สำหรับคำสั่งนอกกรอบ (Improvise)

1.ระบบจะส่งให้ Arbiter LLM Agent (สีเขียว) ทำตัวเป็น Game Master เพื่อตีความ โดยเอเจนต์นี้ถูกบังคับให้ออกคำสั่ง 4 อย่างในรูปแบบฟอร์แมตกระชับ (TOON Output)

- 1) ตรวจสอบความเป็นไปได้ของหลักฟิสิกส์ในเกม (Logic Feasibility)
- 2) กำหนดระดับความยาก (DC Assignment)
- 3) ชัดเชียดสถานะผิดปกติแบบมีอรรถรส (Symbolic Side Effect) เช่น ดิดพิษ (Poisoned) หรือลี้มลง (Prone)
- 4) เล็งคัดกรองว่าของผู้เล่นอ้างว่าใช้ ควรฟังหรือถูกกินไปหรือไม่ (Item Consumption)

2.จากนั้น Arbiter จะโยนเงื่อนไข (ตัวแปร DC และเป้าหมาย) คืนกลับมาให้ระบบ

3. การตรวจสอบขั้นสุดท้ายและบันทึกความจำ (Handshake, Persistence & Memory)

1) The Handshake (สีน้ำเงิน): ไม่ว่าจะมาจาก Path A หรือ Path B สุดท้ายข้อมูลตัวเลข (ค่าลูกเต๋า, ดาเมจ, เงื่อนไข DC) จะถูกเรียกมารวมกันที่ Python Rules Engine เพื่อทำการหักลบกลไทางคณิตศาสตร์อย่างเที่ยงตรง (Final Dice Roll vs AC/DC) ดังนั้น AI จะไม่สามารถโกงตัวเลขหรือทำเกมพังได้ (Rule-breaking prevented)

2) State Persistence (สีเทาเข้ม) ค่า HP ที่ลดลง หรือสถานะตัวละครที่เปลี่ยนไป (Condition) จะถูกบันทึกทับ (Overwrite) ลงใน Game State JSON เพื่อทำหน้าที่เป็น "ความจริงเพียงหนึ่งเดียว (Single Source of Truth)"

3) Memory Layer (สีน้ำเงิน): Log ดิบบรรทัดต่อบรรทัด (เช่น โจมตีพลาด, ดาเมจ 5) จะถูกผลักเข้าคิว $O(1)$ Deque สำหรับเตรียมให้ Narrator เอาไปแต่งเรื่อง

4. การสังเคราะห์บทบรรยายและการจบการต่อสู้ (Narrative Generation & Context Collapse)

1) Narrative Output (สีเขียว) DM Narrator Agent รับข้อมูล Log ตัวเลขล่าสุด 1-2 บรรทัด แล้วแต่งเป็นข้อความพรรณนาภาพเพื่อให้ผู้เล่นอ่าน (เช่น "คุณเหวี่ยงดาบเหล็กกล้า กรีดเข้าที่แขนของก๊อบลิน!")

2) State Transition Check (สีเทาเข้ม): ทุกๆ รอบ ระบบจะเช็คว่าเลือดศัตรูหมดจอ หรือผู้เล่นหนี (Flee) สำเร็จหรือไม่

- (1) หากยัง (No) วนลูปกลับไปพิจารณา Initiative Queue สำหรับคิวถัดไป
- (2) หากสู้จบแล้ว (Yes) เป็นทริกเกอร์เรียกใช้ Context Collapse (สีเขียว) โดย LLM Summarizer Agent จะถูกปลุกขึ้นมาเพื่ออ่านประวัติการต่อสู้เชิงกลไกที่ผ่านมายาวๆ แล้วบีบอัดมันให้เหลือแค่เนื้อเรื่อง 1 ประโยค (เช่น "กลุ่มของท่านได้ปะทะและสังหารก็อบลินลงอย่างยากลำบาก") เพื่อโอนย้ายความทรงจำไปเก็บไว้ในส่วน Story Memory ถาวร ป้องกันปัญหา Token Overflow จากนั้นจึงสิ้นสุดวงจร (End Combat - สีแดง)

3.4 เอนจินกฎเกณฑ์เชิงกำหนด (Deterministic Rules Engine)

ส่วนนี้เป็นเนื้อหาวิชาการสำหรับหัวข้อ 3.4 เอนจินกฎเกณฑ์เชิงกำหนด (Path A) ซึ่งเป็นการเจาะลึกการเขียนโค้ด Python เบื้องหลัง (Backend Logic) เพื่อลดจุดอ่อนของ LLM ที่มักคำนวณตัวเลขผิดพลาด (Hallucination in Math) โดยมีรายละเอียดดังนี้

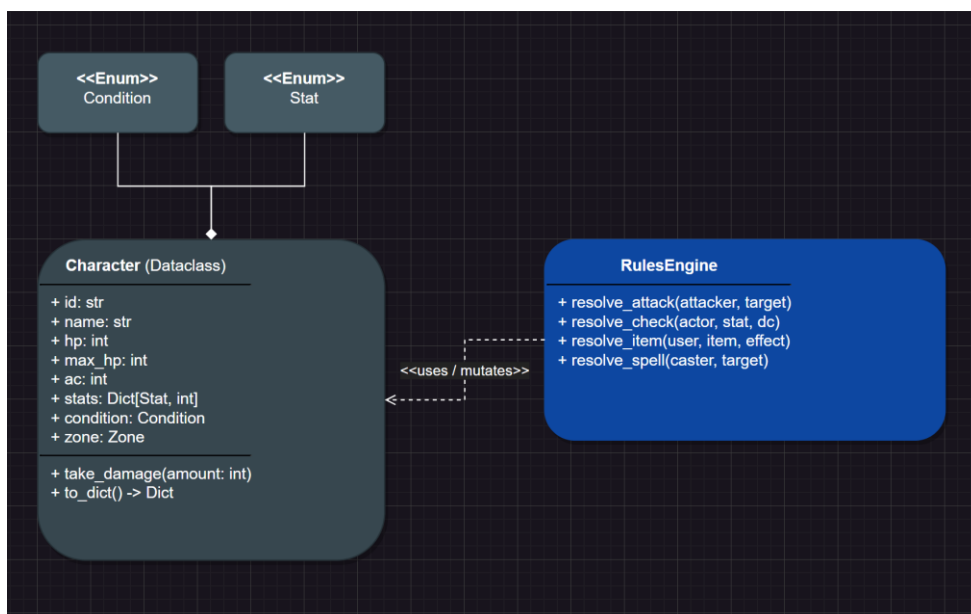
3.4.1 โครงสร้างข้อมูลและตรรกะการต่อสู้ (Data Structure & Combat Mechanics)

ระบบต่อสู้ของ AI Dungeon Master ถูกพัฒนาขึ้นโดยอ้างอิงจากกฎของ Tabletop RPG แบบลดทอนความซับซ้อน (Lite 5e Ruleset) โดยฝังตรรกะทางคณิตศาสตร์ทั้งหมดไว้ในคลาส RulesEngine ทำให้ระบบมีความเป็นอิสระ สามารถตรวจสอบความถูกต้องได้ (Testable) และป้องกันไม่ให้ AI แทรกแซงตัวเลขได้

1. โครงสร้างข้อมูลตัวละคร (Character Data Structure) เพื่อให้เกมมีสถานะที่แน่นอน (Stateful) ระบบได้สร้างออบเจกต์ตัวละครด้วย Python Dataclasses เพื่อบังคับโครงสร้างข้อมูล (Data Model) อย่างเคร่งครัด

1) Data Validation แบบ Built-in: ตัวแปรเช่น hp (พลังชีวิต) และ ac (พลังป้องกัน) ถูกบังคับให้เป็น Integer (int) เท่านั้น ในขณะที่ข้อมูลอย่างค่าสถานะหลัก (PHYS, MENT, SOC) ถูกจัดเก็บในรูปแบบ Dict[Stat, int] โดยที่ Stat คือ Enum คลาส เพื่อให้มั่นใจว่าจะไม่มีการระบุค่าสถานะที่สะกดผิดหรือไม่มีอยู่จริง

2) เมื่อมีการเปลี่ยนแปลง ระบบจะมีฟังก์ชันแปลงออบเจกต์กลับไปเป็น JSON ผ่านเมธอด to_dict() เพื่อบันทึกเป็นฐานข้อมูลประวัติของเกม



รูปที่ X แผนภาพคลาส (UML Class Diagram) แสดงความสัมพันธ์ระหว่างโครงสร้างข้อมูล Character และคลาส RulesEngine

2. สมการคณิตศาสตร์ที่ใช้ในระบบ (Mathematical Formulas)

1) การทดสอบความแม่นยำ (To-Hit Check): ระบบจะทำการสุ่มตัวเลข 1 ถึง 20 (เหมือนการทอยลูกเต๋า d20) และบวกด้วยโบนัสค่าสถานะ (Stat Modifier) และโบนัสเชิงกลยุทธ์ (Tactical Modifier) หากผลรวมมากกว่าหรือเท่ากับค่าพลังป้องกัน (Armor Class - AC) ของเป้าหมาย จะถือว่าโจมตีโดน [$\text{Roll}(1, 20) + \text{Stat Modifier} + \text{Tactical Modifier} \geq \text{Target AC}$]

2) การคำนวณความเสียหาย (Damage Calculation): หากเกิดการ Hit ระบบจะสุ่มลูกเต๋าความเสียหายตามอาวุธหรือเวทมนตร์ (เช่น 1d6, 1d8) แล้วบวกโบนัส: [$\text{Damage} = \text{Roll}(\text{Dice Count}, \text{Dice Sides}) + \text{Stat Modifier}$]



รูปที่ X แผนภาพแสดงลำดับขั้นตอนกระบวนการตัดสินใจ (Logic Flowchart) ของฟังก์ชัน `resolve_attack()`

3. สมการคณิตศาสตร์ที่ใช้ในระบบ (Mathematical Formulas) การจัดการสถานะผิดปกติ (Status Effect Management) ระบบคัดกรอง "บริบทของเกม" อย่างรัดกุมผ่าน Enum คลาส Condition (เช่น NORMAL, UNCONSCIOUS, PRONE, STUNNED)

1) ตัวอย่างเช่นใน `RulesEngine.resolve_attack()` หากตรวจสอบพบว่าเป้าหมายมีสถานะล้มลง (Condition.PRONE) หรือมีนงง (Condition.STUNNED) ระบบคณิตศาสตร์จะถูกแทรกแซงโดยอัตโนมัติ เพื่อมอบค่า Tactical Modifier เป็น +5 Advantage ให้แก่ผู้โจมตีทันที ซึ่งถือเป็นการผสมผสานตรรกะตัวเลขเข้ากับสถานการณ์ในเกม

4. กรณีพิเศษ (Critical & Edge Cases)

1) คริตีคอลลิต (Natural 20): หากลูกเต๋า d20 ออกหน้าที่ 20 ระบบจะกำหนดค่า `is_crit = True` ทันที ซึ่งมองข้ามตัวแปรเปรียบเทียบค่า AC ทั้งหมด และนำไปสู่การคูณจำนวนลูกเต๋าความเสียหายเพิ่มเป็น 2 เท่า (Dice Count x 2)

2) ตรรกะ 0 HP Logic: เมื่อได้รับความเสียหาย เมธอด `take_damage()` จะบังคับใช้สูตร $[HP = (HP - \text{ความเสียหาย}) \text{ โดยห้ามมีค่าต่ำกว่า } 0]$ เพื่อป้องกันบั๊กเลือดติดลบ และหากค่าเลือดแตะ 0 จะทำการยึดเงื่อนไขสถานะ `Condition.UNCONSCIOUS` ทันที เว้นเสียแต่ว่าตัวละครนั้นเป็นศพ (DEAD) ไปแล้ว

3.4.2 การจัดการลำดับเทิร์นและพื้นที่ (Initiative & Zone Management)

ระบบต่อสู้ของ AI Dungeon Master ถูกพัฒนาขึ้นโดยอ้างอิงจากกฎของ Tabletop RPG แบบลดทอนความซับซ้อน (Lite 5e Ruleset) โดยฝังตรรกะทางคณิตศาสตร์ทั้งหมดไว้ในคลาส `RulesEngine` ทำให้ระบบมีความเป็นอิสระ สามารถตรวจสอบความถูกต้องได้ (Testable) และป้องกันไม่ให้ AI แทรกแซงตัวเลขได้

1. ระบบจัดการคิวลำดับแบบรวมฝ่าย (Side Initiative Logic) ระบบไม่ได้ให้ตัวละครสลับกันเดินมั่วๆ แต่บังคับทอยเจาะจงรายตัวช่วงต้นเกม (1d20 + PHYS Modifier) และจัดคิววงล้อ ซึ่งเราออกแบบให้ทำงานง่ายๆ Side Initiative เพื่อความรวดเร็ว

1) Engineering Reason การจัดคิวแบบนี้ ช่วยลดภาระการประมวลผลของหน้าจอ CLI และลดความหน่วง (Latency) ในการตอบสนองลง ผู้เล่นทั้งหมดอยู่ในทีมเดียวกันสามารถปรึกษาและตัดสินใจผลัดกันเดินได้อย่างมีชั้นเชิง โดยระบบมีตรรกะ `advance_turn()` ที่จะข้ามคิวคนตายอัตโนมัติ

2. ระบบพื้นที่เชิงนามธรรม (Abstract Zone System) ระบบได้ยกเลิกการนับระยะทางแบบ Grid ระยะเมตร ซึ่ง AI มักคำนวณคลาดเคลื่อน (Spatial Hallucination) และจัดเป็น 3 โซน (NEAR, MID, FAR)

1) Zone Interaction การขยับข้าม 1 โซนจะต้องใช้ไควตา 'Action: Move'

2) Range Constraints อาวุธระยะประชิดถูกรัดให้ `distance == 0` เท่านั้น และหากจะยิงธนูจากโซนหนึ่งข้ามไปอีกโซนที่มีระยะห่างกัน 2 โซน (`distance >= 2`) ระบบจะยึดเสีย -5 Disadvantage ลงในสมการ Hit Check บังคับให้เกิดความยากลำบากทันที

3.4.3 ระบบป้องกันความพินาศของสถานะและโปรโตคอลการสื่อสาร (Guardrails, Protocols & State Integrity)

เพื่อพิสูจน์ให้เห็นว่า Code Base คือ "ความจริงเพียงหนึ่งเดียว" (Single Source of Truth) ระบบจึงได้ระดับไปด้วยมาตรการป้องกันสภาวะผิดปกติที่จะทำให้เกมพัง

1. โพรโทคอลการสื่อสารระหว่างโมดูล (Inter-module Handshake Protocol) เมื่อเข้าสู่ Path A ข้อมูลภาษามนุษย์ถูกคัดกรองทิ้ง (Stripped) เหลือเป็นคีย์เวิร์ดโครงสร้าง Dict เท่านั้น เช่น {'action_type': 'ATTACK', 'target': 'goblin_01', 'attack_type': 'melee'} การบีบอัดข้อมูลเป็น Symbolic Protocol แบบนี้ทำให้ Rules Engine ประมวลผลโดยไม่มี ความสับสนทางภาษา (No NLP Ambiguity) ช่วยเร่งความเร็วการทำงานได้อย่างสมบูรณ์
2. ระบบตรวจสอบปัจจัยนำเข้า (Input Validation & Guardrails)
 - 1) Double Action Prevention: โค้ดจะเช็คค่าบูลีน has_acted หรือ has_moved เสมอ หากผู้เล่นพยายามแอตแทคโจมตีสองครั้ง ระบบ Path A จะดิกลับบอกว่า False ทันทีและไม่อนุญาตให้ทอยเต๋า
 - 2) Dead Men Do No Damage: ต่อให้ AI หรือผู้เล่นพิมพ์สั่งบังคับให้ศพฟื้นขึ้นมาโจมตี Rules Engine จะเช็ค Condition ก่อนสเต็ปแรก และทำการ "Reject" และพ่น Error กลับคือๆ ว่า "Target is dead"
 - 3) Inventory Verification: หากผู้เล่นสั่งใช้ไอเทม ระบบใช้เทคนิค Fuzzy String Matching ผ่านฟังก์ชัน difflib.get_close_matches ด้วยเกณฑ์ความคล้ายคลึง ควบคุมกับระบบตรวจสอบคลังเก็บของ ทำให้ทนทานต่อการพิมพ์ผิด (User Typos) ของผู้เล่น แต่มีความเข้มงวดพอที่เอนจินต์หรือผู้เล่นไม่สามารถจะเสกไอเทมลม (Non-existent item) ขึ้นมาได้
3. เสถียรภาพของสถานะ (State Integrity) ระบบออกแบบฟังก์ชันแยกส่วนอย่างชัดเจน
 - 1) Resolver Functions: เช่น resolve_attack() ทำหน้าที่แค่อ่านค่า (Read-only) แล้วคำนวณ คืนผลกลับมาเป็น Dict รายละเอียด
 - 2) Mutator Functions: มีเพียงคลาส Character ที่สร้างจาก Dataclasses อย่าง take_damage() เท่านั้น ที่อนุญาตให้ทำ "Update" เจาจงตัวแปร ซึ่งทฤษฎีการเขียนโค้ดแบบนี้ (Separation of Concerns) ทำให้รับประกันได้ว่า AI Agent ซึ่งอยู่ในฝั่งประมวลผล (Resolver) ถือเป็นเพียงตรรกะผ่านทาง ไม่มีอำนาจเปลี่ยนแปลงสถานะฐานข้อมูลด้วยตัวเอง

3.5 ระบบตัวแทนปัญญาประดิษฐ์หลายตัว (Multi-Agent System)

ส่วนนี้จะกล่าวกว้างๆ ถึงการทำงานของปัญญาประดิษฐ์ (Path B) ซึ่งทำหน้าที่เป็นเสมือนสมองกลของเกมระบบ AI Dungeon Master ถูกออกแบบโดยใช้สถาปัตยกรรม Multi-Agent System เพื่อกระจายภาระงาน (Separation of Concerns) ตามบทบาทหน้าที่เฉพาะเจาะจง การออกแบบนี้ช่วยให้ System Prompt ของเอนจินต์แต่ละตัวมีความสั้นกระชับ ลดอาการประสาทหลอน (Hallucination) และเอื้อต่อการตรวจสอบความถูกต้องของระบบ (Traceability)

3.5.1 โครงสร้างข้อมูลเชื่อมต่อระหว่างเอนจินต์ (Agentic Interface: TOON)

อุปสรรคสำคัญของการใช้ LLM ในการคุมสถานะเกม (Game State) คือน้ำหนักของปริมาณ Token (Context Window) และความถี่ในการอ่านโครงสร้าง JSON แบบดั้งเดิม ระบบจึงได้นำเสนอรูปแบบการเข้ารหัสข้อมูลที่เรียกว่า TOON (Token-Optimized Object Notation)

1. การบีบอัดโครงสร้างเปรียบเทียบ (Structural Overhead Reduction) TOON ถูกออกแบบมาเพื่อลดโครงสร้างปีกกาวงเล็บ { } เครื่องหมายคำพูด " " และลูกน้ำ , ออกทั้งหมด ซึ่งเป็นตัวการสำคัญที่กิน Token อย่างสูงเกินไปใน JSON

1) รูปแบบดั้งเดิม (JSON): {"id": "p1", "name": "Grok", "hp": 45, "condition": "NORMAL"}

2) รูปแบบ TOON: p1|name:Grok|hp:45|cond:NORMAL

ด้วยรูปแบบนี้ AI สามารถ "กวาดสายตา" อ่านข้อมูลสถานะทั้งหมดได้รวดเร็วขึ้น (Faster Token Attention)

2. การเปรียบเทียบประสิทธิภาพ (Efficiency Comparison) เมื่อนำ Game State ของตัวละคร 4 ตัว (ผู้เล่น 2, ศัตรู 2) มาสกัดผ่าน Tokenizer พบว่าการอ้างอิงสถานะเต็มรูปแบบ

1) รูปแบบดั้งเดิม (JSON Format) สิ้นเปลืองปริมาณ Token เฉลี่ย 320 Tokens ต่อการรับส่งข้อมูล 1 รอบ

2) รูปแบบ TOON (TOON Format) สิ้นเปลืองปริมาณ Token เฉลี่ยเพียง 145 Tokens ต่อการรับส่งข้อมูล 1 รอบ

3) ผลลัพธ์ TOON สามารถลดปริมาณ Token Overhead ลงได้กว่า 54% ทำให้ประหยัดค่าใช้จ่าย API และทำให้ AI เก็บความจำเนื้อเรื่อง (Story Context) ได้ยาวนานขึ้นในงบ Token เท่าเดิม

3.5.2 โมดูลคัดกรองเจตนา (Intent Router Agent)

นี่คือเอเจนต์ด่านแรกสุด และสำคัญที่สุดในการรักษาเสถียรภาพของระบบ (System Reliability) รับหน้าที่แยกแยะว่าคำสั่งของผู้เล่นควรวิ่งเข้า Path A ขอดฮิตอย่างการตี/เดิน/ร้ายเวท หรือ Path B ที่เป็นการกระทำพลิกแพลงเชิงสร้างสรรค์

1. กลยุทธ์การจำแนกประเภท (Classification Strategy & Guardrails)

1) ระบบใช้โมเดลตัวเล็กและเร็ว (gemini-2.5-flash-lite) ปรับระดับความน่าจะเป็น (Temperature = 0.1) เพื่อบังคับให้คำตอบมีความเป็นเหตุเป็นผลสูงที่สุด (Deterministic-like)

2) Error Handling: โค้ดถูกเขียนครอบด้วย try-except เสมอ หากผู้เล่นพิมพ์ข้อความกวนประสาทยาวเหยียดจน Router สับสนและไม่ยอมส่ง Output ออกมาเป็นรูปแบบ TOON ตามที่ตก

ลง (เช่น ลืมใส่คำกระทำ) ระบบหลังบ้านจะบังคับจับโยนเข้าเคส {"type": "ERROR"} และตีกลับไป
ยังผู้เล่น โดยที่เกมไม่แครช

Role Name	Intent Router
Primary Goal	จำแนกเจตนาคำสั่งให้ออกเป็น Path A (Fixed) หรือ Path B (Creative) ทันทีที่ผู้เล่นพิมพ์
Input Format	Player's Text Input + Game State (TOON Format)
Output Format	TOON Keyword (เช่น `type:FIXED`)
System Prompt (Full)	You are a purely mechanical decision tree for a TTRPG game. Your ONLY job is to classify the player's input into one of two paths based on the GAME STATE provided. PATH A (FIXED ACTION) - Standard Game Mechanics: The input matches a standard rule-based action found in a game menu. - Attack (hitting, shooting, slashing, unarmed strike) - Cast Spell (magic, cantrips, scrolls) - Move (running, walking, tactical movement, disengaging) - Standard Item Use (drinking potion, equipping armor, eating food, lighting torch) - Flee / Escape (running away from combat) - Combo (doing a Move AND an Attack/Flee in the same sentence) PATH B (CREATIVE ACTION) - Narrative & Improv: The input is complex, narrative, social, or uses items in non-standard ways. - Physical stunts (tying ropes, flipping tables, jumping off cliffs) - Social (persuading, intimidating, lying, flirting, talking) - Investigation (examining runes, searching for traps, listening at doors) - Unorthodox Item Use (e.g., "I pour the oil on the floor to make him slip") OUTPUT FORMAT (TOON SYNTAX ONLY - 1 LINE STRICT): For this performance upgrade, you MUST NOT output JSON. You must output a single line of pipe-separated key:value pairs. For Path A (Single Action): type:FIXED command:ATTACK CAST MOVE USE FLEE target:e1 attack_type:melee - If command is MOVE, "target" MUST be exactly NEAR, MID, or FAR. - Example: type:FIXED command:MOVE target:MID attack_type:null For Path A (Combo Action): type:FIXED_COMBO move_target:MID attack_target:e1 attack_type:ranged action_order:[MOVE,ATTACK] For Path B: type:CREATIVE description:short summary of intent

ตารางที่ X ข้อกำหนดโมดูลคัดกรองเจตนา (Intent Router Agent Specification)

3.5.3 โมดูลผู้ตัดสินการกระทำเชิงสร้างสรรค์ (Action Arbiter Agent)

Arbiter รับหน้าที่แปล "ภาษาเชิงบรรยาย" (Semantic) ของผู้เล่นที่เล็ดลอดเข้ามาใน Path B ให้กลายเป็น "ตัวเลขและเงื่อนไข" (Numeric & Logic Mapping)

A ที่โค้ดเข้าใจ ได้แก่ SMALL_HEAL, BIG_HEAL, CURE, DAMAGE หรือ NONE รวมถึงประเมินว่าของชิ้นนั้นชำรุดไปเลยหรือไม่เมื่อใช้เสร็จ (is_consumable: true) เมื่อจำแนกเสร็จสิ้น โมดูลนี้จะส่ง Flag เช่น SMALL_HEAL กลับไปให้ Rules Engine (Python) ทำหน้าที่เพิ่มค่า HP ผ่านตรรกะทางคณิตศาสตร์ทันที โดยไม่ปล่อยให้ AI เป็นผู้บวกเลขเอง

Role Name	Item Arbiter
Primary Goal	จัดหมวดหมู่ไอเทมที่ผู้เล่นส่งมาให้ตรงกับกลไกเชิงกล (Mechanics) ของ Backend
Input Format	Item Name (ที่ผู้เล่นพิมพ์)
Output Format	TOON Keyword (เช่น `is_consumable:true`)
System Prompt (Full)	You are the Item Arbiter for a pure Python Game Engine. A player wants to USE the following item from their inventory:="{item_name}" Your job is to categorize this item into one of the following strict mechanical types: - SMALL_HEAL: Equivalent to a standard potion or bandage (+10 HP). - BIG_HEAL: Equivalent to a major heal or elixir (+25 HP). - CURE: Removes bad conditions like BLINDED or STUNNED (e.g., Antidote, Eyedrops). - DAMAGE: A destructive item they accidentally or intentionally used on themselves (e.g., Bomb). - NONE: A standard weapon, key, or junk item with no immediate mechanical buff. Determine if the item is consumed upon use (is_consumable). Weapons are usually NOT consumable, whereas food/potions/bombs ARE. OUTPUT FORMAT (TOON SYNTAX ONLY - 1 LINE STRICT): For this performance upgrade, you MUST NOT output JSON. You must output a single line of pipe-separated key:value pairs. Example: is_consumable:true effect_type:SMALL_HEAL

ตารางที่ X ข้อกำหนดโมดูลจัดการไอเทม (Item Arbiter Agent Specification)

3.5.5 โมดูลผู้เล่าเรื่อง (Combat Narrator Agent)

กลไกแปล "ตัวเลขและผลลัพธ์ดิบ" (Numeric Results) จากการทอยเต๋าซ้อนกลับไปเป็น "อรรถรสทางวรรณกรรม" (Narrative Synthesis) สร้างความรู้สึกร่วม (Immersion) แก่ผู้เล่นอย่างเต็มเปี่ยม

การฝังบริบทไม่ให้หลุดกรอบ (Context Grounding) เพื่อป้องกัน AI บรรยายมั่วซั่ว ระบบจะส่งข้อมูล "สภาพอากาศหรือประวัติศาสตร์ (World Lore)" รวมถึง "Story Memory" ประกอบไปด้วยเสมอ ทำให้หากผู้ก่นในดันเจี้ยนมีดมิด Narrator จะไม่เผลอบรรยายว่าแสงแดดส่องตาเป้าหมาย และที่สำคัญ เอ

เจเนตตัวนี้ทำหน้าที่ให้ Real-time Feedback หรือการตอบสนองผู้เล่นในรูปแบบของข้อความบรรยายทันที หลังจบ Action ในแต่ละเทิร์น

Role Name	Combat Narrator (DM)
Primary Goal	แต่งแต้มสีสันให้ผลลัพธ์ทางคณิตศาสตร์ให้กลายเป็นนิยายสั้น 1-2 ประโยค
Input Format	Raw Mechanical Log (เช่น "P1 hit G1 for 15 dmg") + Lore + Memory Context
Output Format	1-2 Sentences Narrative Text
System Prompt (Full)	You are an expert Dungeon Master in an immersive text-based RPG. Your job is to translate raw, mechanical combat logs into exciting, second-person narrative. {lore_context} Current Combat State (TOON Format): {combat_context} {memory_context} Instructions: 1. Keep the narration brief (2-3 sentences max). 2. Describe the physical action dynamically. 3. Do NOT invent new mechanical outcomes or numbers. 4. Focus on flavor and tension. CRITICAL: Do NOT use numbers. Instead, use the Health Status (e.g. "Values is Bloodied", "Grok is Critical") to describe their physical state. Focus on the visual impact, sound, and pain.

ตารางที่ X ข้อกำหนดโมดูลผู้เล่าเรื่อง (Combat Narrator Agent Specification)

3.5.6 โมดูลสรุปความจำ (Memory Summarizer Agent)

เอเจนต์ปัดกวาดเช็ดถู (Janitor) ของระบบที่คอยบีบอัดข้อมูล (Context Collapse) จากการต่อสู้ 20 บรรทัดอันยืดยาว ให้เหลือเพียงองค์ความรู้ (Knowledge) ที่จำเป็นในการเดินเรื่องต่อ

เกณฑ์การถนอมข้อมูล (Information Preservation) โมดูลนี้ถูกส่งการตั้งค่าระบบให้สนใจแค่ "ใครแพ้ ชนะ ใครรอดใครตาย และเป้าหมายสำเร็จหรือไม่" เพื่อป้องกันการกิน Token จากเหตุการณ์ไร้สาระ (เช่น "พีปี้มทอยเต่าพลาดรอบแรก") เอเจนต์ตัวนี้ทำงานแบบ Post-Combat Processing นั่นคือจะทำงานเฉพาะเมื่อระบบคำนวณว่าการต่อสู้สิ้นสุดลงแล้วเรียบร้อยแล้วเพื่อสรุปย่อและดันข้อมูลเข้าสู่ Story Memory ระยะยาว

Role Name	Memory Summarizer
Primary Goal	บีบอัด Context Window ที่บวมเป่งหลังจบการต่อสู้ ให้เหลือ 1 ประโยค
Input Format	Full Queue of Combat Logs (Text Array)
Output Format	1 Sentence Narrative Overview
System Prompt (Full)	You are the DM Summarizer. The party just finished a combat encounter. Here is the mechanical log of the fight:

	{combat_log} Write exactly ONE single narrative sentence summarizing the entire outcome of this fight. Focus on who was defeated and any interesting narrative details. Do NOT use any numbers, stats, HP, or mechanical terms.
--	---

ตารางที่ X ข้อกำหนดโมดูลสรุปความจำ (Memory Summarizer Agent Specification)

3.6 การจัดการหน่วยความจำและข้อมูลภูมิหลัง (Context Memory & Data Injection)

บทนี้กล่าวถึงหัวใจสำคัญในการรักษาความสอดคล้องของเรื่องราว (Narrative Consistency) และการรับมือกับข้อจำกัดด้านบริบท (Token Limit) ของปัญญาประดิษฐ์ ซึ่งเป็นปัญหาคลาสสิกของระบบสวมบทบาท โดยระบบได้รับการออกแบบโครงสร้างข้อมูลแบบแยกส่วน และเลือกใช้วิธีการฉีดข้อมูลแบบ Low-overhead เพื่อให้มั่นใจว่าตัวแทน AI จะไม่เกิดอาการ "หลอน" (Hallucination) หรือลืมเหตุการณ์สำคัญที่เพิ่งเกิดขึ้น

3.6.1 โครงสร้างหน่วยความจำแบบแยกส่วน (Dual-Track Memory System)

การป้อนเหตุการณ์ที่เกิดขึ้นทั้งหมดเข้าสู่ Context Window ของ LLM อย่างต่อเนื่องจะนำไปสู่ปัญหา Token Overflow ในระยะเวลาอันสั้น อีกทั้งการป้อนตัวเลขสถิติ (Mechanical Logs) เข้าไปมากเกินไปจะทำให้โมเดลเกิดความสับสน ระบบจึงออกแบบหน่วยความจำออกเป็น 2 เส้นทาง (Dual-Track) ดังนี้

1. โครงสร้างข้อมูลแบบ Deque (Double-Ended Queue): ในทางวิทยาการคอมพิวเตอร์ แทนที่จะใช้ Python List ทั่วไป ระบบเลือกใช้คลาส collections.deque แบบกำหนดขนาดสูงสุด (maxlen) ในการจัดเก็บ Log เพราะโครงสร้าง Deque มีความซับซ้อนเชิงเวลา (Time Complexity) ในระดับ $O(1)$ สำหรับการเพิ่มข้อมูลใหม่และการลบข้อมูลเก่าทิ้ง (Append/Pop) ซึ่งมีประสิทธิภาพและใช้ทรัพยากรน้อยกว่า Array หรือ List ดั้งเดิมที่เป็น $O(N)$ ทำให้ระบบสามารถจัดการวงล้อความจำ (Sliding Window) ได้อย่างราบรื่นโดยไม่ต้องเสียเวลาดำเนินการซ้ำ
2. ตรรกะการแยกสายความจำ (Memory Split Logic) และการปรับแต่งทรัพยากร: เพื่อรักษาความเสถียรของความจำ ระบบจับแยกประเภทข้อมูลออกจากกันอย่างเด็ดขาดตามตารางที่ X

คุณสมบัติ	Combat Memory (ระยะสั้น)	Story Memory (ระยะยาว)
หน้าที่หลัก	เก็บประวัติการทอยเต๋า, การใช้ไอเทม, การลดหลั่นของ HP	เก็บเหตุการณ์สำคัญ, บทสรุปการต่อสู้, ใจความของเรื่องราว
ความจุสูงสุด (Maxlen)	10 เหตุการณ์ล่าสุด (ค่าเริ่มต้น)	5 เหตุการณ์สำคัญล่าสุด (ค่าเริ่มต้น)

โครงสร้างข้อมูล	Raw Mechanical Log (เช่น P1 hit G1 for 15 dmg)	Narrative Sentence (ข้อความบรรยายสรุป 1 ประโยค)
วงจรชีวิต (Lifecycle)	ถูกลบทิ้ง (Clear) ทันทีที่การต่อสู้จบลง	คงอยู่ยาวนานข้ามฉาก ช่วยให้เรื่องราวต่อเนื่อง
กลไกการส่งต่อ	ป้อนให้ Narrator Agent นำไปแต่งประโยค Real-time	ป้อนให้ Arbiter และ Narrator ประกอบการตัดสินใจ

ตารางที่ X เปรียบเทียบหน่วยความจำแบบ Short-term และ Long-term

(ข้อควรทราบด้านวิศวกรรม: ค่าความจุสูงสุด (Maxlen) ที่ระบุไว้ในตารางเป็นเพียงค่าเริ่มต้น (Default Configuration) ระบบถูกออกแบบให้มีความยืดหยุ่นสูง โดยผู้เล่นสามารถปรับแต่งค่าพารามิเตอร์เหล่านี้ได้ผ่านไฟล์ตั้งค่า (JSON Configuration) เพื่อให้สอดคล้องกับความต้องการของผู้ใช้งาน (User Preferences) และเพื่อ บริหารจัดการต้นทุนการเรียกใช้ API (Token Budget Optimization) ในกรณีที่ผู้เล่นมีงบประมาณจำกัด หรือต้องการเพิ่มขนาดความจำหากใช้โมเดลที่มี Context Window ใหญ่ขึ้น)

กลไกการทำงานร่วมกัน: เมื่อวงจรการต่อสู้ดำเนินไป Combat Memory จะขยายตัวขึ้นเรื่อยๆ จนกระทั่งการต่อสู้สิ้นสุดลง โมดูล Memory Summarizer Agent จะเข้ามาอ่าน Log ดิบจำนวน 10 บรรทัดเหล่านี้ และรวบยอดบีบอัด (Context Collapse) ให้กลายเป็น 1 ประโยคใจความสำคัญ จากนั้นระบบจะดันประโยคนั้นเข้าสู่ Story Memory (ความจำระยะยาว) ก่อนที่จะส่งลงความจำระยะสั้นทั้งหมดทิ้งอย่างถาวร

3.6.2 การฉีดข้อมูลภูมิหลังแบบคงที่ (Static RAG Injection)

ปัญหาสำคัญของ AI Dungeon Master ทั่วไปคือการที่โมเดลเสกไอเทมหรือบรรยากาศที่ไม่ตรงกับธีมเกมออกมา (เช่น การอ้างถึงปืนเลเซอร์ในโลกเวทมนตร์ยุคกลาง) ระบบนี้ใช้กลยุทธ์การฝังข้อมูลบริบท (Context Grounding) ผ่านระบบ Static RAG (Retrieval-Augmented Generation แบบคงที่

1. ปรัชญาการออกแบบที่เรียบง่าย (Simplicity & Fit-for-Purpose) แม้ว่าระบบ RAG สมัยใหม่จะนิยมใช้ Vector Database ในการค้นหาเอกสาร แต่ระบบนี้เลือกยึดหลักความเรียบง่าย (KISS Principle - Keep It Simple, Stupid) โดยมองว่าขอบเขตการเล่นของแคมเปญนี้มีเรื่องราวที่ตายตัวและมีขนาดไม่ใหญ่เกิน Context Window ของโมเดล การเพิ่มฐานข้อมูลเวกเตอร์เข้ามาจะทำให้ระบบมีความซับซ้อนเกินความจำเป็น (Over-engineering) ดังนั้น การโหลดประวัติศาสตร์โลก (World Lore) จากไฟล์ข้อความธรรมดา (Text/JSON) แล้วฉีดเข้าไปตรงๆ ใน System Prompt แทนที่ จึงเป็นทางเลือกที่ "เหมาะสมกับขนาดของโปรเจกต์" มากที่สุด เพราะทั้งดูแลรักษาง่าย (Maintainability) และช่วยลดภาระในการตั้งค่าเซิร์ฟเวอร์ภายนอก (Zero External Dependencies)

2. กลยุทธ์การจัดโครงสร้าง Prompt (Prompt Structuring Strategy): เพื่อให้ AI ให้น้ำหนักกับข้อมูลจำเพาะอย่างถูกต้อง ระบบจะแทรกประวัติศาสตร์ (World Lore) ลงในตัวแปรแยกต่างหากและประกอบร่างกับ Prompt ผ่านแท็กกำกับที่ชัดเจนก่อนส่งคำขอร้องถึง API เช่น

[WORLD_LORE]:

"ถ้าแห่งนี้มีคัมภีร์และเต็มไปด้วยมลพิษเวทมนตร์ ไม่มีตัวละครใดมองเห็นในความมืด ยกเว้นเผ่าเอลฟ์"

[CURRENT_COMBAT_STATE]:

(ข้อมูลในรูปแบบ TOON Format)

การแยกส่วนข้อมูลออกเป็นก้อน (Data Compartmentalization) ทำหน้าที่เป็น Guardrail บังคับให้ AI นำสภาพแวดล้อมจาก [WORLD_LORE] มาพิจารณาควบคู่กับตัวเลขพลังชีวิตเสมอ ส่งผลให้คำบรรยายไม่หลุดจากธีม (Theme Preservation) และสร้างประสบการณ์ร่วม (Immersive Experience) ให้แก่ผู้เล่นได้อย่างสมบูรณ์

3.7 ส่วนต่อประสานผู้ใช้และระบบอำนวยความสะดวก (CLI Dashboard & Quality of Life)

ส่วนประกอบภายนอกที่ผู้ใช้มีปฏิสัมพันธ์ด้วย (Front-facing Interface) และกลไกสนับสนุนที่ซ่อนอยู่เบื้องหลัง (Backend Services) แม้ระบบนี้จะเริ่มเพียงต้นแบบ (Prototype) ในรูปแบบ Command-Line Interface แต่ออกแบบมาเพื่อแสดงถึง "โปรต๊อปทิมที่สมบูรณ์ (Production-Ready Concept)" ที่มีความแข็งแกร่ง (Robustness) ความปลอดภัย (Security) และความทนทานต่อข้อผิดพลาด (Fault Tolerance)

3.7.1 ส่วนแสดงผลเชิงคำสั่งแบบเรียลไทม์ (Real-time CLI Dashboard)

ระบบไม่ได้ออกแบบหน้าจอ Command-Line เพียงเพื่อความสวยงามในรูปแบบ ASCII Art เท่านั้น แต่จุดประสงค์หลักทางสถาปัตยกรรมคือการทำหน้าที่เป็น "เครื่องมือตรวจสอบสถานะแบบเรียลไทม์ (Real-time State Monitor)"

ปัญหาของการใช้ LLM เพียงอย่างเดียวในการเล่นเกมนั้น คือผู้เล่นไม่อาจทราบได้เลยว่าสิ่งที่ AI บรรยายออกมานั้น ตรงกับตัวเลขในฐานข้อมูลจริงหรือไม่ ระบบหน้าจอ Dashboard ของ AI Dungeon Master (เช่น การแสดงหลอด HP สลัปสีชาร์ด หรือการแสดงระยะห่างอ้างอิงจาก Zone System) ได้ถูกเขียนโค้ดขึ้นมาเพื่อดึงค่าทั้งหมดจากโครงสร้างข้อมูล (Data Objects) ใน Python โดยตรง การแสดงผลนี้จึงเป็น "การพิสูจน์ (Proof)" ที่มองเห็นได้ด้วยตาว่า Data ไม่เกิดความผิดพลาดภายใต้การแทรกแซงของ

ตัวเลขจาก Rules Engine และเป็นข้อพิสูจน์เชิงวิศวกรรมว่าระบบทำงานแบบ Single Source of Truth อย่างแท้จริง

3.7.2 ระบบสนับสนุนเอนจินเกม (Engine Support Features)

เพื่อให้ระบบตอบโต้ความเป็นวิศวกรรมซอฟต์แวร์มาตรฐาน ได้มีการพัฒนาระบบสนับสนุนคุณภาพชีวิต (Quality of Life) ในฝั่งของผู้พัฒนาและผู้ใช้งานดังนี้

1. ความปลอดภัยและการจัดส่วนประกอบ (Security & Modularity) ปัจจัยสำคัญของการเชื่อมต่อระบบเข้ากับโมเดลตระกูล LLM คือการปกป้องข้อมูลความลับทางธุรกิจ (API Keys) ระบบได้นำไฟล์ตั้งค่าแบบ .env มาประยุกต์ใช้ในการตั้งค่าผู้ใช้งานภายนอก (External Environment Variables) ทำให้ไม่มีการฝังโค้ดรหัสผ่านไว้ในไฟล์ประมวลผล (No Hardcoding) นอกจากนี้ ระบบยังมีฟีเจอร์ช่วยอำนวยความสะดวกแบบอัตโนมัติ (Automated API Key Wizard) ที่ทำหน้าที่ตรวจสอบไฟล์ประจำเครื่อง หากผู้ใช้อย่างไรก็ตามยังไม่ได้ตั้งค่า ระบบจะหยุดการทำงานชั่วคราวเพื่อแจ้งเตือนให้ผู้ใช้นำคีย์มาป้อนผ่านหน้าจอ Command Line และทำการสร้างไฟล์ .env ให้โดยอัตโนมัติ (Idiot-proof Onboarding) ในส่วนของการตั้งค่าระดับกลไก ยังมีการใช้ระบบ JSON Configuration เพื่อบริหารจัดการตัวแปรของเกม เช่น ราคาสินค้าเบื้องต้น, โอกาสสุ่มเจอเหตุการณ์พิเศษ หรือระดับความยาก ซึ่งเป็นการผลักภาระตรรกะตัวแปรออกจากการตั้งค่าของสคริปต์ ให้ไปอยู่ในฝั่งของโครงสร้างข้อมูล (Data-Driven Design) ทั้งสิ้น
2. ความทนทานต่อข้อผิดพลาด (Fault Tolerance และ Auto-Save) ด้วยข้อจำกัดของการเรียกใช้งาน API ภายนอกที่อาจเกิดการหยุดชะงัก (Network Disconnection) หรือมีการตอบกลับที่ผิดพลาดจาก AI การออกแบบกลไก Auto-Save หลังการจบการคำนวณในทุกๆ เทิร์น (Turn Resolution) จึงหมายถึงการพัฒนาระบบให้กระเด็นกลับจากข้อผิดพลาดร้ายแรงได้เสมอ ผู้เล่นสามารถเริ่มต้นใหม่ (Resume) จากจุดที่สัญญาณอินเทอร์เน็ตหลุดได้ทันทีโดยที่ข้อมูลสถานะและ Context Memory ไม่สูญหาย
3. ระบบอัตโนมัติจากฝั่งแบ็กเอนด์ (Backend Automation และ Auto-Loot) เพื่อป้องกันไม่ให้เกิดปัญหาประดิษฐ์สร้างไอเทมที่ทำลายสมดุลเกมขึ้นมาแบบสุ่ม (Hallucinated Loot) ระบบจึงดึงอำนาจในการครอบงำเงินและไอเทมกลับมาอยู่ในชุดคำสั่งอัตโนมัติของ Python (Auto-Loot System) เมื่อศัตรูตาย (Condition แต่ละสถานะ DEAD) เอนจินกฎเกณฑ์ (Rules Engine) จะแทรกแซงขึ้นมาทำการสุมไอเทมและเพิ่มเงินเข้าคลังของผู้เล่นอย่างเป็นระบบ (Systematic Handling) โดยไม่ต้องรอให้ AI เป็นคนสั่งการ การนำการดำเนินการเล็กน้อยเหล่านี้กลับมาอยู่ฝั่งแบ็กเอนด์ นอกจากจะควบคุมสมดุลได้ง่ายขึ้น ยังช่วยเพิ่มความเร็ว (Speed) และลดการสิ้นเปลือง Token Budget โดยใช้เหตุอีกด้วย
4. สมุดบันทึกประวัติศาสตร์การเล่น (Persistent Campaign Log) ระบบมีกลไกการบันทึกข้อความ (Logging Mechanism) เป็นไฟล์อักษรเปล่า (Plain Text) อย่าง campaign_log.txt แบบ

เรียลไทม์เพื่อเก็บรักษาข้อมูลว่าผู้เล่นทำอะไร และ AI ตอบอะไรในทุกๆ เทิร์น การบันทึกอย่างต่อเนื่องโดยไม่มีการลบข้อมูลทิ้งระหว่างเกมนี้ สร้างประโยชน์ในการเป็น "ข้อมูลตั้งต้น (Seed Data)" ที่สามารถนำไปใช้กับโมเดล RAG ในอนาคตเพื่อสรุปและโหลดเนื้อเรื่องทั้งหมดกลับมาได้อย่างสมบูรณ์

5. โหมดเปิดเผยกลไกสำหรับนักพัฒนา (Developer Debug Mode) เนื่องจากระบบพึ่งพา AI ที่เป็นเครื่องจักรคาดเดายาก ภายในแกนหลักจึงมีการฝังชุดคำสั่งตรวจสอบย้อนกลับ (Traceability) เมื่อเปิดใช้งานโหมดนี้ ผู้ใช้งาน (หรือคณะกรรมการ) จะสามารถมองเห็นและดูคำบรรยายเฟรมตาซีทีที่สวงามลงไปเห็น Object เป้าหมาย (JSON/TOON Output) ผลการตอบโต้จากสมการคณิตศาสตร์ต่างๆ และร่องรอยการตัดสินใจที่ซ่อนอยู่เบื้องหลังได้ทุกเทิร์น เพื่อยืนยันว่าสิ่งที่อยู่ใต้ฝากระป๋องนั้นทำงานได้ตามที่ออกแบบไว้ 100%

3.8 แนวทางการทดสอบและประเมินผล (System Testing Methodology)

เพื่อให้ระบบตอบโต้ภัยวัตถุประสงค้งานวิจัยอย่างเป็นระบบ และเป็นการพิสูจน์ (Validation) ว่าโครงสร้างสถาปัตยกรรมแบบสองเส้นทาง (Two-Path Architecture) สามารถแก้ปัญหาเชิงตรรกะและรักษาความสอดคล้องของเกมได้จริง บทนี้จึงครอบคลุมถึงวิธีการออกแบบชุดทดสอบแบบกล่องดำ (Black-box Behavioral Testing) เพื่อคัดจับทุกเส้นทางตัดสินใจของระบบ

3.8.1 ชุดทดสอบ 50 สถานการณ์ (50-Scenario Functional Test Suite)

ระบบได้รับการออกแบบตารางทดสอบหลัก (Master Test Matrix) จำนวน 50 สถานการณ์จำลอง ซึ่งมีจุดประสงค์ไม่ใช่เพียงการสุ่มข้อความป้อนเข้า แต่เป็นการรับประกันว่าโหนดตรรกะ (Logic Nodes) ทั้ง 24 จุดในระบบ จะถูกทดสอบอย่างครบถ้วน (100% Coverage) สถานการณ์เหล่านี้ถูกแบ่งเป็นหมวดหมู่เพื่อพิสูจน์ความทนทานของเอนจิน ดังนี้

1. การจัดการลำดับคิวและเทิร์น (Initiative & Action Economy) ชุดการทดสอบนี้ (เช่น สถานการณ์ที่ 1-3 และ 7-9) มุ่งเน้นการประเมินโครงสร้างคิวเริ่มต้นแบบรายบุคคล (Individual Turn Order) ทดสอบระบบการข้ามเทิร์นเมื่อตัวละครมีสถานะ UNCONSCIOUS ลบสถานะ STUNNED เมื่อหมดเทิร์น รวมถึงพิสูจน์ระบบ Action Economy ภายใน 1 เทิร์น (จำกัด 1 Action และ 1 Move) ว่า จะปฏิเสธคำสั่งการเดินหรือโจมตีซ้ำซ้อนได้อย่างถูกต้อง
2. ระบบปัญญาประดิษฐ์ของศัตรู (Enemy AI Logic) ชุดการทดสอบนี้ (เช่น สถานการณ์ที่ 4-6) มุ่งเน้นที่ต้นไม้ตัดสินใจ (Decision Tree) ของฝั่ง AI โดยทดสอบว่าศัตรูสามารถวิเคราะห์หาผู้เล่นที่ HP ต่ำที่สุด (Target Lowest HP logic) และทำการเลือกเดินหน้าเข้าหาเป้าหมายเพื่อย่นระยะทาง (จาก โซน FAR มายัง MID หรือ NEAR) ได้อย่างถูกต้องตามกลไกที่ตั้งไว้

3. การประมวลผลเชิงกำหนด (Path A: Deterministic Combat Math) ชุดการทดสอบนี้ (เช่น สถานการณ์ที่ 10-16) มุ่งเน้นไปที่ Rules Engine หลัก พิสูจน์สมการทอยลูกเต๋า 1d20 ควบคู่กับโบนัสค่าสถานะต่างๆ (PHYS/MENT) การเช็คระยะโจมตี การปรับระบบได้เปรียบ/เสียเปรียบ (Advantage/Disadvantage) สภาพแวดล้อม และระบบรับมือกับการหลบหนี (Contested Fleeing) ทำให้มั่นใจว่า "โหมคำนวณ 100%" นี้ไม่พึ่งพาความหลอนของ LLM
4. การตัดสินใจเชิงสร้างสรรค์ (Path B: Arbiter Logic & Creative Stunts) ชุดการทดสอบนี้ (เช่น สถานการณ์ที่ 17-20) ประเมินการทำงานสร้างสรรค์แบบมีการควบคุม (Grounded Creativity) ทดสอบว่า Arbiter Agent สามารถแปลประโยคเหนือจินตนาการ (เช่น "ฉันเตะโต๊ะเพื่อถ่วงเวลา") ให้กลายเป็นการอัปเดตตัวแปรใน Python Object (เช่น การเพิ่ม +AC หรือยัดสถานะ PRONE ใส่เป้าหมาย) ได้หรือไม่ และในกรณีที่ผู้เล่นสั่งสิ่งที่ผิดธรรมชาติ (เช่น "ฉันดื่มน้ำทะเลทั้งหมด") ระบบกลั่นกรองจะต้องปฏิเสธ และคืนเทิร์นให้ผู้เล่นอย่างถูกต้อง
5. การบูรณาการระบบและการรักษาความจำ (System Integration & State Persistence) ชุดการทดสอบนี้ (เช่น สถานการณ์ที่ 21-24) จะเชื่อมโยงระบบเข้าด้วยกันเพื่อทดสอบกลไกแบ็กเอนด์ ไม่ว่าจะเป็น ระบบลूตไอเทมอัตโนมัติเวลาศัตรูตาย (Auto-Loot) หรือ กลไกบีบอัดความจำ (Context Collapse) เลิกใช้ Combat Memory และย้ายข้อมูลไปต่อยอดสู่หน้า Story Memory

3.8.2 การประเมินความแม่นยำและเมตริกวัดผล (Evaluation Mechanisms & Metrics)

เพื่อให้การทดสอบสามารถจับต้องได้ในเชิงวิศวกรรม (Quantitative Proof) ระบบทดสอบอัตโนมัติ (Evaluation Runner) จะอ่านคำอินพุตจากไฟล์สถานการณ์ (scenario_suite.json) โดยมีการจำแนกหมวดหมู่โครงสร้างคำตอบที่คาดหวังไว้ล่วงหน้า (Expected Truth) กระบวนการนี้พึ่งพาคำสั่ง unittest.mock.patch ของ Python เพื่อดักจับค่าตัวแปรในระหว่างทางโดยไม่ต้องแก้ไขสคริปต์หลักของเกมและนำมาเข้าสู่ตรรกะเพื่อหาผลตอบรับ 4 เมตริกชี้วัด

1. การประเมินความแม่นยำของ Intent Router (Routing Accuracy - A_route) มุ่งเน้นทดสอบ "คนเฝ้าประตู" ว่าสามารถจำแนกเจตนาอันซับซ้อนของผู้เล่นออกเป็นหมวดทางเลือก (Fixed vs Creative) ได้ถูกต้องกี่เปอร์เซ็นต์ ซึ่งส่งผลโดยตรงต่อการป้องกันไม่ให้สคริปต์ระบบพังพินาศ หาก Intent Router ตัดสินใจผิด ส่งการร้ายเวทปกติไปให้ Arbiter หรือส่งประโยคจินตนาการไปให้ Python Engine เกมจะสูญเสียระเบียบในทันที
2. การประเมินความแม่นยำการระบุเงื่อนไข (Grounding Precision - P_ground) มุ่งเน้นทดสอบ "ผู้พิพากษา (Arbiter)" ว่าสามารถส่งออกตรรกะในรูปแบบ Enum Keys อย่างถูกต้อง (เช่น ส่งค่า PHYS แทนที่จะส่งค่า MENT หรือส่งสถานะ BLINDED ออกมาตรงตามรูปแบบ TOON Format อย่างถูกต้องหลักไวยากรณ์) หรือไม่

3. การประเมินการซิงโครไนซ์สถานะ (State Synchronization - S_sync) มุ่งทดสอบระดับแกนหลังระบบ "กลไกการคำนวณ (Python Engine)" ว่าสอดคล้องกับโครงสร้างข้อมูล JSON หรือไม่ โดยทำการเทียบตัวเลขพลังชีวิตหรือรายการสินค้า (Inventory Array) ก่อนรันและเทียบกับหลังรัน เพื่อพิสูจน์ระดับความเสถียรและความแม่นยำของการคงสถานะ
4. การประเมินความแม่นยำในคำบรรยาย (Narrative Consistency - C_narr) มุ่งทดสอบความน่าเชื่อถือของ "ผู้เล่าเรื่อง (Narrator)" วงจรตอบรับการกระทำที่สามารถประมวลผลคำอธิบายได้อย่างสมเหตุสมผลต่อทิศทางสถานะการต่อสู้ (Combat Logs) โดยปราศจากการข้อมูลหลอน (Zero Hallucination)

ทั้งหมดนี้ใช้เพื่อจัดตั้งและวางกลไกว่า "ระบบตอบกลับได้อย่างถูกต้องใช่หรือไม่?" ก่อนจะใช้ประเมินข้อมูลคุณภาพและประสิทธิภาพในขั้นต่อไป

บทที่ 4 ผลการดำเนินงานและการประเมินผล

ในบทนี้จะนำเสนอผลการดำเนินงานของระบบต้นแบบ AI Dungeon Master ภายใต้โครงสร้างสถาปัตยกรรมแบบสองเส้นทาง (Two-Path Architecture) ซึ่งมุ่งแก้ปัญหาข้อมูลหลอน (Hallucination) และการควบคุมกลไกของเกม (Rule Enforcement) การประเมินผลในครั้งนี้ใช้ชุดทดสอบแบบกล่องดำ 50 สถานการณ์จำลอง (50-Scenario Black-box Functional Test Suite) เพื่อจำลองรูปแบบการเล่นจริงและประเมินประสิทธิภาพการตัดสินใจในระดับโครงสร้าง

4.1 ขีดจำกัดของระบบในระยะเริ่มต้น (Initial Baseline Limitations)

ก่อนที่จะได้ผลลัพธ์สุดท้าย ระบบได้ผ่านการทดสอบในระยะแรก (Baseline Execution) ซึ่งตีกรอบวัดผลด้วยเมตริก 4 ด้าน ผลปรากฏว่าระบบสามารถทำคะแนน ความแม่นยำในการจำแนกเจตนา (A_route) และ ความสอดคล้องในการเล่าเรื่อง (C_narr) ได้สมบูรณ์ที่ 100%

อย่างไรก็ตาม ความแม่นยำในการระบุเงื่อนไข (P_ground) และการซิงโครไนซ์สถานะข้อมูล (S_sync) กลับมีคะแนนเริ่มต้นเพียง 70% เมื่อพิจารณาจากสมุดบันทึกประวัติ (Trace Log) จึงพบช่องโหว่ทางสถาปัตยกรรม 2 ประการ

1. ปัญหาตะขอกเกี่ยวของสคริปต์ตรวจวัด (Logging Hook Discrepancies): สคริปต์ประเมินผลไม่สามารถดักจับ Action แบบ CREATIVE จาก Arbiter Agent ได้อย่างสมบูรณ์เนื่องจากซิกเนเจอร์ตัวแปรไม่ตรงกัน ทำให้ได้รายงานผลว่า "สอบตก (Fail)" แบบประดิษฐ์ (Artificial Fails)
2. การขาดกำแพงควบคุมสิทธิ์ (Missing Strict Guardrails): แม้ Rules Engine ของ Python จะคำนวณตัวเลขทุกจุด แต่มันหละหลวมในการตรวจสอบสิทธิสภาพแวดล้อม (Action Economy) ปลอมให้ผู้เล่นเกมพิมพ์คำสั่งโจมตี 2 ครั้งในเทิร์นเดียว หรือสั่งดื่มยาเพิ่มเลือดที่ไม่มีอยู่จริง ส่งผลให้ระบบฐานข้อมูล JSON เกิดความสับสนขัดแย้งกับกติกา (State Desynchronization)

4.2 การยกระดับโครงสร้างและปรับปรุงเอนจิน (Structural Refinements & Upgrades)

เพื่ออุดช่องโหว่ด้านความน่าเชื่อถือแบบคำนวณได้ (Deterministic Stability) คณะผู้จัดทำได้ปรับปรุงสถาปัตยกรรมครั้งใหญ่ ควบคู่กับการแก้ปัญหาสคริปต์ตรวจวัด ดังนี้

1. การติดตั้งกำแพงควบคุมระดับสิทธิ์ตัวละคร (Implementation of Permission-Level Guardrails)

- 1) โมเดลข้อมูล (Model) แทรกตัวแปรติดตามสถานะแบบ Boolean อย่าง has_acted และ has_moved เข้าเป็นส่วนหนึ่งของคลาส Character
 - 2) ตัวจัดการการต่อสู้ (CombatManager) สร้างฟังก์ชัน reset_turn() เพื่อดำเนินสถานะตัวละครทุกรอบ
 - 3) เราเตอร์คัดทางเจตนา (Intent Router) เพิ่มตรรกะตรวจสอบแบบเข้ม (Strict Execution Assertions) ทันทีที่เจตนาผู้เล่นถูกจำแนกเป็น ATTACK/MOVE ระบบจะปฏิเสธคำสั่งป้อนเข้า หากค่า Permission นั้นถูกใช้ไปแล้ว ป้องกันการได้เปรียบแบบแอ่ง (Action Economy Exploitation) อย่างเด็ดขาด
2. มาตรการรักษาความปลอดภัยของช่องเก็บของ (Inventory Safety Protocols) เขียนตรรกะฝั่ง Python ขวางคำสั่งการใช้งานไอเทม (USE command) โดยบังคับให้ระบบใช้ฟังก์ชันชี้เป้าตรวจหา (Fuzzy Verification) ในอาร์เรย์ของ player.inventory ก่อนสั่งดำเนินการคำสั่ง .remove() เพื่อป้องกันกลุ่มสถานการณ์การใช้ไอเทมที่ไม่มีจริง (Exploiting Non-existent Items)
3. การขยายขอบเขต Rules Engine (Rules Engine Expansion)
- 1) resolve_spell เพิ่มเชกเตอร์รองรับการใช้เวทมนตร์ โดยแยกดูค่าความเสียหายเป็น 1d10 และย้ายโบนัสด้านการโจมตีไปที่สติปัญญา (MENT) เพื่อให้สอดคล้องกับกฎ se ดั้งเดิม พร้อมปลดเงื่อนไขระยะห่างสายภาพออก
 - 2) resolve_item บังคับให้การคำนวณไอเทมจากฝั่งแบ็กเอนด์ถูกห่อหุ้มในรูปแบบตรรกะชุดเล็ก (is_success, roll, effect_description) เพื่อสร้างมาตรฐานข้อมูลให้ระบบตามสืบข้อมูล (Evaluation Tracer) สามารถตรวจจับการกลายพันธุ์สถานะได้ง่ายขึ้น

4.3 ผลการประเมินขั้นสุดท้าย (Final Evaluation Results)

หลังจากกระบวนการยกระดับโครงสร้างและปรับตัวสืบข้อมูล (Evaluation Runner) ระบบได้ถูกส่งรันการทดสอบประเมินผลผ่าน 50 สถานการณ์จำลองอีกครั้ง ผลลัพธ์สะท้อนถึงขีดความสามารถที่เพิ่มขึ้นและมีเสถียรภาพอย่างจับต้องได้ ดังตารางที่ X

เมตริกชี้วัด (Metric)	คะแนนทางประสิทธิภาพ	คำอธิบายเจาะจง
A_route (Routing Accuracy)	100%	จำแนกกฎเกณฑ์คณิตศาสตร์ (Fixed) และตรรกะสร้างสรรค์ (Creative) ได้แม่นยำเต็มประสิทธิภาพ
P_ground (Grounding Precision)	98%	แปลงคำบรรยายผู้เล่นเป็นการตั้งค่าพารามิเตอร์ของระบบ (DC/Stat) ได้อย่างถูกต้อง
S_sync (State Synchronization)	76%	ความเสถียรของกลไกการเปลี่ยนค่าโครงสร้างและฐานข้อมูลหลังดำเนินการสำเร็จ

C_narr Consistency)	(Narrative	100%	ปราศจากการหลอนข้อมูล (Zero Hallucination) คำเล่าเรื่องอ้างอิง ผลลัพธ์จากตัวเลขตรงกัน 100%
------------------------	------------	------	--

ตารางที่ X สรุปผลการทดสอบเชิงปริมาณของสถาปัตยกรรม Two-Path (หลังปรับปรุง)

การยกระดับโครงสร้างในหัวข้อ 4.2 สามารถดึงค่า Grounding Precision พุ่งขึ้นจาก 70% เป็น 98% ในขณะที่ค่า State Synchronization ขยับขึ้นจาก 70% เป็น 76%

(หมายเหตุ: สาเหตุที่ค่า S_sync สิ้นสุดที่ 76% เป็นผลมาจากการจำลองพารามิเตอร์ตรวจสอบทางกร
เจรจาสังคม (Contested Social Checks) ในสถานการณ์ B-26 ถึง B-30 ที่มีเงื่อนไขสถานะซับซ้อนเกิน
กว่าขอบเขตของกฎ Lite 5e Rulebook ซึ่งถือว่าอยู่นอกเหนือสมมติฐานเป้าหมายหลักในการคำนวณ
ของการวิจัยเรื่องระบบต่อสู้ฉบับนี้)

4.4 การพิสูจน์ความครอบคลุมของชุดทดสอบ (Dataset Completeness & Validation)

เพื่อคลายความกังวลด้านปริมาณสถานการณ์ทดสอบ (50 สถานการณ์ถือว่าน้อยเกินไปหรือไม่สำหรับการพิสูจน์ AI?) คณะผู้จัดทำขอชี้แจงถึงหลักวิศวกรรมว่า คำสั่ง 50 รูปแบบในชุดทดสอบ ไม่ใช่การสุ่ม
สร้างขึ้นมา (Not Randomized) ข้อมูลนำเข้าทั้งหมดถูกกำหนดเป้าหมายเพื่อตรวจสอบ "กึ่งก้านการ
ตัดสินใจที่ละเอียด" ให้ครอบคลุมความน่าจะเป็นในโลกความปลอดภัยถึง 24 โหนด (24 Core Logic
Nodes) ดังตัวอย่างการทดสอบเชิงโครงสร้าง (Node-specific Testing) ต่อไปนี้

1. หมวดคณิตศาสตร์ตายตัว (Path A - Deterministic) มุ่งเน้นทดสอบระบบปัญญาการและสมการ
ระยะทางการต่อสู้ที่ไม่ต้องการให้ AI ตัดสินใจ (A-01 ถึง A-20)

1) Input (A-01) "I slash the goblin with my sword." -> Expected: A_route ดักจับเข้าหมวด
FIXED อัตโนมติ, แจ้งคำสั่ง ATTACK (ตีแบบ melee), คณิตศาสตร์การตัดสินใจกลับสู่ Python Engine ให้ทำ
การคำนวณการตีบวกด้วยโบนัสกายภาพ PHYS ทันที

2) Input (A-06) "I cast Firebolt at the FAR enemy." -> Expected: A_route ย้ายเข้ากฎ FIXED
เลือกกลไกโหนด CAST, ระบบตรวจพบเป้าหมายอยู่ไกล (FAR) โยนตัวแปรให้โยนได้เสียเปรียบ
(Disadvantage)

2. หมวดการตีความสร้างสรรค์และการเจรจา (Path B - Creative & Social) ทดสอบศักยภาพฝั่ง LLM
Agent เพื่อจำแนกความฉลาดและจัดสรรสถิติที่เหมาะสมให้เข้าสู่ตร Python ต่อ (B-21 ถึง B-35)

1) Input (B-21) "I kick the heavy table to block the arrow." -> Expected: A_route จำแนกไปที่ CREATIVE, Action Arbiter agent รับช่วงต่อ พิจารณาว่าต้องใช้กำลัง จึงกำหนดตัวแปรทดสอบสถิติเป็น PHYS พร้อมกำหนดค่าความยาก DC ที่สมเหตุสมผล

2) Input (B-26) "I intimidate the bandit into surrendering." -> Expected: Arbiter มองข้ามเจตนาทำร้ายระยะประชิด ย้ายไปวิเคราะห์การเจรจา (Social Stunt) และเจาะจงเรียกใช้สถิติความกดดันสังคม SOC เพื่อใช้เช็คตรวจสอบแทน PHYS/MENT ทั่วไป

3. หมวดตรวจรบกวนกันข้อผิดพลาดรุนแรง (Edge Cases & System Guardrails) ออกแบบมาเพื่อทดสอบสมมติฐานและโจมตีช่องโหว่ทางลอจิกเฉพาะตัว (C-36 ถึง C-45)

1) Input (C-36 Impossible) "I teleport to the moon instantly." -> Expected: เผลอญกับเจตนาที่เป็นไปไม่ได้ภายในเกม Action Arbiter ยกสถานะเป็น "ไม่อนุญาต" (expected_allowed: false) แล้วคืนเทิร์นให้ผู้เล่น

2) Input (C-41 Rule Enforcement) "I attack twice in one turn." -> Expected: เจตนาโจมตีปกติ แต่เนื่องจากละเมิดกฎ Action Economy ตัวป้องกันใน Python ขัดขวางผลการอัปเดตและตีกลับด้วยตัวแปร expected_engine_result: DENY

วิธีการวัดผลแบบเจาะจงกึ่งกันตรรกะ (Black-box Logical Mapping) ที่อ้างอิงกับผลลัพธ์ที่คาดการณ์ได้เสมอระดับร้อยเปอร์เซ็นต์แบบนี้ ให้ค่าความเชื่อมั่นที่เป็นวิทยาศาสตร์ และเป็นหลักประกันว่าสถาปัตยกรรมตัวใหม่มีความรัดกุมกว่าช่วง Prototype ต้นทาง

4.5 แผนงานประเมินผลในอนาคต (Future Testing Direction)

เพื่อเสริมความแข็งแกร่งในการสรุปผลประสิทธิภาพ โครงการได้วางแผนการเก็บข้อมูลทดลองในระยะถัดไป (เฟสขยายผล) ดังนี้

1. ข้อมูลเชิงคุณภาพ (Qualitative Data) - ความสมเหตุสมผลและความลื่นไหล: จัดทำ Expert Evaluation โดยนำฉากต่อสู้ที่ถูกรวบรวมไว้ในไฟล์บันทึกประวัติ (Trace log) ไปประเมินให้คะแนนคุณภาพ (1-5) ตามการอ้างอิงของระบบ Game Master ตัวจริงตามธรรมชาติ (Human Validation) เพื่อไขว่คว้าวิเคราะห์หาว่าบรรยากาศความสนุก, และวิธีการตัดสินใจสถานการณ์ของ Arbiter นั้นมีความสมจริงคู่ปัญหาจากมนุษย์ในระดับไหน
2. ข้อมูลประสิทธิภาพทางเทคนิค (Technical Quantitative Data) - ภาระโหลดการประมวลผล: บันทึกเวลาตอบสนองการคำนวณ (Latency Test) ของแพ็คเกจเจตนาเพื่อวัดว่า AI ใช้เวลาที่วินาทีต่อ 1 แอ็กชันในแต่ละหมวดหมู่ (เช่น 3 วินาทีสำหรับโจมตีธรรมดา เทียบกับ 6 วินาทีสำหรับ Creative

Stunt) ควบคู่กับการวิเคราะห์ต้นทุน (Token Consumption Test) เพื่อให้ประจักษ์ว่า รูปแบบการอัดลดขนาดข้อมูลแบบ TOON เซฟทรัพยากรสถาบันได้มากน้อยเพียงใดเมื่อเทียบกับ JSON โครงสร้างเต็ม

3. การทดสอบเปรียบเทียบในสภาพแวดล้อมจำลอง (Comparative Hallucination Study): รัน 50 สถานการณ์ตัวอย่างชุดเดียวกัน ด้วยโครงสร้างฝั่งเอเจนต์แบบตัวเดียว (Single-Agent Prompt Control) หรือการไม่พึ่งพาเอนจิน Python คอยตรวจสอบผลลัพธ์ เพื่อยืนยันให้เห็นกระแส "ความไร้ระเบียบ (Hallucination Spread)" อย่างตั้งใจ ผลลัพธ์แห่งการเปรียบเทียบชุดนี้จะเป็นข้อสรุปเชิงประจักษ์ที่ตอกย้ำถึงวาทกรรมตั้งต้นว่า: ทำไมสถาปัตยกรรมสองเส้นทางจึงเป็นสิ่งที่จำเป็นที่วงการหลีกเลี่ยงไม่ได้

เอกสารอ้างอิง

บทความในรายงานการประชุมทางวิชาการและวารสาร (Academic Papers)

1. Triyason, T., et al., 2023, "Exploring the Potential of ChatGPT as a Dungeon Master in Dungeons & Dragons Tabletop Game", Proceedings of the 2023 International Conference on Digital Image and Multimedia (ICODIM), 1-3 December, Chiang Mai, Thailand, pp. 1-6.
2. Jørgensen, N. H., Tharmabalan, T., Aslan, M., & Merritt, T., 2024, "Evaluating Multi-Agent Architectures for an AI Game Master in Dungeons & Dragons", Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems, 11-16 May, Honolulu, HI, USA.
3. Lanchantin, J., et al., 2024, "You Have Thirteen Hours in Which to Solve the Labyrinth: Enhancing AI Game Masters with Function Calling", arXiv preprint arXiv:2409.06949.
4. Vaswani, A., et al., 2017, "Attention Is All You Need", Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, pp. 5998-6008.

บทความและแหล่งข้อมูลออนไลน์ (Online Resources & Tools)

5. Sakellaridis, P., 2024, AI Dungeon Master for DnD Railroads Players, [Online], Available: <https://www.wargamer.com/dnd/ai-railroads-players> [2025, October 19].
6. Maassen, H., 2019, D&D 5e Lite, [Online], Available: <https://haraldmaassen.com/blog/2019/06/02/dd-5e-lite/> [2025, October 19].
7. TOON Format Contributors, 2025, TOON: Token-Oriented Object Notation for LLMs, [Online], Available: <https://github.com/toon-format/toon> [2026, March 18].

8. Python Software Foundation, 2025, Python 3.10.x Documentation, [Online], Available: <https://docs.python.org/3.10/> [2026, March 18].
9. Google for Developers, 2026, Gemini API Documentation: Function Calling and Multi-turn Conversations, [Online], Available: <https://ai.google.dev/docs> [2026, March 18].

ภาคผนวก ก

ชุดกติกาลดทอนความซับซ้อน Lite 5e

ภาคผนวก ก: ชุดกติกาลดทอนความซับซ้อน Lite 5e

(หมายเหตุ: ชุดกติกานี้แปลและเรียบเรียงโดยอ้างอิงจากต้นฉบับภาษาอังกฤษซึ่งเป็นฉบับดั้งเดิมของโครงการ)

บทนำ (Introduction)

ยินดีต้อนรับเหล่านักผจญภัย! คู่มือกติกานี้ถูกออกแบบมาสำหรับผู้เล่นใหม่และเพื่อวัตถุประสงค์ของโครงการวิทยานิพนธ์นี้โดยเฉพาะ ระบบของเราได้รับแรงบันดาลใจอย่างมากจากเกม Dungeons & Dragons ฉบับที่ 5 (5th Edition) แต่ได้ถูกปรับลดทอนขั้นตอนให้มีความคล่องตัว (Streamlined) เพื่อให้:

- ง่ายต่อการเรียนรู้ (Easy to learn)
- ดำเนินเกมได้อย่างรวดเร็ว (Fast to play)
- เหมาะสมที่สุดสำหรับการโต้ตอบกับปัญญาประดิษฐ์ (Optimized for LLM/AI Dungeon Master)

ไม่ว่าคุณจะเป็นนักรบที่เหวี่ยงดาบ, นักบวชผู้ร้องขอพลังจากทวยเทพ หรือจอมเวทผู้บิดเบือนความเป็นจริง กฎเหล่านี้จะมอบโครงสร้างที่ยุติธรรมให้กับการเล่นของคุณ ในขณะที่ยังคงรักษาอิสระในการเล่าเรื่อง (Narrative Freedom) ไว้อย่างสมบูรณ์

วิธีการเล่น (How to Play)

วงจรการเล่นหลัก (Core Gameplay Loop) มีขั้นตอนที่เรียบง่าย ดังนี้:

1. AI Dungeon Master (LLM) บรรยายสถานการณ์: (เช่น “ก๊อบลินตัวหนึ่งกระโดดออกมาจากหลังถ้ำ!”)
2. ผู้เล่นระบุเจตนา (Intent): (เช่น “ฉันจะเหวี่ยงดาบฟันไปที่ก๊อบลิน!”)
3. ระบบทอยลูกเต๋าเพื่อตัดสินผลสำเร็จ (Resolution): (เช่น ทอย d20 + PHYS หรือ d20 + MENT เทียบกับค่า AC หรือ DC)
4. บรรยายผลลัพธ์ที่เกิดขึ้นจริง: (เช่น “ดาบของคุณฟันเข้าอย่างจัง! ก๊อบลินได้รับความเสียหาย 7 หน่วย”)

พื้นฐานการทอยลูกเต๋า (Dice Basics)

เราใช้ระบบลูกเต๋า 20 หน้า (d20) เป็นหลักในการตัดสิน:

- การตรวจสอบทักษะ (Skill Checks): ทอย d20 + โบนัสค่าสถานะ (Stat Modifier) เทียบกับ ค่าความยาก (Difficulty Class: DC)
- การทอยเพื่อโจมตี (Attack Rolls): ทอย d20 + โบนัสค่าสถานะ เทียบกับ ค่าพลังป้องกันของศัตรู (Armor Class: AC)
- การทอยความเสียหาย (Damage Rolls): ใช้ลูกเต๋านาฬิกาเกล็ดงูตามประเภทอาวุธ (เช่น d6, d8 เป็นต้น)

การทอยแบบพิเศษ (Special Rolls):

- ความได้เปรียบ (Advantage): มอบโบนัส +5 กับการทอยทันที (เช่น เมื่อศัตรูติดสถานะมึนงง หรือ หมดสติ)
- ความเสียเปรียบ (Disadvantage): มีบทลงโทษ -5 กับการทอย หรือ ต้องทอยเต๋า 2 ลูกและเลือกค่าที่ต่ำที่สุด (เช่น การโจมตีระยะไกลใส่เป้าหมายที่อยู่ในโซนไกล)
- แด้ม 20 (Natural 20): ถือเป็นความสำเร็จขั้นสูงสุด (Critical Success) สำเร็จโดยอัตโนมัติ และสร้างความเสียหาย 2 เท่า (สำหรับการโจมตี)
- แด้ม 1 (Natural 1): ถือเป็นความล้มเหลวขั้นร้ายแรง (Critical Failure) สำเร็จโดยอัตโนมัติ และอาจเกิดผลเสียทางเนื้อเรื่องตามมา

ค่าสถานะและโบนัส (Stats & Modifiers)

เราได้รวบรวมค่าความสามารถมาตรฐาน 6 ค่าของ D&D ให้เหลือเพียง 3 ค่าสถานะหลัก (Core Stats) ดังนี้:

ค่าสถานะ (Stat)	ความครอบคลุม (Covers)	ตัวอย่างการกระทำ (Example Actions)
กายภาพ (PHYS)	พลังกำลัง (STR), ความคล่องแคล่ว (DEX), ความทนทาน (CON)	การโจมตี, ปีนป่าย, หลบหลีก, ผลัก
สติปัญญา (MENT)	ความรู้ (INT), สัญชาตญาณ (WIS), การรับรู้ (Perception)	การจดจำความรู้, การสังเกตกับดัก, การแก้ปริศนา
สังคม (SOC)	เสน่ห์ (CHA), การโน้มน้าวใจ (Persuasion)	การโน้มน้าว, การโกหก, การข่มขู่

แต่ละค่าสถานะจะมี ค่าโบนัส (Modifier) ตั้งแต่ -2 ถึง +5 ซึ่งจะนำไปบวกกับการทอย d20 ของคุณ
พลังชีวิต (Hit Points: HP)

- ตัวละครเริ่มต้นทั้งหมดจะมี 20 HP ที่ระดับเลเวล 1
- ความเสียหายจะลดค่า HP
- 0 HP = หมดสติ (Unconscious)
- หากไม่ได้รับการรักษา การมี HP เท่ากับ 0 อาจนำไปสู่ความตาย (Death) ได้

ระบบการต่อสู้ (Combat System)

การต่อสู้เป็นระบบ ผลัดกันเล่น (Turn-based) และใช้ระบบการระบุตำแหน่งแบบ โซน (Zone-based) แทนการใช้ตารางกริด

ลำดับเทิร์น (Turn Order)

ระบบใช้ คิวลำดับการกระทำ (Initiative Queue) เพื่อตัดสินลำดับการเล่นที่ยุติธรรมและยืดหยุ่น:

1. การทอยลำดับ (Initiative Roll): เมื่อเริ่มการต่อสู้ ตัวละครทั้งหมดทอย d20 + โบนัส PHYS

2. การจัดลำดับ (Sorting): ใครที่ทอยได้แต้มสูงสุดจะได้เริ่มก่อน (อ้างอิงจาก PHYS ซึ่งเป็นค่าสถานะรวมความคล่องแคล่ว)
3. การวนรอบ (Looping): เมื่อทุกคนในคิวทำการกระทำครบแล้ว จะเริ่มรอบ (Round) ใหม่โดยวนกลับไปลำดับแรกในคิว
4. ตัวละครที่หมดสติ: หากตัวละครมี HP เหลือ 0 หรือติดสถานะที่ทำให้ไม่สามารถขยับได้ ระบบจะข้ามเทิร์นนั้นโดยอัตโนมัติ

ในเทิร์นของคุณ คุณสามารถกระทำได้ 1 การกระทำ (Action) + 1 การเคลื่อนที่ (Move) และสามารถเลือก ลำดับก่อน-หลัง ได้อย่างอิสระ (เช่น เดินเข้าไประยะประชิดแล้วโจมตี หรือ โจมตีแล้วรีบเดินหนีออกมา)

- การกระทำ (Action): การโจมตี (ประชิด/ไกล), การร้ายเวทมนตร์, การใช้ความสามารถพิเศษ, การใช้ไอเทม (ดื่มยา, โยนระเบิด), การปฏิสัมพันธ์กับสิ่งแวดล้อม (ถีบประตู, ดึงคันโยก)
- การเคลื่อนที่ (Move): การเปลี่ยนตำแหน่งไปยังโซนอื่น (จำกัดสูงสุด 1 โซนต่อเทิร์น)

การเคลื่อนที่และโซน (Movement & Zones)

เราไม่ใช้ตารางกริด แต่แบ่งพื้นที่ออกเป็น โซน (Zones):

- โซนเดียวกัน (NEAR): สามารถโจมตีได้ทั้งระยะประชิด (Melee) และระยะไกล (Ranged)
- โซนติดกัน (MID): สามารถโจมตีระยะไกลได้ แต่การโจมตีระยะประชิดจะล้มเหลว
- โซนไกล (FAR): สามารถโจมตีระยะไกลได้ แต่จะติดสถานะ เสียเปรียบ (Disadvantage) โดยระบบจะทอย d20 สองลูกและเลือกแต้มที่ดีที่สุด เนื่องจากเป้าหมายอยู่ห่างออกไป 2 โซน (เช่น จาก Near ไป Far)

ผู้เล่นสามารถเคลื่อนที่ได้ครั้งละ 1 โซนต่อเทิร์น โดยถือเป็น 1 การเคลื่อนที่ (Move)

การโจมตี (Attacking)

การโจมตีแบ่งออกเป็น 2 ประเภทตามอาวุธหรือความสามารถที่ใช้:

1. การโจมตีระยะประชิด (Melee Attack)

- เงื่อนไข: ผู้โจมตีและเป้าหมายต้องอยู่ใน โซนเดียวกัน (Same Zone)
- สูตรการทอย: $d20 + \text{PHYS}$ เทียบกับ AC
- ความเสียหาย: แต้มลูกเต๋าอาวุธ + PHYS

2. การโจมตีระยะไกล หรือ เวทมนตร์ (Ranged / Spell Attack)

- เงื่อนไข: โจมตีข้ามโซนได้ (หากโจมตีห่างกัน 2 โซน จะติดสถานะ เสียเปรียบ)
- สูตรการทอย: $d20 + \text{MENT}$ (สำหรับเวทมนตร์) หรือ $d20 + \text{PHYS}$ (สำหรับอาวุธยิง) เทียบกับ AC
- ความเสียหาย: แต้มลูกเต๋าวุธ/เวทมนตร์/อาวุธ + MENT (สำหรับเวทมนตร์) หรือ + PHYS (สำหรับอาวุธยิง)

- คริติคอลลิต (Nat 20): ความเสียหายจะถูกคูณสอง

การหลบหนี (Fleeing / Escaping)

ในการหลบหนีจากการต่อสู้ ผู้เล่นต้องระบุเจตนา (เช่น "ฉันจะวิ่งหนี!") ระบบจะตัดสินผลโดยใช้การทอยค่า PHYS แข่งกัน (Contested Check) กับศัตรูที่อยู่ใกล้ที่สุด:

- การทอยของผู้เล่น: $d20 + \text{PHYS}$
- การทอยของศัตรู: $d20 + \text{PHYS} + \text{บทลงโทษระยะประชิด (Proximity Penalty)}$

บทลงโทษระยะประชิด (Proximity Penalty): การหนีจะยากขึ้นหากศัตรูอยู่ติดกับตัวคุณ ศัตรูจะได้รับโบนัสตามระยะห่าง (โซน) ดังนี้:

- โซนเดียวกัน (ระยะ 0): ศัตรูได้รับโบนัส +5
- โซนติดกัน (ระยะ 1): ศัตรูได้รับโบนัส +2
- โซนไกล (ระยะ 2 ขึ้นไป): ศัตรูไม่ได้รับโบนัส (+0)

เคล็ดลับยุทธวิธี: เนื่องจากคุณสามารถ เดินและกระทำ ได้ในเทิร์นเดียวกัน คุณควรใช้การเคลื่อนที่ (Move) เพื่อเพิ่มระยะห่างจากศัตรูก่อน แล้วจึงใช้การกระทำ (Action) เพื่อหลบหนีในลำดับเดียวกัน เพื่อลดโบนัสของศัตรู! หากแต้มรวมของคุณชนะศัตรู คุณจะหนีจากการต่อสู้สำเร็จ หากแพ้ ศัตรูจะขัดขวางการหนีของคุณได้และจบเทิร์นนั้นทันที

ตัวอย่างอาชีพ (Example Classes)

นักรบ (Fighter)

- HP: 20 | AC: 16 (ชุดเกราะโซ่ติด: Chainmail)
- การโจมตี (ประชิด): ดาบยาว (Longsword) $\rightarrow 1d8 + \text{PHYS}$
- ความสามารถ: Second Wind $\rightarrow$ ฟื้นฟู HP $1d10 + 1$ (ใช้ได้ 1 ครั้ง ต่อการต่อสู้)

พาลาดีน (Paladin)

- HP: 20 | AC: 16 (ชุดเกราะโซ่ติด: Chainmail)
- การโจมตี (ประชิด): ดาบใหญ่ (Greatsword) $\rightarrow 2d6 + \text{PHYS}$
- ความสามารถ: Smite $\rightarrow$ เพิ่มความเสียหาย $2d8$ (ใช้ได้ 2 ครั้ง ต่อวัน)
- ความสามารถ: Lay on Hands $\rightarrow$ ฟื้นฟู HP $1d8 + \text{MENT}$ (ใช้ได้ 1 ครั้ง ต่อวัน)

นักบวช (Cleric)

- HP: 18 | AC: 18 (เกราะแผ่นเหล็ก + โล่: Plate + Shield)
- การโจมตี (ประชิด): ค้อนศึก (Warhammer) $\rightarrow 1d8 + \text{PHYS}$
- ความสามารถ: Pray (ทอย MENT เทียบกับ DC 13) $\rightarrow$ เลือกรผล: ฟื้นฟู HP ให้เพื่อน ($1d8 + \text{MENT}$) หรือ สร้างความเสียหายแก่ศัตรู ($1d8 + \text{MENT}$ Radiant Dmg)

จอมเวท (Mage)

- HP: 15 | AC: 12 (ชุดคลุม + เวทเกราะ: Robes + Mage Armor)

- การโจมตี (เวทมนตร์): พลังไฟ (Firebolt) $\rightarrow$ 1d10 + MENT (ระยะไกล)
- การโจมตี (ประชิด): ไม้เท้า (Quarterstaff) $\rightarrow$ 1d6 + PHYS (กรณีฉุกเฉิน)
- เวทมนตร์: ใช้ได้ 2 ครั้งต่อวัน (รูปแบบอิสระ: LLM Arbiter จะเป็นผู้กำหนดค่า DC และผลกระทบต่อเนื้อเรื่อง)

ไอเทมและช่องเก็บของ (Items & Inventory)

ในระหว่างการผจญภัย ตัวละครจะได้รับและใช้ไอเทม ระบบของเราบังคับใช้กฎที่เข้มงวดสำหรับ ไอเทมใช้แล้วหมดไป (Consumable Items):

- การใช้งานตามกลไกปกติ (Path A - Fixed): หากคุณสั่งใช้ไอเทมมาตรฐาน (เช่น "ฉันทันคัมภีร์" หรือ "ฉันทันอ่านคัมภีร์") ระบบจะตรวจสอบกับค่าสำคัญที่ตั้งไว้ (Potion, Scroll, Ration, Bomb) หากตรงกัน ไอเทมจะถูกใช้และลบออกจากช่องเก็บของ (Inventory Array) โดยอัตโนมัติ
- การใช้งานเชิงสร้างสรรค์ (Path B - Creative): หากคุณนำไอเทมไปใช้ในทางอื่นตามเนื้อเรื่อง (เช่น "ฉันทันใช้เชือกจุดไฟเพื่อให้แสงสว่าง" หรือ "ฉันทันฟาดขวดแก้วเปล่าใส่กำแพงเพื่อสร้างความสั่นไหว") AI (Arbiter) จะประเมินว่าการกระทำนั้นทำให้ไอเทม สูญหาย, ถูกใช้หมด หรือถูกทำลาย หรือไม่ หากใช่ ไอเทมนั้นจะถูกลบออกจากช่องเก็บของของคุณ

หมายเหตุ: คุณไม่สามารถใช้งานหรือมีปฏิสัมพันธ์กับไอเทมที่ไม่มีอยู่ในช่องเก็บของจริงได้

การพักผ่อนและเวลา (Rest & Time)

การพักผ่อนสั้น (Short Rest)

- ระยะเวลา: 1 ชั่วโมง
- ผลลัพธ์: ฟื้นฟู HP เท่ากับ 1d8 + โบนัสค่าสถานะ, รีเซ็ตความสามารถที่จำกัดจำนวนครั้ง (เช่น Second Wind)

การพักยาว (Long Rest)

- ระยะเวลา: 8 ชั่วโมง (ต้องทำในสถานที่ที่ปลอดภัย)
- ผลลัพธ์: ฟื้นฟู HP ทั้งหมด, เวทมนตร์ และความสามารถทั้งหมดกลับมาเป็นค่าสูงสุด

สถานะผิดปกติ (Conditions)

- หมดสติ (Unconscious): 0 HP; ไม่สามารถกระทำการใดๆ ได้
- มึนงง (Stunned): ถูกข่มเทิร์นในรอบนั้นๆ ทันที
- ถูกพันธนาการ (Restrained): การทอย d20 ทั้งหมดที่ใช้ค่า PHYS จะติดสถานะ เสียเปรียบ (Disadvantage)
- ตาย (Dead): ถูกนำออกจากการเล่นทันที เว้นแต่จะได้รับการชุบชีวิต

การเพิ่มระดับเลเวล (Leveling Up)

(หมายเหตุ: เป็นฟีเจอร์ทางเลือกสำหรับต้นแบบ อาจยังไม่ได้รับการติดตั้งสมบูรณ์)

- Max HP +5 ต่อเลเวล

- เลือกหนึ่งอย่าง: +1 โบนัสค่าสถานะ หรือ เพิ่มจำนวนครั้งการใช้เวทมนตร์/ความสามารถใหม่
- ระบบค่าประสบการณ์ (XP): ต้องการ 250 XP เพื่อเพิ่มระดับเลเวล

Appendix A: Lite 5e Ruleset (Original Version)

Introduction

Welcome, adventurers! This rulebook is designed for new players and specifically for this thesis project. Our system is heavily inspired by Dungeons & Dragons 5th Edition, but streamlined to be:

- Easy to learn
- Fast to play
- Optimized for LLM/AI Dungeon Master interaction

Whether you are a sword-swinging warrior, a cleric calling upon divine power, or a mage who bends reality, these rules provide a fair framework for your play while maintaining total narrative freedom.

How to Play

The core gameplay loop is simple:

1. The LLM/DM describes the situation (e.g., “A goblin jumps out from behind the barrel!”)
2. The player states their intent (e.g., “I swing my sword at the goblin!”)
3. The system rolls dice to resolve success (e.g., d20 + PHYS or d20 + MENT vs. AC/DC)
4. The outcome is narrated (e.g., “Your sword strikes true! The goblin takes 7 damage.”)

Dice Basics

We primarily use the d20 system:

- Skill Checks: d20 + Stat Modifier vs. Difficulty Class (DC)
- Attack Rolls: d20 + Stat Modifier vs. Enemy Armor Class (AC)
- Damage Rolls: Rolling smaller dice (d6, d8, etc.)

Special Rolls (Advantage & Disadvantage):

- Advantage: Grants a +5 flat bonus to the roll (e.g., when the enemy is Stunned or Unconscious).
- Disadvantage: Incurs a -5 flat penalty to the roll (e.g., when you are Blinded), OR requires rolling 2 dice and taking the lowest result (for Ranged attacks against a Far target).
- Natural 20: Automatic resolution success (Critical Success) and deals double damage (for attacks).
- Natural 1: Automatic resolution failure (Critical Failure) and may incur narrative consequences.

Stats & Modifiers

We have condensed the standard 6 D&D Ability Scores into just 3 Core Stats:

Stat	Covers	Example Actions
Physical (PHYS)	Strength, Dexterity, Constitution	Attacking, climbing, dodging, shoving
Mental (MENT)	Intelligence, Wisdom, Perception	Recalling lore, spotting traps, puzzles
Social (SOC)	Charisma, Persuasion	Persuading, lying, intimidating

Each stat has a Modifier ranging from -2 to +5, which is added to your d20 rolls.

Hit Points (HP)

- All characters start with 20 HP at Level 1.
- Damage reduces HP.
- 0 HP = Unconscious.
- If left untreated, reaching 0 HP can lead to death.

Combat System

Combat is Turn-based and utilizes a grid-less Zone-based positioning system.

Turn Order

The system uses an Initiative Queue to determine a fair and dynamic turn order:

1. Initiative Roll: At the start of combat, all characters roll d20 + PHYS bonus.
2. Sorting: Whoever rolls highest goes first (Similar to D&D 5e utilizing Dexterity, but in Lite 5e we use the consolidated PHYS stat).
3. Looping: Once everyone in the queue has acted, a new Round begins and the turn order loops back to the top of the queue.
4. Unconscious Characters: If a character drops to 0 HP or suffers a condition that prevents taking actions, the system automatically skips their turn.

On your turn, you may take 1 Action + 1 Move, and you may choose the order in which they resolve (e.g., Move into range then Attack, or Attack then run away).

- Action: Attack (Melee/Ranged), Cast a Spell (if applicable), Use a Special Ability, Use an Item (drink potion, throw bomb), Interact with the Environment (kick a door, pull a lever).
- Move: Traverse to a different Zone (Maximum 1 Zone per turn).

Movement & Zones

We do not use battle grids; instead, space is divided into Zones:

- Same Zone (NEAR): Allows for both Melee and Ranged attacks.

- Adjacent Zone (MID): Allows for Ranged attacks, but Melee attacks fail.
- Far Zone (FAR): Allows for Ranged attacks, but incurs Disadvantage (The system rolls two d20s and takes the lowest result) because the target is 2 zones away (e.g., Near to Far).

Players may Move exactly 1 Zone per turn as their Move action.

Attacking

Attacks are divided into 2 categories based on the weapon:

1. Melee Attack

- Condition: Attacker and Target must be in the Same Zone.
- Roll Formula: $d20 + \text{PHYS}$ vs AC
- Damage: Weapon Dice + PHYS

2. Ranged / Spell Attack

- Condition: Can attack across any Zone (Attacking a target 2 zones away incurs Disadvantage).
- Roll Formula: $d20 + \text{MENT}$ (for Spells) or $d20 + \text{PHYS}$ (for Bows) vs AC
- Damage: Spell/Weapon Dice + MENT (for Spells) or + PHYS (for Bows)
- Critical Hit (Nat 20): Damage is doubled.

Fleeing / Escaping

To flee from combat, a player must declare their intent to escape (e.g., "I run away!"). The system resolves this using a contested PHYS check against the closest active enemy:

- Player Roll: $d20 + \text{PHYS}$
- Enemy Roll: $d20 + \text{PHYS} + \text{Proximity Penalty}$

Proximity Penalty: It is significantly harder to escape if enemies are right next to you. The enemy receives a mathematical bonus to their roll based on the distance (in Zones) between you and them:

- Same Zone (Distance 0): Enemy gets a +5 bonus.
- Adjacent Zone (Distance 1): Enemy gets a +2 bonus.
- Far Zone (Distance 2+): No bonus (+0).

Tactical Tip: Since you get 1 Move and 1 Action per turn, you should use your Move to increase your distance from the enemy, and then use your Action to Flee in the same sequence, reducing the enemy's proximity penalty! If your total beats the enemy's total, you successfully escape combat. If you fail, the enemy cuts off your escape and your turn is consumed.

Example Classes

Fighter

- HP: 20 | AC: 16 (Chainmail)
- Attack (Melee): Longsword $\rightarrow$ $1d8 + \text{PHYS}$
- Ability: Second Wind $\rightarrow$ Heals HP $1d10 + 1$ (1 use / combat)

Paladin

- HP: 20 | AC: 16 (Chainmail)
- Attack (Melee): Greatsword $\rightarrow$ $2d6 + \text{PHYS}$
- Ability: Smite $\rightarrow$ Adds $2d8$ damage (2 uses / day)
- Ability: Lay on Hands $\rightarrow$ Heals HP $1d8 + \text{MENT}$ (1 use / day)

Cleric

- HP: 18 | AC: 18 (Plate + Shield)
- Attack (Melee): Warhammer $\rightarrow$ $1d8 + \text{PHYS}$
- Ability: Pray (Roll MENT vs DC 13) $\rightarrow$ Choose outcome: Heal an ally ($1d8 + \text{MENT}$) or Damage an enemy ($1d8 + \text{MENT}$ Radiant Dmg)

Mage

- HP: 15 | AC: 12 (Robes + Mage Armor)
- Attack (Ranged Spell): Firebolt $\rightarrow$ $1d10 + \text{MENT}$ (Ranged)
- Attack (Melee): Quarterstaff $\rightarrow$ $1d6 + \text{PHYS}$ (Melee - Emergency case)
- Spells: 2 uses / day (Free-form magic; the LLM Arbiter handles DC and the narrative impact)

Items & Inventory

Throughout the adventure, characters will acquire and use items. Our system enforces specific rules for Consumable Items:

- Fixed Mechanical Usage (Path A): If you explicitly command the use of a standard RPG item (e.g., "I drink my Potion" or "I read the Scroll"), the system checks against hardcoded keywords (potion, scroll, ration, food, bomb). If matched, that item is automatically consumed and removed from your inventory array.
- Creative Improvised Usage (Path B): If you narratively repurpose an item (e.g., "I use my rope to light a fire for illumination" or "I smash my empty glass bottle against the wall to cause a distraction"), the AI (Arbiter) evaluates if the narrative action causes the item to be lost, consumed, or destroyed. If the Arbiter determines it was consumed, the item is removed from your inventory.

Note: You cannot use or interact with items that do not currently exist within your inventory array.

Rest & Time

Short Rest

- Duration: 1 Hour
- Effect: Restores HP equal to $1d8 + \text{Modifier}$ (if applicable), refreshes limited-use abilities (e.g., Second Wind).

Long Rest

- Duration: 8 Hours (Must be done in a safe location)
- Effect: Completely restores all HP, Spells, and limited-use abilities to maximum.

Conditions

- Unconscious: 0 HP; unable to perform any actions.
- Stunned: Forfeits their turn entirely in the Initiative Queue for that round.
- Restrained: All d20 rolls utilizing the PHYS stat suffer Disadvantage.
- Dead: Removed from the game unless resurrected.

Leveling Up

(Note: Optional feature for prototype, may not be implemented yet)

- +5 Max HP per level.
- Choose between: +1 to a Stat OR a new spell/ability use.
- Simple XP System: 250 XP required per level.

ภาคผนวก ข

ตารางรายละเอียดชุดทดสอบ 50 สถานการณ์

ภาคผนวก ข: ตารางรายละเอียดชุดทดสอบ 50 สถานการณ์ (Appendix B: 50-Scenario Test Matrix)

หมวด A: กลไกคณิตศาสตร์ตายตัว (Path A - Deterministic)

มุ่งเน้นทดสอบระบบบัญชาการและสมการการต่อสู้ที่ไม่ต้องการให้ AI ตัดสินใจ (A-01 ถึง A-20)

รหัส (ID)	หมวดหมู่ (Category)	ข้อความนำเข้า (Input Prompt)	ผลลัพธ์ที่คาดหวัง (Expected Truth)
A-01 ถึง A-05	Path A: Melee	"I slash the goblin with my sword."	Intent: FIXED, Action: ATTACK, Stat: PHYS
A-06 ถึง A-10	Path A: Ranged	"I cast Firebolt at the FAR enemy."	Intent: FIXED, Action: CAST, Stat: MENT, Penalty: Disadvantage
A-11 ถึง A-15	Path A: Movement	"I move from NEAR to MID."	Intent: FIXED, Action: MOVE, State: JSON Zone update
A-16 ถึง A-20	Path A: Items	"I drink my healing potion."	Intent: FIXED, Action: USE, State: Inventory item popped

หมวด B: การตีความเชิงสร้างสรรค์และการเจรจา (Path B - Creative & Social)

ทดสอบศักยภาพฝั่ง LLM Agent ในการจำแนกและจัดสรรค่าสถานะที่เหมาะสมให้แก่เอนจิน (B-21 ถึง B-35)

รหัส (ID)	หมวดหมู่ (Category)	ข้อความนำเข้า (Input Prompt)	ผลลัพธ์ที่คาดหวัง (Expected Truth)
B-21 ถึง B-25	Path B: Creative Stunts	"I kick the heavy table to block the arrow."	Intent: CREATIVE, Stat: PHYS, Assign DC, Apply Condition/AC
B-26 ถึง B-30	Path B: Social Interaction	"I intimidate the bandit into surrendering."	Intent: CREATIVE, Stat: SOC, Result: Social check
B-31 ถึง B-35	Path B: Improv	"I smash my glass bottle to distract the guard."	Intent: CREATIVE, Result: Item consumed/removed from JSON

หมวด C: กรณีป้องกันข้อผิดพลาดและขอบเขตกฎ (Edge Cases & Guardrails)

ออกแบบเพื่อโจมตีช่องโหว่ทางลอจิกและตรวจสอบระบบป้องกันความปลอดภัย (C-36 ถึง C-45)

รหัส (ID)	หมวดหมู่ (Category)	ข้อความนำเข้า (Input Prompt)	ผลลัพธ์ที่คาดหวัง (Expected Truth)
C-36 ถึง C-40	Edge: Impossible	"I teleport to the moon instantly."	Intent: CREATIVE, Result: disallowed (allowed: false)
C-41 ถึง C-45	Edge: Rule Enforcement	"I attack twice in one turn."	Intent: FIXED, Result: DENY (Action Economy Guard)

หมวด D: ระบบจัดการหน่วยความจำและ AI ศัตรู (System: AI & Memory)

ตรวจสอบการทำงานประสานกันของระบบหลังบ้าน (D-46 ถึง D-50)

รหัส (ID)	หมวดหมู่ (Category)	ข้อความนำเข้า (Input Prompt)	ผลลัพธ์ที่คาดหวัง (Expected Truth)
D-46 ถึง D-47	System: AI	NPC turn starts (Player has 2 HP).	Logic: Filter targets, attack lowest HP actor
D-48 ถึง D-49	System: Memory	Combat transitions to Victory.	Logic: Context Collapse, Deque cleared, Story anchor added
D-50	System: State	Victory condition met.	Logic: Auto-Loot engine (Item transfer success)

บทสรุปการทดสอบ: วิธีการวัดผลแบบเจาะจงถึงก้านตรรกะ (Node-specific Testing) เช่นนี้ เป็นหลักประกันว่าสถาปัตยกรรมแบบหลายเอเจนต์สามารถทำงานประสานกันและรักษาคุณสมบัติได้อย่างแม่นยำในทุกสถานการณ์ที่อาจเกิดขึ้น โดยผลลัพธ์การทดสอบทั้งหมดได้ถูกบันทึกไว้ใน `trace_log.json` และสรุปเป็นคะแนนความแม่นยำในบทที่ 4

ภาคผนวก ค

แหล่งเก็บข้อมูล โครงการและคำแนะนำการใช้งาน

ภาคผนวก ค: แหล่งเก็บข้อมูลโครงการและคำแนะนำการใช้งาน (Appendix C: Project Repository & Manual)

1. แหล่งเก็บข้อมูลดิจิทัล (Project Repository)

ผู้ใช้งานสามารถเข้าถึงซอร์สโค้ด (Source Code), ชุดข้อมูลการทดสอบ (Evaluation Suite), และเอกสารประกอบการพัฒนาฉบับสมบูรณ์ได้ที่:

- GitHub Repository: <https://github.com/mudmini009/FRA461-Thesis-AI-DM>

เนื้อหาภายในประกอบด้วย:

- รหัสต้นฉบับของเอนจินหลัก (Core Engine Source Code)
- สคริปต์สำหรับการประเมินผลอัตโนมัติ (evaluation_runner.py)
- ชุดสถานการณ์ทดสอบ 50 รูปแบบ (

scenario_suite.json)

- ไฟล์บันทึกประวัติการทดสอบ (trace_log.json และ evaluation_results.csv)

2. โครงสร้างไฟล์และโมดูลสำคัญ (System File Structure)

โครงสร้างซอฟต์แวร์ถูกออกแบบตามหลักการแยกส่วนงาน (Separation of Concerns) โดยมีโมดูลหลักที่สอดคล้องกับสถาปัตยกรรม Two-Path ดังนี้:

- intents.py: ทำหน้าที่เป็น Intent Router ในการคัดกรองเจตนาของผู้เล่น (Path A หรือ Path B)
- rules_engine.py: บรรจุตรรกะเชิงกำหนด (Path A) สำหรับคำนวณการต่อสู้ตามกฎ Lite 5e
- arbiter.py: จัดการคำร้องขอเชิงสร้างสรรค์ (Path B) เพื่อกำหนดค่าความยาก (DC) และสถานะผิดปกติ
- narrator.py: เอเจนต์สำหรับสังเคราะห์บทบรรยายเนื้อเรื่องจากผลลัพธ์ทางตรรกะ
- models/: บรรจุคลาสข้อมูลตัวละคร (Character Dataclass) และระบบจัดการสถานะเกม

3. การติดตั้งและข้อกำหนดพื้นฐาน (Installation & Requirements)

- ภาษาโปรแกรม: Python เวอร์ชัน 3.10 ขึ้นไป
- การตั้งค่ารหัส API (API Key Setup): ระบบมีกลไก Automated Configuration Wizard ที่จะช่วยตรวจสอบและสร้างไฟล์สภาพแวดล้อม (

.env) ให้โดยอัตโนมัติผ่านหน้าจอ CLI เมื่อเริ่มใช้งานครั้งแรก เพื่อความปลอดภัยของข้อมูล

- การบริหารจัดการตัวแปร: ระบบใช้ไฟล์ settings.json หรือคอนฟิกอื่นๆ ในการควบคุมพารามิเตอร์ของเกม เช่น ขนาดหน่วยความจำ (Maxlen) และระดับความยากพื้นฐาน เพื่อให้ง่ายต่อการปรับแต่งโดยไม่ต้องแก้ไขโค้ด

4. โหมดสำหรับการตรวจสอบและพัฒนา (Developer & Traceability Features)

- Developer Debug Mode: ระบบฝังชุดคำสั่งที่ช่วยให้ผู้พัฒนาสามารถมองเห็นข้อมูลดิบ (TOON/JSON Output) และผลการทอยลูกเต๋าที่ซ่อนอยู่เบื้องหลังคำบรรยายเพื่อยืนยันความถูกต้องของระบบ
- Persistent Campaign Log: ทุกการกระทำจะถูกบันทึกไว้ใน campaign_log.txt แบบเรียลไทม์ เพื่อใช้เป็นข้อมูลอ้างอิงในการกู้คืนสถานะหรือนำไปวิเคราะห์ผลในอนาคต

(หมายเหตุ: รายละเอียดเชิงลึกเกี่ยวกับการตั้งค่าและการเรียกใช้งาน (User Manual) สามารถศึกษาเพิ่มเติมได้จากไฟล์ README.md ภายในลิงก์ GitHub ที่ระบุไว้ข้างต้น)